The PCS Programming Language

by Dr. Paul J. Tadrous
27.02.2020 - 14.06.2026

Table of Contents

1 Introduction

PARDUS is the **P**rogrammable **A**ffordable **R**emote-controlled **D**rive **U**tility **S**tandard – an open source system of rules, communications protocols and hardware specifications that may be implemented to allow certain types of robotic systems with sensor feedback to be controlled, locally or over a network connection. It was designed with robotic microscopy in mind as a primary use-case but it can be used for other types of robot as well.

This document is the reference manual for the PARDUS Command Script (PCS) programming language. PCS provides the means to program robotic systems that use the PARDUS hardware and software standards.

Programs written in the PCS programming language are called ‘scripts’ and they are written in plain text files which are conventionally given the extension .pcs. This extension is not a strict requirement but programs that use pcs files may have file load filters that only look for files with a .pcs extension. The use of the .pcs extension here is context specific and should not be confused with other kinds of '.pcs' files such as those used for embroidery machines or video games.

Tasks for the robot and its sensors may be written directly and manually into a .pcs text file by a human programmer. However, it is envisioned that intermediate computer programs might also create properly formatted .pcs files to facilitate more complex tasks by presenting the user with an intuitive graphical interface or a higher level scripting language. This includes, but is not limited to, the use of AI coding agents. At the time of writing the only such non-AI ‘PCS-helper’ tools that exist are those incorporated into the pardclient program and they translate GUI actions into PCS commands to provide a partial GUI interface to control a PARDUS robot. However the language was designed to have several 'AI-friendly' features to enable AI coding agents to efficiently learn and code in PCS. See the dedicated chapter in this manual for a summary of those features (PCS and AI Coding Agents).

The PARDUS system of programs

PCS is an interpreted language, not a compiled language. It is interpreted by programs following the PARDUS standards such as my current implementation which is composed of the following programs:

- pardclient
- pardaemon
- pardserver
- parduino
- pardcap

pardclient

This is a cross-platform GTK4 C program it can be run on can be run on Linux, MS Windows and macOS. pardclient is the main user interface for PARDUS robotics control. pardclient is the

program that reads in PCS scripts, checks and parses the code, puts the code into effect (either locally for local commands or via communication with the other programs) and displays output back to the user. Currently pardclient, has no script editing facility so PCS programmers must compose and edit their scripts in a text editor or IDE like Geany, Kate, NotePad, etc.

In pardclient, prior to interpretation, the script undergoes pre-processing by a preprocessor function and checking for syntax and value range errors by a script checker function. The script checker cannot detect all possible errors that may arise during run-time. For example the values in variables used as arguments for commands may go out of the legal range of those arguments as a consequence of operations that occur during run time. Also, hardware errors may cause an otherwise 'syntax-correct' script to fail with an error at run-time. So the script checker function of the interpreter is only a first line of defence.

pardaemon

This is a cross-platform background process execution program. In the past this was a separate program written specifically for PARDUS. However, in the modern PARDUS implementation this functionality is provided by the GLib 'g_spawn_' set of functions and is therefore built into the pardclient software and so uses the OS' native background process execution systems.

pardserver

This is a Linux program that is the 'behind the scenes' workhorse and coordinator of the PARDUS robotics control system. It does not need to be run on the same computer as pardclient. A user will use pardclient to log in to the pardserver. Only after successful connection to a pardserver will the pardclient interpreter be fully active. You cannot run PCS scripts on the client if you are not logged in to a pardserver. The pardserver connects to the robotics control breakout board and also (optionally) to one or more pardcap servers. The pardserver acts as a 'go between' linking the user at their pardclient station with the robotic actuators and sensors (via the breakout board) and any cameras attached to the system (via the pardcap server(s)).

parduino

This is the software running on the robotics actuator and sensor controlling breakout board which, in the current implementation of PARDUS, is an Arduino Mega 2560 hence the name of the program (the sketch is called parduino.ino). The breakout board communicates with the pardserver via USB serial comms.

pardcap

This is a Linux GTK4 C program otherwise known as PARD capture. Each pardcap instance communicates with an imaging device via v4l2 protocols. While pardcap may be run in full GUI stand alone mode it may also be run in a completely headless server mode and that is the mode we are considering in this document. More than one instance of pardcap can run on the same Linux computer. However, pardcap communicates with the pardserver via TCP/IP where pardcap takes on the role of server and pardserver is client. Thus the pardserver may connect to may pardcap servers

such as to provide multi-camera support to the pardclient. The various pardcap servers may be running on physically separate Linux PCs to the one the pardserver is running on.

Why PARDUS and not some other software?

Some have asked me why did I write the PARDUS system with a whole new programming language (PCS) instead of simply using pre-existing tools and languages to control my robotic microscope, like G-code, either on its own or via software like GRBL, Linux CNC or Klipper.

The answer is that G-code suffers from the limitations of being a mostly ‘one-way’ language with very limited facilities for interactivity and feedback control. G-code is OK to create a pre-programmed path for a tool to follow then ‘send and forget’ those instructions to the robot (usually a CNC machine or 3D printer) to execute.

However, robotic microscopy involves the possibility or interactive control, video feedback and video and other sensor cybernetic feedback to modify the motion of the stage and other actuators in real time. It is also useful to have customised graphical and text-based feedback and the ability to run custom external processes in response to feedback which, in turn, may influence the current motion path of the microscope and other actuators of the robot. G-code simply does not have those abilities. For example G-code can’t run one or more camera systems, process images from those cameras and alter the stage motion path according to analysis results of those images from those cameras. PCS can.

In addition, ***PCS is a Turing complete, fully fledged programming language*** while G-code is not - it is fundamentally just a command language. With PCS you may implement complex computational algorithms closely linked to the robot's actions and sensors in cybernetic feedback loops and with the input of external processes (Kalman filters, machine vision and so forth).

At the other extreme are general robotics operating systems like ROS and ROS2. While these can do all those things I just described, they are very general-purpose systems which go way beyond the needs of many robotic microscopy experiments. They *can* be used for microscopy but PCS was *designed* for microscopy and so is more targetted and lean in that respect.

Finally some readers familiar with research microscopy might be wondering about the possible use of the MicroManager open source software. MicroManager, however, is a front end interface. It has no intrinsic driver control ability for microscopes. It might be possible to make a MicroManager ‘device adapter’ module that interfaces with PARDUS robotic systems, but the PCS level driver (via the PCS interpreter) is ultimately what controls the microscope stage, cameras, etc.

2 General specifications

A script written in PCS is a plain text file (ASCII or UTF-8 encoded).

Each script line must be either a properly formatted coding line or a permitted non-coding line.

Permitted non-coding lines are of one of two types:

1. A blank line containing only white-space characters (this include a line containing only solitary newline character). These will be skipped by the interpreter.
2. A line beginning with a # (hash) that is not immediately followed by a preprocessor directive. These lines may be used for comments and may contain arbitrary text after the # (with the exception of preprocessor directives). These lines will also be skipped by the interpreter.

Comments

Comments are preceded by the hash character (#). Any text after a hash on any line will be ignored by the interpreter unless the characters immediately after the # form a recognised preprocessor directive.

A comment hash may appear as the first non-white space character on a line (in which case the whole line will be skipped) or it may appear after a command on the same line as the command (in which case the interpreter stops reading that line from the hash onwards). This implies that you cannot have any interpretable code *after* a hash symbol until a newline character is encountered (i.e. comments can occur *after* a command but not *within* or *before* a command on the same line).

There is currently no facility to comment off a whole block of code with opening and closing comment block symbols (equivalent to the `/*` and `*/` of the C programming language).

Coding lines

Items on a coding line are separated by white space that is composed of one or more space and/or tab characters.

PCS code is case sensitive.

The very first line in a PCS file must have three items on it (and no more) in the following order:

1. The word Pardus spelt with an initial capital P and the remainder lower case
2. The letters pcs
3. The version number as a single integer.

So, for version 1 pcs files the first line must read:

```
Pardus pcs 1
```

Each coding line after the first must begin with a command or preprocessor directive (optionally preceded by white space).

The command may then be followed by a number of additional coding symbols separated by white space (the term 'coding symbol' refers to any interpretable non-white space word, number or character). The exact syntax and number of any additional coding symbols will be determined by the initial command on that line (this is sometimes called the ***prime command***) according to the defined permitted usage of that command and the syntax rules of the PCS programming language, all of which will be described in detail below.

Only one prime command per line is permitted. One or more secondary commands may be permitted for certain prime commands in which case those secondary commands must be included in the correct sequence on the same line as the prime command. Details will be given of such cases in the detailed description of the prime commands later in this manual. For example, this is legal:

```
set_condexp idx > 20    # Correct - single conditional expression command
loop_while Condexp      # Correct - single loop_while evaluation command
goto mylabel if Condexp # Correct - goto permits a secondary 'if' command
```

but this is illegal:

```
loop_while idx > 20      # Wrong! Attempt to combine 2 commands in one line.
goto mylabel if idx > 20 # Wrong! Attempt to combine 3 commands in one line
                         # when only 2 are permitted.
```

(Note that `Condexp` is a 'system variable' - these will be discussed in more detail later).

The prime command `exit` signifies the end of the script. Any additional code or text in any subsequent lines after the first use of this prime command will be ignored by the interpreter as well as the preprocessor. Thus you may add any free text annotation to your scripts below this line without concern for PCS-compatible rules or formatting. Use the area below `exit` for persistent memory, scratchpad logic, or storing JSON metadata that doesn't need to be parsed by the PCS interpreter.

Line numbers are not used for programming in a PCS script but the interpreter will keep track of which line number each script interpretable line is on so errors can be reported with reference to the source line. Goto targets are defined by line labels, which the user sets (as detailed in the definitions of command usage below), and not by line numbers.

The general rules for white space in a script are these:

- The amount of white space that precedes the first non-white-space character on a line is irrelevant (i.e. indentation position of the command and alignment are irrelevant).
- The amount of white space that follows a command is also irrelevant as long as it does not overflow the read buffer before a newline character is encountered (at the time of writing the line buffer is 1024 characters).

- The amount of white space between arguments is also irrelevant. For example if you separate command arguments by 5 spaces or 1 space it makes not difference. This is particularly relevant to specifying words to add to a string variable - you can't just add more spaces between words by simply separating those words by more spaces in the command. See specific string handling commands for details.

Argument Length and Unicode Limits

The interpreter enforces a limit on the maximum size of any single argument. This is governed by the MAX_ARGLEN value in the source code, which is currently 64 bytes.

Certain commands (such as `#include` or `preview_file`) that handle file names or directory paths allow for longer arguments up to SCNFNAME_MAX bytes. The value of SCNFNAME_MAX is determined by the Operating System's maximum path length at the time of compilation of the pardclient (i.e. the interpreter) program. On MS Windows, this is typically 259 bytes. On Linux/Unix systems, this is typically 4095 bytes. See specific commands for when these exceptional limits apply.

Note: When trying to keep within the MAX_ARGLEN limit, it is important to distinguish between bytes and characters. While standard ASCII characters are 1 byte each, UTF-8 Unicode characters (like emojis or box-drawing symbols) typically consume 3 to 4 bytes each. Example: The "═" symbol uses 3 bytes. A single 64-byte argument can contain a maximum of 21 of these characters before truncation.

> **Caution**: Because the interpreter truncates at the byte level, exceeding the limit with Unicode characters will result in a malformed UTF-8 sequence. This may cause the string to fail to render or appear as "garbage" text in the console.

Note: Just because an argument is generally limited to MAX_ARGLEN bytes, does not mean that every argument can be that long. For example some arguments that specify the name of an actuator (using `def_actuator`) or the custom name of a limit switch (`act_set`) may be limited to fewer characters (31 printable bytes in the two examples mentioned) so be guided by the details of the command descriptions in this manual.

Note: Because indexed arrays must not contain white space, they are treated as a single argument. This allows for powerful nested dereferencing, such as:

```
MyArray[MyIndexList[MyIndex]]
```

With the current 64-byte limit, the entire construction (Base Name + Brackets + Nested Indexes) must not exceed 64 bytes. If the combined length of the names and brackets exceeds this limit, the interpreter will truncate the string. Example: If you try to write this:

```
Very_Long_Array_Variable_Name[Long_Inner_Array_Name[Long_Index_Variable]]
```

it will be trunctated to this upon interpreter reading:

```
Very_Long_Array_Variable_Name[Long_Inner_Array_Name[Long_Index_V
```

and this will result in a Syntax Error because the closing brackets were lost during the read.

Note: At each stage, the interpreter will attempt to perform the script action. If a fatal error is encountered (such as a syntax error caused by truncation), the interpreter will terminate execution and print the error type and source line number to aid debugging.

3 Data types, the ‘varval’ and casting

There are 3 data types in PCS:

1. integer number – referred to as **int**
2. floating point number – referred to as **float**
3. general charter string – referred to as **string**

int

This is a 32 bit signed integer. There is no unsigned integer data type for user-created variables in PCS but some *system variables* which the PCS programmer has read-only access to (see later for the definition of a 'system variable') may be of unsigned int type.

float

This is a double precision floating point value (defined as a `double` in the C source code the PCS interpreter is written in).

string

This is a general character string.

Strings may include UTF-8 Unicode characters.

Strings are not defined between delimiters of any kind (like single or double quotes). If you use delimiters in specifying a string then the delimiter characters will literally be incorporated into the string. There is no concept of an 'escape character' in PCS.

For example, in the following code, the variable `msg` will end up containing 4 characters like this: `["][3]["][\0]` and not simply `[3][\0]`:

```
set_str msg "3"
```

This is important to understand, not just for printing text but because the autocasting algorithm will fail to cast `["][3]["][\0]` to an int when such a cast is intended and required by the script writer because the included quote marks are considered non-numerical characters. However, the autocasting algorithm will succeed in casting `[3][\0]` to an integer.

When setting or defining a string variable with `def_var` and `set_var` you can only use one PCS argument for the value to set into the string variable. So, when using a literal string constant as RHS-value[1] this is limited to a single command line argument of length MAX_ARGLEN (currently

1 In this manual I use the terms RHS-value (meaning Right Hand Side value) and LHS-value (Left Hand Side value) to refer to the left and right values in an expression. So, in the expression 'myval += incval' the variable 'myval' is the LHS-value and the value 'incval' is the RHS-value. In the PCS programming language we would not write that expression in that way. Instead we would write: `add_var myval incval` but the LHS-value and RHS-value will be as just described. These LHS and RHS terms should not be confused for the modern programming language terms L-value and R-value which mean something quite different (although historically related to Left and Right).

64 printable bytes) with no white spaces. However, you can build up bigger strings with the `add_var` command like this:

```
add_var mystring rhsstring
```

To add a space the system variable `Space` is available. Also, to add a hash symbol '#' (which cannot be entered as an argument in a command because the script interpreter treats '#' as the beginning of a comment and so will not read it in as an argument) the system variable `Hash` is available. So if you wanted a space between the existing contents of mystring and rhsstring you can do it like this:

```
add_var mystring Space
add_var mystring rhsstring
```

and if you wanted to have a hash between them you would do it like this:

```
 add_var mystring Hash
 add_var mystring rhsstring
```

This can be a cumbersome way of making larger multi-word strings so the `set_str` and `cat_str` commands are available for that (which see). However those commands do not allow any casting or variable evaluation into strings so a combination of all these commands can be useful for making complex strings.

Permitted operations with strings are `set_var`, `add_var`, `set_str` and `cat_str` and valid comparisons between strings are `==` and `!=`. Other comparisons and operations are not permitted with string varvals when both the LHS-value and RHS-value are a string variable.

String variables can be used as an argent to the `goto` command in lieu of a string constant label so the control of flow can be more dynamic (dependent on how a string variable contents are manipulated) than in other programming languages that only use constant unchangeable line numbers or string constants for goto labels. This can also make analysing the control of flow of the code more difficult in advance of execution.

See below for automatic casting rules and strings. More detail about string handling will be supplied in the sections on the string handling commands and system variables.

varval

The term 'varval' is used in this manual when describing command arguments to mean an argument that may be either a ***var***iable or a constant ***val***ue. A varval is not a data type.

For example the command `print_value` has the following structure:

```
print_value <varval> [format_options]
```

This means if you want to print the character '3', for example, and you have defined a variable called myint that currently holds the value 3, you can either supply the name myint or a literal 3 as the <varval> argument and it will have the same effect.

The use of <varval> for an argument distinguishes it from arguments that require exclusively either a constant value or a variable. For example there are some commands where a literal value will not be allowed, such as those commands that need a receptacle to place some result value in - a value cannot be placed in a literal constant value but only in a writeable variable of the a compatible data type. That receptacle argument therefore cannot be described as a 'varval'.

The binary choice varval

You will see multiple occasions where a command argument is described as '***a binary choice varval***'. This means it can be either a variable or a constant value, of any data type, that can be evaluated to a zero or non-zero value and so convey a binary state back to the command according to the following rules.

If it is a numerical data type (float or int) then its value (whether a constant or contained in a variable) will be taken literally and the command will receive a 1 (conveying the logical meaning of 'ON' or 'TRUE') or a 0 (conveying the logical meaning of 'OFF' or 'FALSE')

If it is a string then its value (whether a string constant or contained in a string variable) will be evaluated as follows:

If the string value is a character representation of a numerical value, that numerical value will be extracted (as if by casting to int or float) and the rules for a numerical data type described above will be followed.

If the string value is not a legitimate numerical representation then it dealt with as follows:

if the string is a complete match for the word "TRUE" or the word "ON' (case sensitive and without the quotes) then the command will receive a 1 (conveying the logical meaning of 'ON' or 'TRUE') .

If the string is a complete match for the word "FALSE" or the word "OFF" (case sensitive and without the quotes) then the command will receive a 0 (conveying the logical meaning of 'OFF' or 'FALSE').

If the string contains anything else, this will result in either a syntax error (during script checking) or a run-time error.

casting

In PCS, casting is done (or at least attempted) automatically by the interpreter as appropriate to the context. While there is no specific 'casting syntax' to allow the user to directly cast a variable or value into a particular type, the overall effect of a user-deliberate cast can be achieved in several steps as demonstrated in the code snippets shown later in this section.

As an example of PCS auto-casting, in many cases that require a floating point number as an argument, if you supply a string variable that contains a string which is a properly formatted floating point number, then the value of that number will automatically be used by the interpreter (akin to use of the C function atof). Likewise if the result of some command is an integer, the variable argument of that command that accepts the result will be set to that integer value even if that variable is of float type. An error will only be raised if casting is not possible. For example, use

of a string variable as an argument that needs to be a numerical value but the contents of the string is not a properly formatted number. Some general rules follow. For more specifics about how casting works see the dedicated sections on the commands `act_get` and `math_var`.

Rules for integer and float Casting:

For all commands that require a varval of type `float` you may also use a varval of type `int` and the integer value will automatically be converted to a float value.

For some (but not all) command arguments that expect integer values, if you supply a floating point value this will be cast to an integer using the 0.5 round-off rule i.e. int = (int)(float+0.5) if float is >0.0 and int = (int)(float-0.5) if float is <0.0. If the floating point number exceeds the range of an integer then an error will result and the command will fail. This automatic conversion is done in all of the variable manipulation commands (i.e. commands ending in `_var`).

For command arguments that expect a floating point number, if you supply an integer this will be directly cast as a floating point number so if you supply 34 it will be treated as 34.0. This should never result in an error (obviously an error may still occur if you supply an illegal value such as 0 for a denominator in a divide-by operation).

For variable manipulations (e.g. addition, subtraction, multiplication and division) the value supplied as the value to operate with will be converted to the same type as the variable to be operated upon. So if you try to multiply an integer varval by a floating point varval the floating point value will be rounded to an integer to do the manipulation (the actual floating point varval will not itself be altered but the value it supplies for this specific purpose will be converted prior to being used for that purpose). As a specific example, if you have an integer variable with value 38000 and you want to multiply it by 1.25 to get the result 47500, if you try this directly you will just get 38000 because the 1.25 will be rounded to the nearest integer i.e. 1 :

```
def_var int ivar1 38000
mul_var ivar1 1.25
print_value ivar   # This will show a result of 38000
```

The correct way to do this is via casting using an intermediate variable of float type:

```
def_var int ivar1 38000
def_var float fvar1 ivar1   # If fvar1 was previously defined
                            # then just use 'set_var fvar1
                            # ivar1' to do the cast
mul_var fvar1 1.25
set_var ivar1 fvar1
print_value ivar   # This will show a result of  47500
```

Rules for string casting:

When the interpreter evaluates an argument (using its internal var_evalvv function), any argument that is not the name of an already defined user variable or a standard system variable will be treated as a string constant value (not a string variable).

The permitted operations that have a string varval as the LHS-value of that operation can use other (non-string) data type values for the RHS-value and they will be cast into a string for the sole purpose of that operation (without affecting their intrinsic data type if they are variables).

For operations and comparisons where the LHS-value is not a string but the RHS-value is, the RHS-value string will be attempted to be cast to a compatible value. For example if the string variable holds a valid float number like "5.431"[2] then this will be cast to a float (or rounded to an int where and int value is needed). If the string variable cannot be cast because it does not represent a valid number, for example "micrometers", then an error will be thrown.

When you set a string variable with an int or a float then the string variable will take the ASCII / UTF-8 form of that numerical value. For example:

```
def_var float f1 5.431

def_var string str1 f1
```

At the end of the above commands the variable str1 will contain "5.431" (without the quotes).

In the source code of the interpreter this 'casting a number to a string' is done using sprintf like this: `sprintf(tmpstr,"%g",f1)`. This cast is automatic and imperative whenever a numerical value is required by the context - you don't get to chose whether it happens or not. Such casting happens in the opposite direction to that described above so:

```
def_var string str1 5.431

def_var float f1 str1
```

will make f1 have the actual numerical value of 5.431 because the string would have been converted to an actual float (double) in the interpreter program using C code like `sscanf(str1,"%lg",&f1)`. However this means you will get the following behaviour which might not seem intuitive at first:

```
def_var string str1 5,431

def_var float f1 str1
```

Note the use of the comma instead of a decimal point. At the end of these two commands, f1=5 (not 5431 or 5.431). This is because the interpreter's internal `sscanf(str1,"%lg",&f1)` code will convert the first number it comes to which is in the form of a standard float number. The comma here will act as a delimiter to separate 5 from 431.

2 The quotes here are just for emphasising that this is a string to the reader of this manual. Recall that quotes are not used to define strings in actual PCS variables and PCS string constants.

Because a non-string variable can be used as an RHS-value in the `set_var` and `add_var` commands that set or add to a string variable, this gives the user way of incorporating numerical values into strings similar to how the C language function itoa and ftoa work or a rudimentary form of the C language printf formatting specifiers %d %g and %f but without their rich set of formatting options. For example this sequence of commands:

```
add_var mystring Space
add_var mystring =
add_var mystring Space
add_var mystring myint
```

can be used to construct a string for printing. For example if `mystring` was initially "The_mean_value" before reaching this sequence of commands and if `myint` had a value of 356 then the result of the above 4 add_var operations would be to make `mystring` have a value of: "The_mean_value = 356" (without the quotes).

In situations where the value in a string variable cannot be cast to a number it will default to type string (that is it will be treated as a literal string). So if the command that uses the variable requires a number this will cause an error but it the command doesn't care (like `print_value`) then it will simply use the literal string.

4 Actuators and actuator types

Since PARDUS is a robotics control system, the PCS programming language has specific features to allow for defining various types of physical actuators like motors, pumps, and so on. Different actuators have different abilities. Some might only be able to be switched on or off with no intermediate states. Others, like a stepper motor, may have multiple discrete states like 'enabled', 'disabled', 'direction', 'step' and so on. Other actuators may have continuous states like a lamp or heating element or fan that may not only be 'on' or 'off' but may have a continuously varying property like 'intensity'.

Because different actuators have different abilities, the PCS programming language abstracts this to the programmer by means of the concept of an ***actuator type***. This is the physical world analogy to the 'data type' of a variable or constant (which, to continue the analogy, may be seen as an 'information actuator' in this general scheme).

The actuator types are described below. In the very earliest versions of the PCS interpreter only the 'Stepper' actuator was programmed but the other types will be implemented as they are required in future versions. The code infrastructure for them already exists even in the earliest PCS interpreter version.

The following are 'actuator type' constants as defined in the interpreter source code:

```
#define ACT_STEPPER  1
#define ACT_BINARY   2
#define ACT_MLEVEL   3
#define ACT_SERVO    4
```

In a PCS script the user must refer to them by the integer number but you can use an include these same #defines in your PCS script (or in an include file for your script) so that you may refer to them by these names.

Each of the above specifies a category of actuator and in each category there may be none, one or more specific actuators of that type and each such actuator instance will be identified by a serial number integer which is the actuator 'ID'.

Each type of actuator type will have its own list of properties that a PCS script writer may access. Some of these may be read-only and others read-write for the script writer. These properties will usually be realised in the interpreter source code as members of a structure.

Here are what the actuator type categories mean:

Stepper

A stepper motor (open or closed loop but not servo-controlled). The first 3 steppers defined in a script will, by definition / convention, be the stage XYZ motors and will have the following ID numbers:

0 = the X stepper motor

1 = the Y stepper motor

2 = the Z stepper motor

In addition, two more stepper ID numbers will be reserved by the interpreter for specific combinations of stage steppers. These ID number will not refer to any additional individual stepper motors. They are:

254 = X and Y stepper motors together

255 = All three stepper motors together

Using 254 and 255 for the special group IDs of the stage steppers means the interpreter can allocate consecutive serial numbers from 0 to 253 to handle up to 254 steppers defined by the PCS script writer. From the perspective of the PCS script writer, use of 254 and 255 as a stepper ID in a script will only be legal if the X and Y (for 254) or all three stage motors (for 255) have already been previously defined.

Binary

These can only be switched on or off and have no intermediate state or 'amount' of operation. Some examples are:

- A solenoid valve to control on/off filters or fluid lines
- An on/off device like a heater or cooler or UV lamp, etc.
- A trigger signal for some action. This may be an 'activate' signal for a sensor that does not continuously gather data but only when told to do so (like a camera).

M-Level

These operate at some level between off and some maximum degree of operation as opposed to just 'on'. That is, they are multilevel (M Level) actuators. Some examples are:

- An analogue motor with speed settings to control the speed of fans or pumps or spindles.
- A lamp or heater that has intensity control (not just on/off)

Servo

A multi-position actuator that travels to one of several precise positions such as a filter wheel or objective changer. In these cases the actuator provides some feedback on its position that is automatically handled in a servo feedback loop without needing the PC to tell it when to 'stop'.

5 The PCS preprocessor

When a PCS script is loaded into the interpreter program (such as pardclient) it undergoes preprocessing before it is syntax checked. The preprocessor function does two things:

It includes any external files the script requests via `#include` directives.

It makes any string substitutions the script requests via `#define` directives.

For version 1 of PCS that is all the preprocessor does.

To be a valid directive there must:

- be no non-white space characters before the '#'
- be no white space characters between the '#' and the rest of the directive
- be at least one white-space after the end of the directive before its argument
- be the required number of arguments after the directive.

Using a '#' in your PCS file without following the first 3 rules means the '#' will be interpreted as a commend. Not following the last rule will result in a script check error.

The preprocessor analyses the script file that is loaded and outputs its resulting pre-processed version of that script to a file called _pppcs.pcs so do not try to load that file directly into the client or a file access error may result.

You may read the _pppcs.pcs file in a separate text editor if you would like to see the results of pre-processing on your input file. The actual PCS file on disc that you load into the client program is not altered itself. After successful pre-processing, the script checker loads the _pppcs.pcs file to check it and the script interpreter runs the commands in the _pppcs.pcs file.

Although superficially resembling preprocessor directives of the C programming language PCS preprocessor directives behave differently in that only file include and simple string substitutions are supported and the directives all have global scope no matter where, in the script, they occur.

The #define directive

Usage:

`#define <find_string> <replace_string>`

Description:

The `#define` allows the programmer to make string substitutions in their script.

Arguments:

`<find_string>` - a character string, without white space, to be replaced in the commands.

`<replace_string>` - a character string, without white space, to replace the `<find_string>` with when it is encountered in the commands of the script.

Note: Only simple string substitution is supported in version 1 of PCS. There is no facility to use arguments in parentheses or substitute in multi-word expressions.

The main use is to allow the script writer to use human-meaningful words in their script where numerical values are required by the main PCS syntax. For example to use the word 'ACT_STEPPER' in place of the numerical value '1' by means of the preprocessor directive:

```
#define ACT_STEPPER  1
```

Without such a preprocessor string substitution ability, the only other way to use human meaningful labels for numbers would be to define a variable with the human-friendly name and set its value to the number value but doing so would incur a cost in terms of RAM allocation and variable evaluation processing time every time that variable is used in the script at run-time.

It is convention to use all capitals for the `<replace_string>` to distinguish them from the names of variables or PCS commands but it is not mandatory.

Neither argument is expected to be encased in quote marks - if quote marks are used they will be interpreted as part of the literal string to be found or replaced.

The #include directive and header files

Usage:

`#include <filename>`

Description:

The `#include` directive allows the programmer to include a PCS header file in their script.

Arguments:

`<filename>` - the name or full path and name of a file containing more pre-processor directives.

Note: The `<filename>` must not have any spaces in its name or in a path and the angled brackets are not part of the syntax (unlike in C). So an example is:

`#include myfile.pch`

Any file used for inclusion must have a properly formatted header on line 1 containing only 2 words followed by one integer number and nothing else (other than comment coming after these three words or any preceding white space, all of which, will be ignored by the line parser). The two words are, in order and case sensitive: Pardus pch

Currently the integer number (which is the file format version number) is 1, so a complete first line for an acceptable include file is this:

`Pardus pch 1`

After this can come comments, blank lines and other text, as with a PCS script. Also, as with a PCS script, leading and trailing white space and blank lines are ignored.

The header file must be terminated with an `exit` command, just as for a PCS script, and all text after the `exit` line is ignored.

Unlike a PCS script file, any text before the `exit` which is not a preprocessor directive is also ignored, regardless of formatting or content, because this preprocessor does not look for acceptable PCS commands but only for preprocessor directives - it ignores anything else.

By convention, header files have the extension .pch but this is not mandatory for proper reading and parsing of the files.

6 Variables and arrays

There are two classes of variables in PCS, user-defined variables and system variables.

All variables have global scope.

All variables are arrays (single variables are just arrays of size 1)

The names of variables are case sensitive.

The possible data types for variables are int, float, and string. These are described in more detail, together with the rules on casting, in the chapter 'Data types, the ‘varval’ and casting'.

User-defined variables

These are created with the `def_var` and `def_array` commands and, once created, they may be destroyed using the `undef_var` command. All user-defined variables will be destroyed automatically when the script terminates for any reason. User-defined variables are read-write capable. Because all variables are arrays, when the term 'variable' is used in this manual it automatically also refers to 'array'.

Dynamic memory allocation and scope

The size of an array may be specified by another variable at the time the array is created with `def_array` and does not need to be pre-determined before the script is run. This allows for dynamic array size allocation in PCS scripts. The size of an array may also be changed during run-time using a `def_array -> undef_var -> def_array` sequence. See the detailed description of `def_array` for more information.

All variables have **global scope** but may only be used from the point in the script at which they are defined and they may not be used (as variables[3]) from the point in the script at which they are undefined (if they are ever undefined). This gives the PCS programmer the ability to manually determine the **temporal scope** of a variable. See `undef_var` for more detail.

Limitations imposed by the syntax checker on variable use

There are 3 rules imposed by the PCS syntax checker that you must be aware of to avoid syntax errors or run-time behaviour you might not otherwise have expected.

1. The use of dereferencing brackets: The PCS syntax rules make a distinction between scalar variables (that is, arrays of size 1) and vector variables (that is, arrays of size >1). All scalars must be used with their base name only and no de-referencing brackets. All vectors must be used with

3 If you attempt to use a variable after it is destroyed then the variable evaluation process will search for the value of that variable, it will see that the string you supplied does not exist in the list of extant variables and so it will treat it as a string literal. If that string literal is compatible with the attempted use then no error will be emitted but your program will not behave as you expect. This type of bug may be hard to identify so take care to avoid it in the first place by careful use of the `undef_var` command during coding, paying attention to the details of interpreter rules as described in this manual.

dereferencing brackets with the sole exceptions of use in the commands `def_array` and `undef_var` where only their base name must be supplied.

2. Limitations on the temporal scope of a variable: The interpreter's syntax checker goes through the script *line-by-line sequentially from the start* of the file to the first use of the `exit` command regardless of any conditional branching or looping rules. As soon as it sees a `def_var` or `def_array` command it will attempt to create the variable. As soon as it sees an `undef_var` command it will attempt to destroy that variable and forbid is use as a variable from that line on unless or until the variable is re-created with an other def_var or `def_array` command. Any attempt to use a variable name (with or without a dereferencing bracket component) that has been destroyed will be treated as though you are supplying a string literal, not a variable, and this may not result in an error but in behaviour you might not expect during run time. This means that if you place a `def_array` or an `undef_array` inside a conditional block, like a `do_if` block or a function that may or may not be called, then those variable creation and deletion events will be registered by the script checker when it comes to that line in the script regardless of any conditional code that would block that line from being executed during run time.

3. The checker only uses the initialisation value of a variable: Because the syntax checker executes the actual creation and deletion of variables as it comes across them, and because the syntax checker does not perform any operation that may change the values of variables already created, it means that if you use a variable as an argument to some other command (including as a size argument to a `def_array` command) then the value you supplied as the initialising value at the the time of `def_var` or `def_array` will be the value used by the syntax checker whenever that variable is used. For this reason, make sure you supply an initial value that will not cause a syntax error wherever that variable is used. Before actually running the script, all variables created during the syntax checking phase will be totally deleted so this will not affect any calculations your script makes during run time (e.g. for dynamic allocation of array sizes). The type of error that may occur at the syntax check phase due to improper initialisation value selection is illustrated in the following examples:

a) In this example you will get a syntax error because you initialised the `sz` integer to 0 and you cannot use 0 as the value of size for defining an array:

```
def_var sz 0

def_var mp 10

add_var sz mp           # This will not be performed by the syntax checker so

def_array myarray sz 0  # sz will remain 0 hence syntax check will fail here.
```

b) In this example you will get a syntax error because you initialised `sval` to '_' and '_' cannot be cast to an integer:

```
def_var sval _

set_str sval 10          # This will not be performed by the syntax checker so

def_array myarray sval 0 # sval will remain '_' and the check will fail here.
```

The correct thing to do in example a) would be to define `sz` with a value >= 1 such as '`def_var sz 1`' and in example b) define `svar` to a value that can be cast to an integer such as '`def_var sval 1`'.

System variables

These are a set of variables that always exist and the script writer has read-access to. Some of these variables can also be written to by the user but only via specific commands. Other system variables are strictly read only for the PCS script writer. There follows a list of all current system variables with a description of their contents and / or purpose.

Name	Type	Description
Space	string	Contains a single space character ' '. It is for adding spaces to strings during concatenation with the add_var command or creation with the def_var or setting with set_var or comparison operations.
Hash	string	Contains a single hash character '#'. It is for adding hashes to strings during concatenation with the add_var command or creation with the def_var or setting with set_var or comparison operations.
DateTime	string	This is a string that is updated whenever it is evaluated and contains a time-date string up to the nearest second of the format: YearMonthDay_HourMinuteSecond for example 20260303_124911
Condexp	int	This value of 0 or 1. It is set to have a value of 0 by default at the start of execution of a script. The value of Condexp may be set or changed by means of issuing the 'set_condexp' command in a script (which see). There is no other way for a user to set the value of Condexp. The value of Condexp is retained throughout the running of a script until it is set or changed by a set_condexp command. If you want to use the current value of Condexp even after another set_condexp command is run then you must set a user-defined variable to the current value of Condexp before calling the next set_condexp command.
Sysval	int	This may take on any integer value. This is set to have a value of 0 by default. It is potentially changed any time the 'int system(const char *command)' function is run because it takes the return value from this function (so may change whenever the command 'execute_system' is processed, for example). The user cannot alter this variable at will but can copy its current value to any varval that accepts integer values.
TRUE	int	This holds a numerical value of 1
FALSE	int	This holds a numerical value of 0.
ON	int	This holds a numerical value of 1.
OFF	int	This holds a numerical value of 0.

NoTissue	int	This may have an integer value of 0 or 1. It is set to have a value of 1 by default at the start of execution of a script. Whenever a grab_image command or 'get_preview_tc ON' command is executed and tissue-content thresholding is used (see tc_threshold), if the tissue-content threshold test failed then the NoTissue variable will be set to 1 (and an image will not be returned in the case of the grab_image command). It will retain this value until a grab_image does result in an image being taken - or a 'get_preview_tc ON' command returns a tissue content value that passes the threshold test - at which point it will be set to 0.
HasTissue	int	This is a convenience variable that will have the complementary value to NoTissue (it will be 1 when NoTissue is 0 and vice versa).
Ps_SatL_R	int	After a get_preview_stats command this reports the number of undersaturated pixels in the red channel of a colour image (or in a monochrome image).
Ps_SatL_G	int	After a get_preview_stats command this reports the number of undersaturated pixels in the green channel of a colour image.
Ps_SatL_B	int	After a get_preview_stats command this reports the number of undersaturated pixels in the blue channel of a colour image.
Ps_SatU_R	int	After a get_preview_stats command this reports the number of oversaturated pixels in the red channel of a colour image (or in a monochrome image).
Ps_SatU_G	int	After a get_preview_stats command this reports the number of oversaturated pixels in the green channel of a colour image.
Ps_SatU_B	int	After a get_preview_stats command this reports the number of oversaturated pixels in the blue channel of a colour image.
Ps_Min_R	int	After a get_preview_stats command this reports the minimum pixel value in the red channel of a colour image (or in a monochrome image).
Ps_Min_G	int	After a get_preview_stats command this reports the minimum pixel value in the green channel of a colour image.
Ps_Min_B	int	After a get_preview_stats command this reports the minimum pixel value in the blue channel of a colour image.
Ps_Max_R	int	After a get_preview_stats command this reports the maximum pixel value in the red channel of a colour image (or in a monochrome image).
Ps_Max_G	int	After a get_preview_stats command this reports the maximum pixel value in the green channel of a colour image.
Ps_Max_B	int	After a get_preview_stats command this reports the maximum pixel value in the blue channel of a colour image.
NSteppers	int	After a get_actuators command this reports the number of stepper motors currently defined on the server.
NBinaries	int	After a get_actuators command this reports the number of binary actuators currently defined on the server.
NMlevels	int	After a get_actuators command this reports the number of M-level actuators currently defined on the server.
NServos	int	After a get_actuators command this reports the number of servos currently defined on the server.

CurrActIdx	int	This reports the index number of the currently selected actuator. It is updated upon successful completion of a def_actuator command or a act_getid command.
CurrActType	int	This reports the actuator type number of the currently selected actuator. It is updated upon successful completion of a def_actuator command or a act_getid command.
NReads	int	This reports the result of any call to the command fset_var. The default value at the start of any script is -4.
NWrites	int	This reports the result of any call to the command fput_var. The default value at the start of any script is -4.
NewLines	int	This reports the number of logical line endings encountered on the input stream immediately following a string-based fset_var operation. The default value is 0.
NBytes	int	This reports the total number of raw bytes consumed from the file during the "reading phase" of the last fset_var command and the total number of bytes written to a file by the fput_var command.
Millis	int	This is a rolling millisecond counter and is updated whenever it is used. Although it is an int (signed) it will never be negative because it rolls over to 0 after 2147483647 (about 24 days) so effectively acts as a 'uint31_t'.
PMRep1	float	The last known step count (in atomic steps) of the X axis stepper motor.
PMRep2	float	The last known step count (in atomic steps) of the Y axis stepper motor.
PMRep3	float	The last known step count (in atomic steps) of the Z axis stepper motor.
PMRep4	float	The last known step count (in atomic steps) of the last moved stepper motor.
Ps_Integral_R	float	After a get_preview_stats command this reports the sum of all pixel values in the red channel of a colour image (or in a monochrome image).
Ps_Integral_G	float	After a get_preview_stats command this reports the sum of all pixel values in the green channel of a colour image.
Ps_Integral_B	float	After a get_preview_stats command this reports the sum of all pixel values in the blue channel of a colour image.
Ps_Var_R	float	After a get_preview_stats command this reports the variance of all pixel values in the red channel of a colour image (or in a monochrome image).
Ps_Var_G	float	After a get_preview_stats command this reports the variance of all pixel values in the green channel of a colour image.
Ps_Var_B	float	After a get_preview_stats command this reports the variance of all pixel values in the blue channel of a colour image.
Ps_Mean_R	float	After a get_preview_stats command this reports the mean of all pixel values in the red channel of a colour image (or in a monochrome image).
Ps_Mean_G	float	After a get_preview_stats command this reports the mean of all pixel values in the green channel of a colour image.
Ps_Mean_B	float	After a get_preview_stats command this reports the mean of all pixel values in the blue channel of a colour image.

Ps_DCT_R	float	After a get_preview_stats command this reports the mid-frequency discrete cosine transform power of all pixel values in the red channel of a colour image (or in a monochrome image).
Ps_DCT_G	float	After a get_preview_stats command this reports the mid-frequency discrete cosine transform power of all pixel values in the green channel of a colour image.
Ps_DCT_B	float	After a get_preview_stats command this reports the mid-frequency discrete cosine transform power of all pixel values in the blue channel of a colour image.
Ps_MAL_R	float	After a get_preview_stats command this reports the mean absolute Laplacian of all pixel values in the red channel of a colour image (or in a monochrome image).
Ps_MAL_G	float	After a get_preview_stats command this reports the mean absolute Laplacian of all pixel values in the green channel of a colour image.
Ps_MAL_B	float	After a get_preview_stats command this reports the mean absolute Laplacian of all pixel values in the blue channel of a colour image.
Ps_Focus_R	float	After a get_preview_stats command this reports the value of the chosen focus statistic of all pixel values in the red channel of a colour image (or in a monochrome image).
Ps_Focus_G	float	After a get_preview_stats command this reports the value of the chosen focus statistic of all pixel values in the green channel of a colour image.
Ps_Focus_B	float	After a get_preview_stats command this reports the value of the chosen focus statistic of all pixel values in the blue channel of a colour image.
Ps_Npix_R	float	After a get_preview_stats command this reports the number of pixels used in the mean and variance calculations for pixels in the red channel of a colour image (or in a monochrome image).
Ps_Npix_G	float	After a get_preview_stats command this reports the number of pixels used in the mean and variance calculations for pixel values in the green channel of a colour image.
Ps_Npix_B	float	After a get_preview_stats command this reports number of pixels used in the mean and variance calculations for pixel values in the blue channel of a colour image.
Ps_Msupport	float	After a get_preview_stats command this reports the number of pixels in the region of support if a preview mask is applied. Note that other statistics values described above as pertaining to 'all pixel values' will actually be for all pixel values within the region of support of any applied mask.
CurrTC	float	The tissue content value of the image. This is updated with each call to 'get_preview_tc ON'

Arrays

Only 1-dimensional arrays are allowed in PCS. In the PCS interpreter all variables are treated as arrays. If you define a variable as an array of size 1 using the `def_array` command this is entirely equivalent to if you used the `def_var` command instead. Thus, `def_var` is redundant. and is provided purely for convenience as a 'programming style' choice for the PCS programmer. Using `def_var` makes it clear to anyone reading the script that this is a single variable (a scalar) and so

cannot be indexed with square brackets. For more details see the dedicated descriptions of the `def_var` and `def_array` commands.

Protected names

When a user defines a variable they can select the name they want to give it but certain names are forbidden - these are the 'protected names'. In addition to the names of all the system variables, the following are also protected.

The system symbol `CurrAct`. This may be used in any command that requires the argument `<act_id>` which means the unique string name that defines an actuator. Using `CurrAct` in such cases will cause the interpreter to use the values of the two system variables `CurrActIdx` and `CurrActType` as the actuator index and the actuator type arguments for that command call.

The system symbol `FITS` (case sensitive) is reserved for use as an argument in some imaging commands so cannot be used as a name for a user-defined variable even though it is not a variable in itself.

The system symbols that refer to combined stage motors are also protected names. These are `StageXY` and `StageXYZ`.

More will be said about these system symbols and the system variables in the detailed descriptions of the PCS commands elsewhere in this document.

7 Functions

In version 1 of PCS there are two ways to implement a function-like subroutine to avoid repeating code blocks. One uses the ***formal function*** definition and calling system. The other uses non-local `goto` jumps. The 'functions' implemented by that method are sometimes referred to as ***subroutines*** to distinguish them from formal functions. This chapter deals with formal functions. The `goto` method will be described in the section detailing the `goto` command.

Defining a function: `def_func` and `return`

The code for a function begins with a `def_func` command that tells the interpreter where the function block begins and assigns a name (its 'label') to the function.

```
def_func <function_label>
```

Each `def_func` command must be followed at some point by one - and only one - `return` command. The return command has no arguments; it defines the end of the function block of code.

There is no concept of 'declaring' a function separate to defining it. Functions may be used (called) before they are defined in a script.

Function definitions cannot be nested - you cannot have a `def_func` command following another `def_func` command without a return command in between the two `def_func` commands.

The function label `main` is special - see section on 'main' below. All non-main function must be defined after the `main` function block. All function labels must not contain white space and you cannot have duplicate labels. For `def_func` the <function_label> argument must be a hard-coded string, not a variable. There is no other restriction for naming functions.

Calling a function: `call_func`

Functions are invoked with the `call_func` command:

```
call_func <function_label> [if <varval>]
```

Functions may be called recursively (so there is also the risk of a stack overflow).

Note that the `<function_label>` argument can be a string variable meaning that the same `call_func` command in a script can be used to call different functions if the script sets the string variable argument to different values (different function labels) according to some algorithm. That is, `<function_label>` does not have to be a hard-coded string constant.

Note the option of making a function call conditional on the value of a varval being non zero (similar to the extended `goto` command). This way you could shorten PCS command sequences. For example you could re-write this:

```
act_getid X_Axis

set_condexp CurrActIdx >= 0
```

```
do_if Condexp
  act_halt CurrAct
  act_set CurrAct INFO_ENA_STATE 0
enddo

act_getid Y_Axis
set_condexp CurrActIdx >= 0
do_if Condexp
  act_halt CurrAct
  act_set CurrAct INFO_ENA_STATE 0
enddo

act_getid Z_Axis
set_condexp CurrActIdx >= 0
do_if Condexp
  act_halt CurrAct
  act_set CurrAct INFO_ENA_STATE 0
enddo
```

to just this (by also functionalising the halt and disable code into the function `halt_disable`):

```
act_getid X_Axis
set_condexp CurrActIdx >= 0
call_func halt_disable if Condexp

act_getid Y_Axis
set_condexp CurrActIdx >= 0
call_func halt_disable if Condexp

act_getid Z_Axis
set_condexp CurrActIdx >= 0
```

```
call_func halt_disable if Condexp
```

Function arguments

An important difference compared to functions in other languages is that PCS has no facility for local or automatic variables - all variables in a PCS script are global. Thus you cannot pass arguments to function in the same way you can for other languages.

The way this is handled in PCS is that if you require 'pass-by-value' arguments then you set aside some user-defined variables for use as argument-substitutes for your functions. Then, when the time comes to call your function you do the local copying of values to those argument-substitute variables and ensure your function uses those argument-substitute variables in its processing.

Return values

PCS functions do not return anything. They are essentially just re-usable blocks of code. If you want to use the results of their processes on a variable, for example, then you should make the function put its result in some variable and, because all variables are global, you can then use that result either directly (by using the variable) or you copy the result to some other variable for use when the function returns.

main

Unlike some languages (for example, C) there is no mandatory requirement for a 'main' function entry point and, in fact, no mandatory requirement for the use of any functions at all in a PCS script.

However, if you do choose to define even one formal function then you must also define `main`. You can choose to define `main` alone, without any other function if you want to. This will give you the option of defining other functions at some later time. If you do not define `main` (and, by extension therefore, you do not define any functions at all) then the script will simply execute each line it encounters following whatever control-of-flow logic you have until it encounters the exit command.

The `main` function is the only one where the name of the function is strictly defined - it must be called main.

If you define `main` then this must be the very first function to be defined in your script.

Any other function may have a name of your choosing but it must be unique (you can't have two or more functions with the same name).

The `return` command from `main` is equivalent to the `exit` command - the script will terminate upon enacting this `return` command.

8 Code blocks, loops and control of flow

In PCS, code blocks are defined by specific pairs of commands only - not by any kind of brackets and not by indentation level. The command pairs that define a block of code are the following:

The function block:

```
def_func
return
```

The while loop block

```
loop_while
endloop
```

The 'if' (with optional 'else') block(s):

```
do_if
else
enddo
```

In this `if` construction there are three options of code block, the `do_if ... enddo`, the `do_if ... else` and the `else ... enddo` blocks.

Control of flow

This is achieved in the form of:

- Sequential flow to the next script line (the default)
- The evaluation of the conditions associated with the `do_while` or `do_if` commands.
- The presence of an `else` command in a `do_if` block
- The value of the function label argument in a `call_func` command and its optional conditional form: `call_func <function_label> if <condition>`. Recall that, unlike the `def_func` command, the `<function_label>` argument in `call_func` can be a string variable. Therefore by including logic that alters the contents of that string variable the program may call a different function each time it visits that same call_func line in a script.
- The `goto` command and its optional conditional form: `goto <label> if <condition>`
- The `return` command from the main function initiates controlled exit from the script

- The `exit` command initiates controlled exit from the script. This may be reached as a prime command in the script or as a failure option in the `autofocus` command if the programmer has selected that option via `af_setup`.
- The use of conditional expressions with the `set_condexp` command and `Condexp` system variable.
- Details of the above commands can be found in the chapters 'Functions' and 'List of commands'.

9 PCS and AI Coding Agents

PCS was designed at a time when AI coding assistants were becoming commonly available and widely adopted. For this reason PCS was designed to be 'AI-friendly' in that is has certain features built into the language that make it easy and efficient for AI agents to learn, to debug and to code in.

While PCS remains fully accessible to human programmers, certain design choices prioritise the way Large Language Models (LLMs) and automated planners process logic, tokenise syntax, and manage state.

The AI-friendly design principles of PCS

The following summarise the core principles of the language which are of particular value to AI coding agents.

1. Atomic and Predictable Syntax

PCS avoids nested functional complexity (e.g., x = func1(func2(y))). Instead, it uses a flat, instruction-based format where each line performs a single, verifiable operation. This reduces "token attention" drift and ensures the AI can maintain perfect syntactic accuracy over long scripts.

2. Operator Abstraction (Dynamic Dispatch)

In PCS, logical and mathematical operators (like ==, >, sin, or pow) are often treated as data tokens or variables. An AI can store the argument of a `goto` command, a function name in a `call_func` command or a comparison symbol in a string variable (including an array of strings) and "decide" at runtime which operation to perform. This allows for highly compact, meta-programmable logic that bypasses the need for massive if/else trees.

The use of this feature in the `goto` command as well as the `call_func` command means the AI agent is free from the restrictions of using only run-time string constants (a restriction found in many other languages like C). This enables '***first-class control flow***'. By treating control flow elements (labels, functions, and operators) as first-class variables, PCS allows AI agents to architect flexible, non-linear workflows. This removes the brittleness of hard-coded logic and empowers the script to adapt its own structure in response to environmental data. Such ***meta-programming*** allows for the use of a '***Jump Table***' approach that is more efficient for LLMs to manage than complex branching logic.

Here is a skeleton template example for PCS meta-programming:

```
Pardus pcs 1
verbosity 1


def_array string CompOp 6 ==
```

```
def_var int CompIdx 0
def_array string FuncName 100 func_1
def_var int FnIdx 0
def_array string GotoLabel 20 exit_label
def_var int GtIdx 0
def_var float fvar1 0.0
def_var float fvar2 0.0
def_var int ivar 0
def_var int ivar 0

def_func main

  # Populate the comparison operations array
  set_var CompOp[0] ==
  set_var CompOp[1] !=
  set_var CompOp[2] >=
  set_var CompOp[3] >
  set_var CompOp[4] <=
  set_var CompOp[5] <

  # Likewise populate the function name array and goto label array
  # ... populating code here ...

  # Other code, manipulating CompIdx, fvar1, fvar2, FnIdx, GtIdx, etc.
  # Note that the 'if <varval>' component of call_func and goto are optional
  # but, as an example, we could do something like this:

  set_condexp fvar1 CompOp[CompIdx] fvar2
  call_func FuncName[FnIdx] if Condexp

  # Other code here ...
```

```
  set_condexp ivar1 CompOp[CompIdx] ivar2
  goto GotoLabel[GtIdx] if Condexp   # Also remember that a PCS goto is
                                     # 'stack aware' and capable of non-local
  # etc.                             # jumps.

return

# Other functions defined below here.

exit_label:
exit
```

3. Transparent State Management

PCS uses a Global-Only variable environment. By eliminating local scope and variable "shadowing," the AI always has a clear, unambiguous view of the entire machine state. This is particularly effective for Non-Local Jumps (goto), allowing the AI to move between different processing modes without the risk of stack-frame corruption or hidden memory leaks.

4. Vectorised Reasoning via Universal Arrays

Every variable in PCS is an array. This encourages the AI to think in "batches" or "lookup tables." By storing function labels, branch targets, or mathematical operators in arrays, an AI can navigate complex behavioural states using simple index-based dispatching.

5. Explicit Data Casting

PCS employs a "Left-Hand Master" rule for automatic casting. When comparing or operating on different data types, the interpreter clearly defines how the conversion occurs. This predictability ensures that an AI-coder can accurately anticipate how the system will handle "3" (string) versus 3 (integer).

Example:

`set_var myStr 5`

results in `myStr` containing the string composed of ['5']['\0'] whereas, using this same `myStr`, if we do

`set_var myInt myStr`

this results in `myInt` containing the numerical value 5.

6. Minimalist Delimiter Logic

By dispensing with quotes and brackets for strings and complex expressions, PCS minimises "syntactic noise." This reduces the likelihood of the AI "forgetting" a closing character and ensures that the literal data provided is exactly what sits in memory.

7. The Scratchpad zone

The PCS exit command allows for "free text annotation" below it in any script file. This is a very useful feature for LLMs. The area below exit may be used for persistent memory, scratchpad logic, or storing JSON metadata that doesn't need to be parsed by the PCS interpreter. AI models appreciate having a designated "scratchpad" area.

Guide for AI prompting

When using an AI LLM to generate or debug PCS scripts, providing a clear "System Prompt" is essential. Because PCS uses a flat, global-only architecture with unique string handling, you should explicitly instruct the AI to adopt the following coding style:

The "PCS Expert" Prompt

To get the best results, start your request to the AI with a prompt similar to this:

```
*"Act as an expert PCS programmer. Write a script for version 1 that follows these rules:

    • Provide a 'Plan' in the Scratchpad zone below the exit command before writing the main code.
    • Use a flat logic structure (avoid deep nesting where possible).
    • Remember that strings have no delimiters; quotes are literal data.
    • All variables are global; use unique prefixes for function-specific variables.
    • Use the set_condexp result Condexp for all branching logic.
    • All variables are arrays; use var[index] syntax for sizes > 1."*
```

Guidelines for AI Code Generation

- **Atomic Steps**: Ask the AI to "break complex maths into individual math_var or add_var steps." This prevents the model from trying to use C-style inline equations which will fail the PCS syntax check.
- **String Formatting**: Remind the AI that "spaces between arguments are collapsed." If the AI needs to generate a formatted UI, instruct it to "use the Space system variable with `add_var` (or `cat_char` with a value 32) to build precise whitespace."

- **Dynamic Labels**: Encourage the AI to "use string variables for goto labels to create state-machine transitions." LLMs are excellent at managing state tables when given the permission to do so.
- **Verification**: Ask the AI to "explain the logic of its set_condexp calculations" before providing the code. This forces the model to verify its Boolean-to-Integer maths (e.g., exp1 + exp2 == 2 for an AND gate).

Avoid blind trust

Since PCS is a language that controls hardware, mistakes can lead to physical damage (just as mistakes in G-code can damage a 3D printer or CNC machine).

It will likely be a long time (and after a lot of specialist PCS training) before an AI can generate code that approximates to 'safe' code.

It is highly advisable ***not*** to simply 'trust and run' PCS scripts written by an AI. You should examine the code to ensure it does what you intend in a safe way (for example, that all extant motors have the correct limit switch pin assignments) before running it on a live robot, CNC stage or other mechanism. Close supervision of the robot / stage is also advised, especially on the first run and be ready to hit the kill switch if necessary.

10 List of commands

Here is an alphabetically ordered list of current prime commands and their usage notes.

act_accel

`act_accel <act_type> [args based on <act_type>]`

This sets essential common parameters for motion of an actuator of type <act_type> which will be used for all non-homing motions with actuators of that type till you change those parameters with another call to this command.

<act_type> - for the options for this integer see def_actuator. The arguments that are expected to follow <act_type> will depend on <act_type>.

For <act_type> = 1 (stepper) the following format is expected:

`act_accel <act_type> <prop_acc> <isd_min> <isd_max_min>`

Their meanings, data types and value ranges are as follows:

`<prop_acc>` - a float number between 0.0 and 1.0. This is the proportion of the motion steps that are involved in acceleration and deceleration (as opposed to the remaining steps which will be done at a constant (maximum) velocity in between the acceleration and deceleration phases.

`<isd_min>` - For stepper motors this is the minimum inter-step delay (isd) in microseconds. This determines the maximum speed of travel once travelling at constant velocity (i.e. at the end of any initial acceleration phase from a standstill). It must be > 1 (and is usually a few hundred or more. A value much lower than this probably won't be honoured in the motion because the various C source code statements that occur between motor pulse steps (like checking for limit switch status, etc.) will take about 150-200 microseconds to complete meaning any value you supply here much less lower this will just get over-ridden with whatever minimum time it takes to do those statements).

`<isd_max_min>` - This is the longest isd you are willing to tolerate at the start and end of the motion. This number should be considerably larger than `<isd_min>` or the motion profile calculation will fail. A value in the low thousands would be typical. Both `<isd_min>` and `<isd_max_min>` are integers representing a whole number of microseconds.

Note: This command needs to be called at least once per login session in order for other motion commands (apart from homing) to be allowed.

act_get

`act_get <act_id> <info_ID> <var> [<var_y> <var_z>]`

This reads the value of the <info_ID> struct member of the actuator defined by <act_id> and sets it as the value of the variable <var>.

`<act_id>` - See `undef_actuator` for a full description of this.

`<info_ID>` - must be an integer corresponding to a readable item from the actuator's data structure. For stepper actuators, the currently valid readable info item integers are shown below (taken from parddefs.h - see that header file for details of the struct members and their meanings but an extract from that file is provided in the 'Notes' section below for quick reference here). Note that only the integer value of the information item must be supplied, not the 'INFO_' token string, *unless* you make a pch header file containing these #defines (excluding the C comments) and #include it in your script, then you may use these #defined 'INFO_' token strings in your PCS script to make it more readable:

```
#define INFO_ENA_STATE 1  // Ardu only    // Ask Ardu
#define INFO_HOMED_TO  2                  // Ask Ardu  // Read only
#define INFO_DIRN1     3                  // LG
#define INFO_DIRN2     4                  // LG
#define INFO_DIRNC     5                  // Ask Ardu
#define INFO_DIRNP     6  // Ardu only    // Ask Ardu  // Read only
#define INFO_MSDENOM   7                  // LG
#define INFO_MSDEMAX   8  // Server only  // LG
#define INFO_PULWIDTH  9                  // LG
#define INFO_BLSTEPS   10                 // LG        // Read only
#define INFO_BLSTMAX   11 // Server only  // LG
#define INFO_CALIMAX   12 // Server only  // LG
#define INFO_STEPCOUNT 13                 // Ask Ardu
#define INFO_SW_LIM1   14                 // LG
#define INFO_SW_LIM2   15                 // LG
#define INFO_NHS       16                 // LG
#define INFO_ISD_HOBL  17                 // LG
#define INFO_LIMSTATE  18                 // Ask Ardu  // Read only
#define INFO_PIN_PUL   19                 // LG
#define INFO_PIN_DIR   20                 // LG
#define INFO_PIN_LIM1  21                 // LG
#define INFO_PIN_LIM2  22                 // LG
#define INFO_PIN_DIAG  23                 // LG
#define INFO_PIN_ENA   24                 // LG
#define INFO_NAME_LS1  25 // Server only  // LG
#define INFO_NAME_LS2  26 // Server only  // LG
```

```
#define INFO_NAME_SW1  27 // Server only  // LG
#define INFO_NAME_SW2  28 // Server only  // LG
#define INFO_NAME_UNI  29 // Server only  // LG
#define INFO_MATURITY  30 // Server only  // LG        // Read only
#define INFO_XYZCOORDS 31                 // Ask Ardu
#define INFO_DURALAM   32 // Ardu only    // Ask Ardu  // Read only
#define INFO_STEPLAM   33 // Ardu only    // Ask Ardu  // Read only
#define INFO_ONESTEP   34                 // LG        // Read only
#define INFO_FASTMSD   35                 // LG
#define INFO_MMODE     36 // Ardu only    // Ask Ardu
#define INFO_TMCSGVAL  37 // Ardu only    // Ask Ardu  // Read only
#define INFO_TMCSTATUS 38 // Ardu only    // Ask Ardu  // Read only
#define INFO_TMCCHCONF 39 // Ardu only    // Ask Ardu  // Read only
#define INFO_SG_BUFFPC 40 // Ardu only    // Ask Ardu
#define INFO_SG_THRESH 41 // Ardu only    // Ask Ardu
#define INFO_TURBO_BST 42 // Ardu only    // Ask Ardu
#define INFO_TMCMSCNT  43 // Ardu only    // Ask Ardu  // Read only
#define INFO_TENSTEPS1 44                 // LG
#define INFO_TENSTEPS2 45                 // LG
#define INFO_UNIT_TYPE 46 // Server only  // LG        // Read only
```

The items labelled 'Server only' are data only stored on the server for this actuator (they are not stored on the Arduino) so accessing these items does not require the server to communicate with the Arduino. Conversely the items labelled 'Ardu only' are only held on the Arduino so can only be accessed by means of the server communicating with the Arduino and that could cause a stutter or short delay in otherwise continuous actions (like motion) being performed by the Arduino.

All the other items are maintained in sync on both the Arduino and the server. To save unnecessary comms with the Arduino, only those items that may be set or change as a result of Arduino actions are read from the Arduino ('Ask Ardu') by the server instead of being looked up on the server's local copy of the structure (local get or 'LG') in response to a client's `act_get` command. Obviously, an `act_set` command will require communication with the Arduino to set the data on the Arduino's version of the actuator structure with the exception of 'Server only' data items.

For the command to succeed `<act_id>` must represent an actuator that is currently defined on the server, `<info_ID>` must correspond to a readable member of that actuator's structure and `<var>` must be a user-defined variable (not a literal constant value or an internal system variable) and must be of a data type that can accept the requested info item (i.e. a string, float or int).

The optional two arguments <var_y> <var_z> are used in the cases where <info_ID> specifies three values. These are:

INFO_NHS

INFO_XYZCOORDS

In the latter case the supplied <act_id> is overridden by the stepper actuator IDs of 0, 1 and 2 (the default XYZ motors of the stage) and this command will only be allowed if those three steppers are defined.

Note: The <info_ID> of INFO_UNIT_TYPE is labelled as 'read only' because it can only be used with act_get but not act_set. However you can set this value on the server using the separate command act_unit (which see).

Note: Here is an extract from the parddefs.h file that describes the actual stepper motor struct members the above info items relate to (as well as struct members the PCS script writer does not have direct access to):

```
// Actuator Definition Structure for a Stepper Motor


#ifdef ARDUINO
// This is the structure as used on the Arduino:
typedef struct {
   TMC2209Stepper *tmc_driver; // Pointer to the TMC driver object
   uint8_t  ena_state;   // The enabled status (one of the ENA_ #defines)
   uint8_t  motion_mode; // MM_STEALTHY or MM_PRECISE
   uint8_t  turbo_boost; // 0 = off, 1 = transient on, >1 = latched on
   uint8_t  homed_to;    // Which limit switch the motor is currently homes to
                         // (one of the LIM_ #defines except SOFT1 and SOFT2 )
   uint8_t  dirn1;       // The direction value (1 or 0) that causes the
                         // motor to spin in the direction that causes motion
                         // towards limit switch 1.
   uint8_t  dirn2;       // The direction value (1 or 0) that causes the
                         // motor to spin in the direction that causes motion
                         // towards limit switch 2.
   uint8_t  currdirn;    // The current direction (one of the DIRN_ #defines)
   uint8_t  prevdirn;    // The last direction the motor moved in (DIRN_ #defs)
   uint32_t msdenom;     // The microstep denominator (2,4,8,16,32,64,etc)
   uint32_t fastmsd;     // The microstep denom. for the  fast phase of homing
   uint8_t  pulwidth;    // The number of microseconds for a pulse (high part)
```

```
   uint32_t blsteps;     // The number of backlash steps to take when needed
   uint32_t tensteps1;   // The number of (de)tensioning steps to take in dirn1
   uint32_t tensteps2;   // The number of (de)tensioning steps to take in dirn2
   uint32_t onestep;     // The number of maxmsdenom steps that a single step in
                         // the current msdenom is worth. This will be >= 1.0 .
   uint32_t stepcount;   // The number of maxmsdenom steps from the homed pos.n
   uint32_t stallcount;  // The number of stall signals (for StallGuard use).
   uint32_t duralam;     // The duration (in microseconds) of the last motion.
   uint32_t steplam;     // The number of steps taken in the last motion.
   uint32_t sw_lim1;     // Stop if stepcount is >= this software limit.
   uint32_t sw_lim2;     // Stop if stepcount is <= this software limit.
   uint32_t  nhs0;       // (n)ull (h)ome (s)teps - initial dirn2 steps
   uint32_t  nhs1;       // (n)ull (h)ome (s)teps - dirn1 steps
   uint32_t  nhs2;       // (n)ull (h)ome (s)teps - final dirn2 steps
   uint32_t isd_hobl;    // The inter-step delay to use for homing and backlash
   uint8_t  limstate;    // The current state of the limit switches (LIM_ #defs)
   uint8_t  sg_buffpc;   // Stall isd percentage buffer (<100 deactivates SG)
   uint8_t  sg_thresh;   // The SGTHRS value for StallGuard4 (0 deactivates SG)
   uint8_t  pin_pul;     // Arduino pin to use for the PUL signal (digital)
   uint8_t  pin_dir;     // Arduino pin to use for the DIR signal (digital)
   uint8_t  pin_lim1;    // Arduino pin to use for the LIM_SWTC1 signal (analogue)
   uint8_t  pin_lim2;    // Arduino pin to use for the LIM_SWTC2 signal (analogue)
   uint8_t  pin_diag;    // Arduino pin to use for the LIM_STALL signal (analogue)
   uint8_t  pin_ena;     // Arduino pin to use for the ENA signal (digital)
 } Act_StepperNode;
#endif
#ifndef ARDUINO
// This is the structure as used on the server PC:
typedef struct {
   uint8_t  homed_to;    // Which limit switch the motor is currently homes to
                         // (one of the LIM_ #defines except SOFT1 and SOFT2 )
   uint8_t  dirn1;       // The direction value (1 or 0) that causes the
                         // motor to spin in the direction that causes motion
                         // towards limit switch 1.
```

```
uint8_t  dirn2;       // The direction value (1 or 0) that causes the
                      // motor to spin in the direction that causes motion
                      // towards limit switch 2.
uint8_t  currdirn;    // The current direction (one of the DIRN_ #defines)
uint8_t  pulwidth;    // The number of microseconds for a pulse (high part)
uint32_t blsteps;     // The number of backlash steps to take when needed
uint32_t tensteps1;   // The number of (de)tensioning steps to take in dirn1
uint32_t tensteps2;   // The number of (de)tensioning steps to take in dirn2
uint32_t maxblsteps;  // The backlash steps at maximum microstep denom
uint32_t msdenom;     // The microstep denominator (2,4,8,16,32,64,etc)
uint32_t fastmsd;     // The microstep denom. for the  fast phase of homing
uint32_t maxmsdenom;  // The maximum microstep denom
uint32_t onestep;     // The number of maxmsdenom steps that a single step in
                      // the current msdenom is worth. This will be >= 1.0 .
uint32_t stepcount;   // The number of maxmsdenom steps from the homed pos.n
double   maxcalibri;  // The number of maxmsdenom steps in a calibrated unit.
uint32_t sw_lim1;     // Stop if stepcount is >= this software limit.
uint32_t sw_lim2;     // Stop if stepcount is <= this software limit.
uint32_t  nhs0;       // (n)ull (h)ome (s)teps - initial dirn2 steps
uint32_t  nhs1;       // (n)ull (h)ome (s)teps - dirn1 steps
uint32_t  nhs2;       // (n)ull (h)ome (s)teps - final dirn2 steps
uint32_t isd_hobl;    // The inter-step delay to use for homing and backlash
uint8_t  limstate;    // The current state of the limit switches (LIM_ #defs)
uint8_t  pin_pul;     // Arduino pin to use for the PUL signal (digital)
uint8_t  pin_dir;     // Arduino pin to use for the DIR signal (digital)
uint8_t  pin_lim1;    // Arduino pin to use for the LIM_SWTC1 signal (analogue)
uint8_t  pin_lim2;    // Arduino pin to use for the LIM_SWTC2 signal (analogue)
uint8_t  pin_diag;    // Arduino pin to use for the LIM_STALL signal (analogue)
uint8_t  pin_ena;     // Arduino pin to use for the ENA signal (digital)
char    *lsname[5];   // List of limit switch names ([0] is always "Unhomed")
char    *unitstr;     // Name of units motor is calibrated to.
char     Name[32];    // Name of this actuator as supplied by the user.
uint8_t  maturity;    // Must reach a certain value for the actuator to be
                      // usable (starts off as 0 when the actuator is first
```

```
                        // created.
 } Act_StepperNode;
#endif
```

and here are some of the relevant #defined constants with their explanations:

```
// The ENA values
#define ENA_DISABLED 0  // Disabled, non-energised, coils
#define ENA_ENABLED  1  // Enabled, energised, coils but not currently spinning
#define ENA_SPINNING 2  // Motor is currently undergoing motion steps



// The limit or limit switch (LIM_) identifier values. These are used to
// identify which limit switch a motor is homed_to and also which limit or limit
// switch is currently triggered in the limstate member:

#define LIM_UNSET   0  // Not homed (or no limit switch triggered)
#define LIM_SWTC1   1  // Physical switch 1 (triggered by moving in dirn1)
#define LIM_SWTC2   2  // Physical switch 2 (triggered by moving in dirn2)
#define LIM_NULL1   3  // Null-homed 1 (step count goes to 0 by moving in dirn1)
#define LIM_NULL2   4  // Null-homed 2 (step count goes to 0 by moving in dirn2)
#define LIM_SOFT1   5  // Software limit 1 activated (cannot be homed to)
#define LIM_SOFT2   6  // Software limit 2 activated (cannot be homed to)
#define LIM_ZERO    7  // Motor attempted to move to a negative step count. This
                       // can happen if a limit switch is a bit noisy or has
                       // some error or if there is some error in the axes such
                       // that returning to a step count of zero does not
                       // trigger the limit switch. Alternatively this can
                       // happen in cases of null homing where there is no
                       // physical end stop switch to trigger and step count is
                       // all we have.
#define LIM_STALL   8  // The StallGuard4 threshold was exceeded - likely stall.
                       // If a stall is detected then the Arduino sends back the
                       // the error code  to the user but that does not tell you
                       // which motor triggered the stall event. However the
                       // Arduino also sets the limstate member of the motor
```

```
                        // that triggered the stall to this LIM_STALL value. So
                        // the user can do a 'cold read' of the limit state of
                        // each motor in turn (using act_get with INFO_LIMSTATE)
                        // to find out which motor triggered the stall.
```

Note: You can supply all three receptor variables in cases where only one is needed. This will not generate an error, the other two variables will just be ignored. However, you must supply the additional two variables in cases where the requested information item requires them or a syntax error will result.

Note: `act_get` is a *mandatum interruptivum*, a command that is allowed to interrupt other commands. Only some 'other commands' will look for and so act on *mandata interruptiva*. These include motion commands for steppers.

Note: For guidance on whether to use float or int var arguments, see the entry on the command `act_unit`.

Note: When getting coordinates or distances using `INFO_XYZCOORDS`, `INFO_STEPCOUNT`, `INFO_SW_LIM1` and `INFO_SW_LIM2`, the value you will receive from the server using `act_get` will be in atomic units regardless of what you have set `act_unit` to be.

Note: For the INFO_TMCSTATUS option, which only applies to steppers, your supplied `<var>` will be assigned an int32_t cast of the original uint32_t status register but the interpreter's private internal global TMC_Status_Reg variable will also be updated to the unaltered uint32_t register value. You cannot access this TMC_Status_Reg variable directly via the script but you can access the status information items it contains by means of the `tmc_status_parse` command, which see. However, the sign of the signed value you receive in your `<var>` here provides you with one piece of status information - if it is positive it means the driver's standstill status flag is unset and if it is negative it means that the flag is set.

Note: For the INFO_TMCCHCONF option, which only applies to steppers, your supplied `<var>` will be assigned an int32_t cast of the original uint32_t chop conf register but the interpreter's internal private global TMC_ChopConf_Reg will also be updated to the unaltered uint32_t register value. You cannot access this TMC_ChopConf_Reg variable directly via the script but you can access the status information items it contains by means of the `tmc_chconf_parse` command, which see. However, the sign of the resulting value will reflect the state of the 'Short protection low side' feature in the register, with a positive value indicating this protection is ON and a negative value indicating this protection is OFF.

Note: Any call to get a data item that requires communication with the Arduino (marked as 'Ask Ardu' in the list above) can cause a delay or jitter in motion due to extending the inter-step delay (while the Arduino processes your request) and possibly even more of a time disruption will occur with those information requests that involve UART comms with the stepper driver (i.e. anything that begins with INFO_TMC). So avoid making such requests during sensitive motion actions.

Note: Setting TurboBoost mode with INFO_TURBO_BST to some non-zero number will mark that motor for using extra RMS ampage for its next movement command - whatever that command may be. The TurboBoost setting will take place immediately (the motor must be stationary and an enforced 100 ms delay without any steps will be imposed at the Arduino when making the change) so setting TurboBoost without motion will increase the holding current. After a motion command is executed for that motor the TurboBoost status will automatically be reset to 0 and current will return to normal if and only if the TurboBoost status was previously set to 1 (as opposed to any other non-zero number). If TurboBoost was set to 2 or more then that signals TurboBoost to latch on and so it will not reset to 0 until the PCS programmer deliberately sets it to 0 or until they set it to 1 thereby allowing it to automatically clear back down to 0 after the next motion command. The actual current values for TurboBoost and 'normal' (non-Turbo) current are set in #defines near the start of the Arduino source code. Typical values may be (for non-Turbo) 600 mA RMS during motion and 300 mA at rest and for TurboBoost that will increase to 670 mA RMS during motion and 335 mA at rest but clearly you can change that via the Arduino sketch source code (parduino.ino). Leaving motors on TurboBoost at rest should only be done when necessary and you should manually set TurboBoost to 0 when you no longer need the stronger holding rest current. This is to avoid unnecessary heating of the motors and the drivers.

act_getid

`act_getid <name_string>`

This will update the internal system variables `CurrActIdx` and `CurrActType` with the index and actuator type (respectively) of the actuator of name `<name_string>`. The `CurrActType` integer value will correspond to the actuator's type thus:

1 = Stepper

2 = Binary

3 = Mlevel

4 = Servo

If there is no extant actuator with the supplied name, the values of `CurrActIdx` and `CurrActType` will both be set to -1. In this way you can test if an actuator of a certain name exists. See also `act_getname`.

Note: This works by searching the local (client side) database of currently extant actuators and does not communicate with the server. You should ensure, therefore, that the command `get_actuators` has been called at least once before using this command.

act_getname

`act_getname <act_type> <act_index> <string_variable>`

This gets the name of the actuator of type <act_type> and index number <act_index> and sets the string variable <string_variable> to that name.

<act_type> must be one of these integers or the command will give a run-time error:

1 = Stepper

2 = Binary

3 = Mlevel

4 = Servo

If there is no extant actuator with that type and index, the value of <string_variable> will be set to "null" (without the quotes). So, in this way you can test if an actuator exists. See also act_getid.

Note: This works by searching the local (client side) database of currently extant actuators and does not communicate with the server. You should ensure, therefore, that the command get_actuators has been called at least once before using this command.

Note: def_actuator and act_getname are the only two commands that require you to input two separate numbers to identify an actuator instead of the single <act_id> argument (described, for example, under undef_actuator)

act_halt

act_halt <act_id>

Causes an actuator to stop what it is doing as soon as the act_halt instruction is received, even if the actuator has not finished its task.

<act_id> - See undef_actuator for a full description of this.

Note: act_halt is a *mandatum interruptivum*, a command that is allowed to interrupt other commands. Only some 'other commands' will look for and so act on *mandata interruptiva*. These include motion commands for steppers.

act_home

act_home <act_id> <home_type>

Home the actuator defined by <act_id> in a way defined by <home_type>

<act_id> - See undef_actuator for a full description of this.

<home_type> - an integer which corresponds to one of the following:

If the actuator is of type stepper these integers are potentially valid:

```
#define LIM_SWTC1  1 // Home to limit switch 1 (triggered by moving in dirn1)
```

```
#define LIM_SWTC2  2 // Home to limit switch 2 (triggered by moving in dirn2)
#define LIM_NULL1  3 // Null-homed 1 (step count goes to 0 by moving in dirn1)
#define LIM_NULL2  4 // Null-homed 2 (step count goes to 0 by moving in dirn2)
```

I say 'potentially' valid because any of them may be invalid if, for example, the stepper structure has not been set up for null homing or with a limit switch pin set to an Arduino pin.

You cannot home to a software limit.

An actuator of type stepper cannot be moved with any other command until it is homed.

When you select one of the two null homing types, your choice of LIM_NULL1 or LIM_NULL2 will not necessarily be what ends up as the value of the homed_to member of the motor structure. All these options do at this stage is tell the server to initiate a null homing procedure. What ultimately determines whether the motor is homed_to LIM_NULL1 or LIM_NULL2 is your choice of the nhs0, nhs1 and nhs2 step counts as set using the `act_set` command (if the homing procedure continues to completion) because the last direction the motor moves in when homing ends, the homed_to value will be the opposite of that direction (if the last direction was dirn1 then the motor will be homed_to=LIM_NULL2 and *vice versa*).

If the homing procedure is halted before completion homed_to will be LIM_UNSET (= un-homed).

The reason the null homing switch number is opposite to the last moved-in direction is to make the terminology of null homing 'limits' analogous to the terminology used for physical end stops - LIM_SWTC1 and LIM_SWTC2. So, for example, when homed to physical end-stop LIM_SWTC2 it means that this is the switch that will get triggered when the motor moves enough steps in dirn2. So moving *away from* LIM_SWTC2 causes step count to rise. Likewise, in null homing where the last moved-in direction is dirn1, it will be like homing to a virtual end stop switch 2 where moving away from this switch (i.e. in dirn1) will cause the step count to rise and moving towards the switch (in dirn2) will cause the step count to decrease.

Only actuators of a type that can be homed will be accepted by this command. If the `<act_id>` specifies any other type then the command will return an immediate error.

The command will return OK if the server finds no problem with the arguments and the Arduino does not have a problem with the arguments. This means the Arduino has 'agreed' to attempt the homing procedure and will get on with it but the Arduino returns immediately after the command passes its tests and does not wait for the homing procedure to start or finish. If an error occurs during homing the 'PARD move report' (PMR) that you can check with the command `act_movereport` (which see) may contained information it its error code that may help you understand the reason for failure.

You, the PCS script writer, can check for success or failure of this command by doing an `act_get` command on the same actuator's `<homed_to>` member. If you make this request before homing completes you could get a 'busy' response. Otherwise if the `<homed_to>` member equals the value you set as `<home_type>` it means the homing completed OK. Any other value suggests an error occurred.

Note: Because the Arduino returns control back to the user immediately this avoids lengthy delays in comms (that might cause a timeout error) and also allows the user to do other things - like generating a live preview - while the homing action is taking place thereby allowing for visual feedback assessment of the homing procedure which can give another means to know if something went wrong and when the procedure has ended.

Note: For null-homing no physical limit switch will be checked during the homing procedure and no physical limit switch will be checked for any subsequent motion unless you have also set a pin for a physical end stop. If you have not set any physical end stop pin then it is entirely the user's responsibility to take measures to avoid those physical limits being hit and potentially damaging the hardware.

Note: Homing will set the currdirn value on the breakout board. Until that is done the currdirn may show as UNSET when attempts are made to `act_get` it.

Note: Immediately after successful homing of any type, the step count will be reset to 0 and you can only move in the direction away from the limit switch you homed to (you can't move to a negative step count). For null homing this is also true. for example, if you have null homed to LIM_NULL1 this means that motion in dirn2 increases your step count from 0 and motion in dirn1 decreases your step count but if you try to move below a step count of 0 this will be treated as if a physical limit switch was triggered (and limstate will take the value LIM_ZERO to indicate this). However, in the case of a null homed motor where the is no physical limit stopping you, you may want to move in this 'negative' direction (e.g. to operate a filter wheel that may rotate in any direction without a physical stop or to operate a pump with similar lack of restriction). To enable this, just issue an `act_set` command after homing to set the current step count to some (usually large) positive number. Then you will be able to move in any direction until the step count reaches zero (or until a physical limit switch is triggered in cases where you have one setup for that direction (set up both in terms of physical hardware and stepper actuator configuration with `act_set` commands).

Note: Even with physical limit switches you may get a halt triggered by step count reaching zero (the homed position) even if the physical limit switch does not trigger at this point due to some error in motion (stalls or other error) or simply due to some error/variation in the triggering position of some limit switches. In that case you can tell whether this has occurred simply because you reached a step count of zero without triggering the switch (limstate will show LIM_ZERO) compared to whether the switch was actually triggered (limstate will show LIM_SWTC1 or LIM_SWTC2) by checking the limstate value with `act_get`. In the case of a LIM_ZERO halt you should be able to continue to move the motor in the forbidden direction by setting the step count to some positive number and carry on moving in that direction but this will only work till the actual physical limit switch triggers - a triggered physical limit switch will override all attempts at motion regardless of step count.

Note: For stepper motors you can check when the motion is complete in three main ways. The first is the preferred way because it does not interrupt the Arduino while the motor is moving and so cause potential slight disruptions to the regularity of the inter-step delay and this is by using the `act_movereport` command (which see). If that method is not available for some specific reason then another way is by doing an `act_get` for the stepper for `<info_ID>` = 1 (INFO_ENA_STATE).

The ena_state will be 2 when the motor is spinning and 1 when at rest. You could alternatively do an act_get for <info_ID> = 2 (INFO_HOMED_TO). The difference between the latter two methods is that the homed_to state will only change from un-homed to homed (to the appropriate limit) if the motion completes successfully but ENA state will change from 2 to 1 when the motion ends even if, for some reason, it ended prematurely such that the homing could not be complete. This may occur if an act_halt command stopped it before it could finish (for example).

Note: When homing starts, the homed_to member of the motor structure is set to LIM_UNSET (to indicate that it is un-homed) so that if homing is interrupted or fails this will be the resulting state of the motor.

act_moveby

```
act_moveby <act_id> <duration> <distance> [<dist_y> [dist_z]]
```

This moves the actuator identified by <act_id> by a certain distance from its current position in its current direction (current direction can be set using the act_set command, which see) and attempts to do this motion over a time of <duration> microseconds.

<act_id> - See undef_actuator for a full description of this. In addition to the options described there, this command may also accept an <act_id> of StageXY or StageXYZ (case sensitive) which will attempt motion of 2 (X and Y) or 3 (X, Y and Z) axes of the stage. This means stepper actuators of ID 0 for X, 1 for Y and 2 for Z. In the case of <act_id> = StageXY you must supply one more argument, <dist_y>, which will apply to the Y axis motion (the X axis motion being governed by the <distance> argument). If you supply <act_id> = StageXYZ then you must supply the [dist_z] argument in addition to <distance> and <dist_y>. The motion will be done using the multidimensional Bressenham line algorithm to effect simultaneous coordinated motion for 2 or 3 motors. If you only want to move the X and Z motors then use <act_id> = StageXYZ and set the <dist_y> value as 0. You can use these StageXY and StageXYZ options for single motor activation (if all but 1 of the distance arguments are 0). In that case the PCS script interpreter will simply convert your multi-motor move command into a single motor move command in the program (because there are less overheads in doing single motor movements because they do not need Bressenham calculations). Also you can use these StageXY and StageXYZ options for no motion at all (all the distance arguments are 0) - the script will just do no motion in that case.

<duration> - an integer which may be 0 or positive. If 0 it means you want the duration calculated automatically based on your choice of <isd_min> (from the act_accel command) and the total number of steps the movement will require (which will be calculated over on the server from your choice of microstep denominator and <distance> and <unit> (from the act_unit command). If it is positive then it represents the number of microseconds you want the whole motion to take. Your choice here may cause an acceleration curve to be impossible to satisfy resulting in failed calculation falling back to the default uniform motion - so use this manual option for <duration> with care.

<distance> - a float-capable value >= 0.0 specifying the number of distance units to move.

<dist_y> - a float-capable value >= 0.0 specifying the number of distance units to move for the stage Y axis motor when <act_id> is StageXY or StageXYZ.

[dist_z] - a float-capable value >= 0.0 specifying the number of distance units to move for the stage Z axis motor when <act_id> is StageXYZ.

Note: All distances are expected to be supplied in either calibrated units (if you set act_unit to > 0 for steppers) or in atomic steps (i.e. 1/maxmsdenom microsteps). No negative values are allowed. Because a stepper motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply here for distances and positions will be rounded to the nearest ***onestep*** value (which is calculated internally in the server program using the formula onestep = maxmsdenom / msdenom). For example, if you set one of the distance arguments to 15 when maxmsdenom in 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) then this will result in an effective distance of 0 and the motor will not move at all because (uint32_t)(0.5 + (15.0 / 32.0)) = 0. However if you set a distance argument = 16 then the effective value will be 1 because (uint32_t)(0.5 + (16.0 / 32.0)) = 1 and your motor will travel 1 msdenom step meaning the step count will increment or decrement by 32 atomic units or less. I say 'or less' for the potential cases where you have been moving around with different msdenom settings such that, for example, if your current atomic step count is 10 and your act_moveby command results in a request to move back by 32 atomic steps, no movement will actually occur but the step count will be set to 0. Such 'drift' should not be possible if you are careful to observe msdenom and distance discipline when operating at different msdenom settings in one login session.

Note: In order for this command to be processed you must have, at least once during the current log in session, made a call to act_accel for the type of actuator you want to move with this command. Failure to do this will result in the command being denied with an error (the script will terminate or fail to check out OK).

Note: The movement you requested may not be fulfilled exactly as you requested. For example a limit might be reached (either a software limit or a physical end-stop) that terminates the motion before the full <distance> is achieved or the <duration> you specified might not be possible for some reason and the movement may take longer (or shorter if a limit is triggered). Thus you might want to get information on the actual motion, for example so you can make corrections in future motions (like decrease <duration> in any future motion requests to try to make up for motions that took longer than expected). You can do this as follows:

1. To find out the actual steps taken you can use act_get to get the step count before and after the motion.
2. To get the duration you can use act_get to read the 'INFO_DURALAM' member of the actuator structure which stores the last know duration for the last movement it made (apart from homing).
3. Just as with homing, the command will return just prior to actually beginning the motion so you can tell when the motion has ended, e.g., by using the act_movereport command or querying the ena_state with act_get (it will be 2 while in motion and 1 when at rest).

Note: If act_moveby is called with a <distance> (or one of the other distance arguments) value of 0 then a 'dummy' PMR is set up at the server side to respond to any call to act_movereport. This dummy PMR will just send back all the PMRep variable values as they were last (without changing their values) and also the actuator's duralam and steplam members will not have changed from when the actuator last moved so will not be reliable in those circumstances. In the case of <act_id> = StageXY, this dummy PMR situation will only occur when both the <distance> and <dist_y> values are 0. In the case of <act_id> = StageXYZ, this dummy PMR situation will only occur when all three of <distance>, <dist_y> and [dist_z] values are 0.

Note: When doing multi-stage motor movements (<act_id> = StageXY or StageXYZ) and 1 or more of the distance values are 0 - and especially if all but 1 of the distance values are 0 - this may result in different times for motion to complete if you are running speeds that are near the limit of capacity of the breakout board. This is because the breakout board might be able to honour your choice of minimum ISD if only 1 motor is being activated but might not be able to do so and so will effect a longer minimum ISD than you chose or than was possible with only 1 motor activated.

act_movereport

`act_movereport <int_result> <actype> <actid>`

This listens on the incoming serial port of the server for a PARDUS move report (PMR) but it does not wait for one. If a PMR is present when it looks it will do the this:

1. set the value of <int_result> to 1
2. set the value of <act_type> to the type of actuator the PMR pertains to
3. set the value of <act_index> to specific actuator the PMR pertains to
4. set the system variables PMRep1, PMRep2, PMRep3 and PMRep4 to values relevant to a report for that actuator type and the specific actuator.
5. terminate the script with an error message if the PMR conveys that an error has occurred. The error message conveyed to the client via the server will contain information about the nature of the error as provided in the PMR.

In the case that the last movement was that of a stepper motor, these values will contain the current step counts of the X, Y, Z motors (in PMRep1, PMRep2 and PMRep3 respectively) if any of those motors are defined, plus the step count of the most recently activated motor (in PMRep4) if that motor was defined and if it was the motor whose motion ending triggered the PMR. If any of those motors are not defined, the respective PMR value will be 0.

All step counts conveyed by the PMRep variables will be in either calibrated units or atomic (1/maxmsdenom) microsteps depending on your selection for act_unit.

If the last motion (the motion whose ending triggered the PMR) was by a different type of actuator, the values will have meanings relevant to that actuator.

If a PMR is not present when it looks it will set the value of `<int_result>` to 0 and will not alter the values of `<actype>`, `<actid>` or the PMR system variables.

`<int_result>` - a user-defined variable capable of accepting an integer result.

`<actype>` - a user-defined variable capable of accepting an integer result.

`<actid>` - a user-defined variable capable of accepting an integer result.

Note: Once a PMR is read by act_movereport, it is effectively 'used-up' and so it no longer exists. Subsequent calls to act_movereport will return 0 (unless a new PMR is pushed onto the serial port from the Arduino by another actuator completing its motion).

Note: Any command that sends an order to the Arduino will clear the incoming buffer as part of the order send communications system. So, if there was an unread PMR waiting there, it will be cleared (i.e. flushed and lost) and not stored anywhere. You will have lost the opportunity to read it with act_movereport. You may still use other methods to check for end of motion like sending an `act_get` order to look for the current ena_state of a stepper motor, for example.

Note: Because several motion orders, like act_moveby, do not wait for the motion to complete but return as soon as the motion order is accepted by the Arduino, there needs to be some way for the script writer to know when a motion has completed so they do not send non-*interruptivum* commands to the Arduino during the motion and so trigger a 'Busy' error that will terminate the script. One way to find out this information is to periodically do an act_get command to look for the ena_state of a stepper motor (for example) but this necessitates a to-and-fro communication with the Arduino while the motion is potentially still being carried out and this can briefly interrupt the motion making it un-smooth. It is for this reason that this alternative `act_movereport` method is available because it does not require the Arduino to interrupt its current operation to receive its instruction as an order, interpret it, process it and send a report before getting back to whatever it was doing - as it must do for mandata interruptiva like `act_get`. Instead, act_movereport passively checks the incoming serial buffer on the server PC. The Arduino is programmed to send a PMR at the end of any motion command so this does not interfere with the motion while it is happening. By periodically calling `act_movereport` therefore you do not interrupt the Arduino but you still get to find out when a motion command completes - just as if you had periodically called `act_get` for ena_state but without the downsides of that.

act_moveto

```
act_moveto <act_id> <duration> <position> [<posn_y> [posn_z]]
```

This moves an actuator to a specific position.

`<act_id>` - see `act_moveby` for a full description of this.

`<duration>` - an integer which may be 0 or positive. See `act_moveby` for more.

`<position>` - a float-capable value that represents some point along the stroke of the actuator. The units are whatever you set the units to be using the `act_unit` command (which see).

<posn_y> - a float-capable value >= 0.0, as for <position> but pertaining to the Y axis when <act_id> is StageXY or StageXYZ.

[posn_z] - a float-capable value >= 0.0, as for <position> but pertaining to the Z axis when <act_id> is StageXYZ.

Note: All position units are expected to be supplied in either calibrated units (if you set act_unit to >0 for steppers) or in atomic steps - that is, in (1/maxmsdenom microsteps). Positions are measured from 0 (= the homed position) and no negative values are allowed. Because a stepper motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply will be rounded to the nearest *onestep* value (see page 54 for the meaning of onestep). For example, if you set <position> to 15 (assuming we are using atomic steps and not calibrated units) and the motor is currently at position 30, then the difference between current position (30) and desired destination (15) will be calculated as 15 atomic steps. However, when maxmsdenom is 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) this 15 atomic steps will result in an effective motion of 0 (no move at all) because (uint32_t)(0.5 + (15.0 / 32.0)) = 0.

However if you set <position> = 14 making the difference between current position (30) and destination (14) = 16 then the effective value will be 1 step in the current 1/8 step size (= 32 atomic steps) because (uint32_t)(0.5 + (16.0 / 32.0)) = 1. The motor will therefore try to move 32 atomic steps back towards home but it will not actually move because this would take it past 0. This is because the Arduino code has this check in it prior to taking any step:

```
    if(Stepperlist[id]->stepcount < Stepperlist[id]->onestep){
       Stepperlist[id]->stepcount = 0.0;
       Stepperlist[id]->limstate=LIM_ZERO;
       return 1;
      } else Stepperlist[id]->stepcount-=Stepperlist[id]->onestep;
```

So the step counter will be set to 0 (even though it was 30 and even though no motion will be done) and the step function will return a 1 (usually triggering an 'end of motion' signal). This scenario is unusual and should only occur if you switch microstep size to a different value during some motion session. This is because you could never get a stepper position of 15 if you only ever moved in 1/8 steps (in the case of 1/8 steps your motor position will be incremented in units of 'onestep' = 32).

Note: In order for this command to be processed you must have, at least once during the current log-in session, made a call to act_accel for the type of actuator you want to move with this command. Failure to do this will result in the command being denied with an error (the script will terminate or fail to check out OK).

Note: This command works by making a calculation on the server of the difference between the actuator's current position and the <position> you supply here. From this difference it works out the absolute distance the actuator has to travel and in what direction. It sends a command to the Arduino to set the direction and then sends another command to do the motion - that command

literally being an act_moveby command. Thus act_moveto ultimately boils down to an act_moveby command so see act_moveby for further details.

Note: Whether actuator succeeds at moving to the position you specify will depend on whether it meets any limits (like software limits or limit switches) during the motion and whether the motion is halted by an act_halt command. So you should not assume the actuator has gone all the way and you should check the actuator's position at the end of the motion using an act_get command.

Note: The Arduino will not wait for the motion to complete before returning control to the server (and hence to the PCS script). It will return control as soon as a valid move command has been verified and before the motion begins. So as for how to know when the motion is complete, use the act_movereport command (preferably) or do a act_get for things like step count or ena_status (although performing act_get commands while the actuator is in motion might cause slight interruption or jitter to the motion which is why act_movereport is preferred).

Note: If the act_moveto is repeated to the same position (i.e. there are no steps to take because the actuator is already there) then a 'dummy' PMR is set up at the server side to respond to any call to act_movereport. This dummy PMR will just send back all the PMRep variable values as they were last (without changing their values) and also the actuator's duralam and steplam members will not have changed from when the actuator last moved so will not be reliable in those circumstances. In the case of <act_id> = StageXY, this dummy PMR situation will only occur when both the <position> and <posn_y> values result in 0 steps of movement. In the case of <act_id> = StageXYZ, this dummy PMR situation will only occur when all three <position>, <posn_y> and [posn_z] result in no movement.

Note: When doing multi-stage motor movements (<act_id> = StageXY or StageXYZ) and 1 or more of the position values result in no movement - and especially if all but 1 of the distance values result in no movement - this may result in different times for motion to complete if you are running speeds that are near the limit of capacity of the breakout board. This is because the breakout board (Arduino) might be able to honour your choice of minimum ISD if only 1 motor is being activated but might not be able to do so and so will effect a longer minimum ISD than you chose or than was possible with only 1 motor activated.

act_set

```
act_set <act_id> <info_ID> <varval> [<varval_y> <varval_z>]
```

This sets the value of the <info_ID> struct member of the actuator defined by <act_id> to be equal to the value of the variable or constant value <varval>.

<act_id> - See undef_actuator for a full description of this.

<info_ID> - an integer corresponding to a writeable member of that actuator's struct and the value of that member must be compatible with the type of value conveyed by <varval> (string, float or int). Integer values for <info_ID> that can be used for a stepper actuator are all those listed in the section on act_get (which see) with the exception of those labelled 'Read only'.

Note: Physical limit switches may only be associated with analogue pins on the Arduino. You cannot specify 'A' here in any of the varval arguments, like A0 or A4. Instead just use the integer value without the 'A' - it will automatically be treated as if it has an 'A' in front of it. The fact that you are setting the value for a limit switch is what tells the program to interpret this as an analogue pin number. When setting pins for any other purpose the program will interpret the number you give to be be for a digital Arduino pin. Again, do not use 'D', just the integer value and note that values 0 and 1 will be forbidden for digital pins (and so result in an error) because D0 and D1 are reserved for the USB communications that allow the pardserver to communicate with the Arduino. For the limit switch analogue pins you may use 0 and 1 without problem.

Note: When setting the names of limit switches (i.e. using info items like INFO_NAME_LS1) the name you set is limited to 31 printable bytes. Longer names will just have the first 31 printable bytes copied over and the rest discarded. Remember that these are bytes - not logical characters (Unicode UTF-8 characters count as 3 bytes each if you use UTF-8).

Note: If you want to change the identity of the Arduino pin that an actuator pin member is currently associated with, (e.g. a limit switch pin on a stepper) then you must do this in two `act_set` commands. In the first one you must pass the value 255 (this causes the pin to be 'unset'). Then you will be allowed to set the new pin by making another call to `act_set` to assign your new Arduino pin to that actuator pin. You can't just change from one Arduino pin to another in a single `act_set` command.

Note: Be aware that this command may change more than just the member you requested it to change. For example, if you use `act_set` to set a limit switch pin to 'unset' this command will reassess the homed_to member value and, if it was set to be homed to that limit switch, it will cause the homed_to member to be unset also so you will need to re-home the actuator before you can move it again. Likewise if you use `act_set` to alter a software limit or the current step count of one or more steppers then the limitstate member will be reassessed and compared to the current step count to see if it needs to change the limstate member and so you might not be able to move the actuator in the direction of the new software limit (these software limit checks only occur if limstate is not currently set to one of the hardware limit switches because they take priority over software limits). Also the backlash steps are recalculated automatically whenever you change the microstep denominator (msdenom), or the maximum values pertaining to those things (maxmsdenom, maxblsteps). This is why the current backlash steps is read-only to the PCS script writer and should not be set manually by altering the C source code of the interpreter.

Note: You can supply the three varval arguments in cases where only one is needed. This will not generate an error, the other two variables will just be ignored. However, you must supply the additional two varval arguments in cases where the requested information item requires them or a syntax error will result.

Note: For guidance on whether to use float or int varval arguments, see the entry on the command `act_unit`.

Note: The following will be forbidden:

1. setting maxmsdenom to < 1 (i.e. 0),

2. setting msdenom to anything if maxmsdenom is <= 1
3. setting msdenom to be greater than maxmsdenom.
4. setting maxcalibri to <1.0e-3
5. unless specifically stated to be admissible, setting any value negative.

Note: Setting TurboBoost mode with INFO_TURBO_BST to some non-zero number will mark that motor for using extra RMS ampage for its next movement command - whatever that command may be. The TurboBoost setting will take place immediately (the motor must be stationary and an enforced 100 ms delay without any steps will be imposed at the Arduino when making the change) so setting TurboBoost without motion will increase the holding current. After a motion command is executed for that motor the TurboBoost status will automatically be reset to 0 and current will return to normal if and only if the TurboBoost status was previously set to 1 (as opposed to any other non-zero number). If TurboBoost was set to 2 or more then that signals TurboBoost to latch on and so it will not reset to 0 until the PCS programmer deliberately sets it to 0 or until they set it to 1 thereby allowing it to automatically clear back down to 0 after the next motion command. The actual current values for TurboBoost and 'normal' (non-Turbo) current are set in #defines near the start of the Arduino source code. Typical values may be (for non-Turbo) 600 mA RMS during motion and 300 mA at rest and for TurboBoost that will increase to 670 mA RMS during motion and 335 mA at rest but clearly you can change that via the Arduino sketch source code (parduino.ino). Leaving motors on TurboBoost at rest should only be done when necessary and you should manually set TurboBoost to 0 when you no longer need the stronger holding rest current. This is to avoid unnecessary heating of the motors and the drivers.

act_unit

`act_unit <act_type> <unit>`

This sets the way distance and position values (as supplied in motion command arguments) are interpreted for actuators of type `<act_type>`.

`<act_type>` - for the options for this integer see `def_actuator`

`<unit>` - this is an integer varval the meaning of which depends on the actuator type specified previously. For `<act_type>` = 1 (stepper motors) the `<unit>` options are:

0 - this means the distance or position values (including the specification of software limit positions) supplied in any subsequent stepper motor command are to be interpreted in terms of number of steps at the maxmsdenom microstep size (these are called 'atomic units' or 'atomic steps').

1 - this means the distance or position values supplied in any subsequent stepper motion command are to be interpreted in terms of calibrated units and so the same distance or position value will result in a different number of steps being done by the motor depending on the current microstep denominator setting and values for maxmsdenom and maxcalibri.

Note: To read the current value of `<unit>` from the server you use act_get with the info item `INFO_UNIT_TYPE` and submit an extant actuator of the `<act_type>` of interest (the specific actuator index does not matter for this act_get because only the actuator type is used for the look up on the server but the actuator must exists in order to pass the pre-command checks in the interpreter).

Note: The units you choose shall also apply to values you send to the server for setting using the `act_set` command but they do not apply to values you receive back from the server (for example in motor coordinate or 'stepcount' `act_get` requests). This is why coordinates (i.e. step 'counts') and distances are supplied via `act_get` in float variables instead of integers - because these may need to convey integer atomic steps or float-capable calibrated units (for example, a calibrated 'stepcount' may be something like 20156.832 and will be interpreted in the units you have calibrated the stage to e.g. microns). However, when you *receive* such distances and counts with an `act_get` you will always receive the value as a raw atomic integer so you should supply integer-capable receptacle variables to receive the result regardless of the `act_unit` status.

Note: There are a few specific exceptions to these general rules:

1. All nhs steps (used for both for null homing and limit switch homing) are always interpreted (supplied and received) as integer number of steps in the current microstep (msdenom) step size, regardless of maxmsdenom or calibration factor.
2. maxcalibri will always be a float capable number. It does not have to be a whole number of atomic steps. However it must not be <1.0e-3.
3. blsteps will always be interpreted as literal steps in the motor's current microstep (msdenom) size. This is read-only for client. It is calculated by the server as:

```
Stepperlist[ui82]->blsteps = Stepperlist[ui82]->maxblsteps /
                                                    Stepperlist[ui82]->onestep;
```

4. maxblsteps will always be interpreted as atomic steps.
5. tensteps1 and tensteps2 will always be interpreted as atomic steps. The values held in the Arduino will be automatically converted to steps in the current microstep (msdenom) step size as was done for blsteps.

Note: The default unit type for stepper motors will be 0 (number of atomic steps) until you call `act_unit` for stepper motors at least once during a log in session and this choice will remain at what you set it till you log out (so it is not set back to default at the end or start of any script).

add_var

`add_var <varname> <varval>`

This adds a value to a user-defined variable.

`<varname>` - The name of the variable whose value you want to change

<varval> - The value (or variable containing the value) you want to add to <varname>

Note: This is symbolically equivalent to <varname> += <varval> in C.

Note: When the data type of <varname> is 'string' then the <varval> argument will be evaluated and that value cast as a string (if it is not already a string) and printed onto the end of whatever string is in <varname> . This feature can be used to print numerical values into strings.

af_ccrit

af_ccrit <max_iterations> <max_difference> <min_true_peak> <timeout>

This sets convergence criteria parameters for the autofocus command search algorithm. Use this command in conjunction with the af_setup command to fully set up the auto focus (AF) system which may then be invoked via the autofocus command.

<max_iterations> - a varval of type int and must be positive (>= 1). This is the maximum number of iterations (without finding an acceptable peak focus) the search algorithm will tolerate before giving up the search as a failure.

<max_difference> is a varval of type float and must be >= 0.0 . This is the maximum distance in Z (in either direction) from the existing best focus Z position that can be accepted as a new best focus after a search (see 'Note' below). If the AF search results in a position further away than this then AF fails and the pre-existing best Z position is preserved. This helps to limit the acceptable search area and prevent the algorithm converging to a local peak that is far off true focus (e.g. for microscopy this would mean focus on debris or pen marks on a coverslip in an image with little tissue contrast). If you do not want to use this limit, set the value to 0.

<min_true_peak> is a varval of type 'float' and must be >= 0.0. This is the minimum peak value (returned from the goodness-of-focus function) acceptable as a success. If, after a search, a maximum in the goodness-of-focus function is found which is less than this value then the procedure is considered a failure even though a peak was found. This is so false peaks / local maxima can be avoided. The optimum value for this criterion will be found by empirical methods because it may vary by the kind of specimen / object being imaged as well as magnification and imaging modality (i.e. how much contrast can be expected in a focussed area) and the goodness-of-focus function used.

<timeout> This is as an integer in milliseconds. If the AF search takes longer than this to complete it will fail. Use 0 if you do not want a timeout to be used.

Note: All positions / coordinates / distances are expected to be supplied in either calibrated units (if you set act_unit to >0 for steppers) or in atomic steps (1/maxmsdenom microsteps). Positions are measured from 0 (= the homed position) and no negative values are allowed. Because a stepper motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply here for distances and positions will be rounded to the nearest *onestep* value (see p.54). For example, when using atomic steps, if you set <max_difference> to 15 when

maxmsdenom in 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) then this will result in an effective `<max_difference>` of 0 because (uint32_t)(0.5 + (15.0 / 32.0)) = 0. However if you set `<max_difference>` = 16 then the effective value will be 1 because (uint32_t)(0.5 + (16.0 / 32.0)) = 1.

af_setup

`af_setup <initial_focus_z> <initial_step_size> <max_retries> <dirn_up> <col_channel> <failure_action>`

This sets all but the convergence criteria parameters and focus function for the autofocus (AF) procedure. Use this command in conjunction with the `af_ccrit` and `prv_toggle` commands to fully set up an AF search system which can then be called via the `autofocus` command. Autofocus is a Z motor-specific facility provided as part of the PARDUS standard. See the `autofocus` command for more details about the actual procedure.

`<initial_focus_z>` - a varval of type float and is the initial Z axis position (coordinate) of current best focus (must be >= 0.0). After this initial setting, this value is automatically updated every time the autofocus action succeeds so the value supplied here is just the initial value used prior to the first autofocus success. If the autofocus fails the Z axis moves to the last AF-successful value of this position (or to the value provided here if AF has not had any success since its initial attempt) as the fall-back default focus position.

`<initial_step_size>` - a varval of type float and must be positive (> 0.0). It is the search step distance for Z motion in the hill-climbing search (not to be confused with motor step size).

`<max_retries>` - a varval of type int that must be a >= 0. If the AF search fails first time round it may succeed if repeated so this specifies how many times to repeat before finally giving up.

`<dirn_up>` - An integer representing the direction that moves the Z axis up (must be 1 or 0)

`<col_channel>` - An integer representing which colour channel to use for the focus statistic. These are #defined in pardcapdefs.h as the 'CCHAN_' values. Currently only 2 (= Red), 3 (= Green) and 4 (= blue) are allowed.

`<failure_action>` - a string keyword that determines how to proceed if the iterative AF ultimately fails. This is one of: ignore, exit.

- `ignore` - this just carries on regardless - the last best value for focus will be used as best focal position and the program will continue. This can be perfectly legitimate if, for example, the auto-focus attempt took place over an area with no relevant object in the field of view of the camera (e.g. blank area of slide for microscopy).
- `exit` - this will terminate the program immediately.

Note: All positions / coordinates / distances are expected to be supplied in either calibrated units (if you set `act_unit` to >0 for steppers) or in atomic steps (1/maxmsdenom microsteps). Positions are measured from 0 (= the homed position) and no negative values are allowed. Because a stepper

motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply here for distances and positions will be rounded to the nearest *onestep* value (see p.54). For example, when using atomic steps, if you set `<initial_step_size>` to 15 when maxmsdenom in 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) then this will result in an effective `<initial_step_size>` of 0 because (uint32_t)(0.5 + (15.0 / 32.0)) = 0. However if you set `<initial_step_size>` = 16 then the effective value will be 1 because (uint32_t)(0.5 + (16.0 / 32.0)) = 1.

Note: The focus function used for AF is selected using the frame grabber settings commands such as `prv_toggle` with the 'fc_...' (focusser statistic) options. See `prv_toggle` for details.

autofocus

`autofocus`

This causes the Z axis to perform an autofocus iterative hill-climbing focus-function maximisation search that involves taking a preview image, measuring its goodness-of-focus value with a focus-function then moving the Z motor (which is assumed to control focussing motion for this to work) and iterating to find the Z position with the best (maximum) value of the focus function.

This command expects no arguments because all the autofocus settings are set using the `af_setup`, `af_ccrit` and `prv_toggle` commands (which see). These should be called prior to first use of `autofocus`.

Note: If tissue content thresholding is enabled, an autofocus Z-search will only be performed if the tissue content threshold is passed (i.e. if there is tissue detected in the field) - otherwise the autofocus function will exit reporting AF_SUCCESS but without actually doing anything (it is not an error).

average_scale_means

`average_scale_means <ON|OFF>`

This determines whether (ON) or not (OFF) to scale the mean of any image (captured as part of a multiframe average series) to the mean of the first image captured in the series.

`<ON|OFF>` - a binary choice varval.

call_func

`call_func <function_label> [if <varval>]`

This calls a function. This command can be simple (i.e. just go to the function) or extended as a conditional call (i.e. only go to the function if some condition is satisfied).

<function_label> - this is a string varval. The string must be a label that is part of a def_func command.

if - If an extended (conditional) function call is required then the second argument must be the word if and must be followed by a varval of a type described below.

<varval> - a binary choice varval. If the value of <varval> evaluates to a non-zero numerical value or to TRUE or ON then the script interpreter will immediately go to the line in the script where the function is defined. Otherwise the script will continue execution from the next coding line immediately following this call_func line.

Note: The argument for call_func can be a string variable meaning that the same call_func command in a script can be used to call different functions if the script sets the string variable argument to different values (different function labels) according to some algorithm. That is, <function_label> does not have to be a hard-coded string. For def_func the <function_label> argument must be a hard-coded string, not a variable.

Note: Inside the interpreter program this does two things:

1. It creates a new element on the RetPoint[] stack pointing to just after the call_func line in the script.
2. It sends control of flow to the function defined as <function_label>.

AI Tip: For high-performance state machines, store your destination function labels in a string array. This allows you to dispatch control flow using an index (e.g., call_func state_labels[idx]). This 'Jump Table' approach is more efficient for LLMs to manage than complex branching logic.

camera_set

```
camera_set <Control_ID> <varval>
```

This requests a change to a setting (called a 'control' in v4l2 terminology) on a v4l2 imaging device connected to the server (such as brightness, contrast, etc. - whatever user-alterable controls the imaging device may have).

<Control_ID> - a varval of type int that must be >= 0 and be the v4l2 ID number of a writeable video device setting (the setting you want to change). Control ID numbers are those retrieved by the get_camsettings command (which see).

<varval> - a varval of type int representing the new value you want the setting to change to. This value must be in the permitted range for the setting as defined by maximum and minimum values shown in the camera settings information (retrieved by the get_camsettings command).

cat_char

```
cat_char <STR_VAR> <char_as_int>
```

This allows you to add a single UTF-8 Unicode compatible character to the end of a string variable by specifying the character's integer reference number.

<char_as_int> - must be an integer in the range 0 to 1114111 inclusive. This may be supplied in an integer variable or as an integer constant or a value that may be cast as such.

Note: For this command (only) you may supply the <char_as_int> integer constant in a UTF-friendly format that begins with 'U+' or '0x' (i.e. you may specify constants in hex format (e.g., 0x41) or Unicode format (e.g., U+1F600)). The checks for those start characters are applied to the literal text you use for the argument you pass in the script so if you have made a user variable with a name that starts with 'U+' or '0x' then supplying the name of that variable for the argument <char_as_int> will result in an error - either a syntax error if the full variable name does not comply with a valid Unicode or hex number or it will pass the syntax check but the value specified by the *name* of your variable will be used and not the value stored *in* your variable. Thus don't use variables with such names with this command if you want to avoid this behaviour. *Example*: If a user names a variable U+10, then the code:

```
cat_char my_str U+10
```

will add character 16 (hex 10) regardless of the value stored inside the variable U+10.

Note: You must not supply <char_as_int> as a string variable unless that string contains a number in the conventional decimal format (without 'U+' or '0x')

Note: If you want to add a space or a hash character with this command you cannot use the system variables Space or Hash because those are strings, but you may use this command to the same effect by supplying <char_as_int> as 32 (for space) or 35 (for the hash character: #).

cat_str

```
cat_str <STR_VAR> [[str1] [[str2] ...]]
```

This allows you to add a list of white-space separated words to the end of a user-defined variable of 'string' type.

<STR_VAR> - the string variable to be modified

[[str1] [[str2] ...]] - a list of between 0 and 24 string constants separated by white space. These will be added to the end of (concatenated to) the existing contents of <STR_VAR>

Note: Up to 24 <STR_VAR> string arguments may be used in one command. If no [str] arguments are provided the command has no effect. The limitations on what arguments are permitted are the same as for the command set_str (which see).

Note: Each [str] argument will be appended to the <STR_VAR> separated by a single space (ASCII character 32) regardless of whatever type or number of white space character(s) was used to separate arguments in the command. No space will be added between the initial value of <STR_VAR> and the first [str] argument. No space will be printed to the end of the list of argument strings.

Note: Each `[str]` argument will be treated as a string constant and not a variable (of any data type). If a variable name is included in these arguments it will be treated just as a string and will not be evaluated (so the variable's name will be concatenated 'as is'). If you want to add the value contained in some variable to the end of a string then use the `add_var` command instead (which see).

Note: The interpreter has a limit on the maximum size of any argument. This is set by the MAX_ARGLEN value in the interpreter source code which, at the time of writing is 64 bytes. It is important to note that bytes do not necessarily equate to characters. While standard ASCII characters are 1 byte each, UTF-8 Unicode characters (such as box-drawing symbols or emojis) typically consume 3 or 4 bytes each. For example, the ═ symbol uses 3 bytes; therefore, a single argument can contain a maximum of 10 of these characters before exceeding the 64-byte limit.

Caution: Do not exceed MAX_ARGLEN bytes per argument. If a string exceeds this limit, the interpreter will truncate it at the byte level. This may result in **malformed UTF-8 sequences** for Unicode characters, likely causing the string to fail to render. For ASCII text, the string will simply be truncated.

corrections_mask

`corrections_mask <OFF|ON|image_file>`

This determines whether to load and use a corrections mask for flat field and dark field correction during full frame image capture (separate commands exist for preview images).

`<OFF|ON|image_file>` - a string varval. This must evaluate to either the (case sensitive) strings `ON` or `OFF` or the file name (optionally to include the full path) of an image to be used as the mask image. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: If you select to supply an image file for the mask it only loads the file and passes it to the server. It does not set the 'Use mask' flag to ON. Whether to use that file or not depends on you issuing this command again with a specific `OFF` or `ON` argument (it defaults to `OFF` before you first make a definitive choice).

Note: Permitted file formats for a corrections mask are pgm (P5), ppm (P6) or bmp (8bpp or 24bpp). For a file name to be acceptable it must have a ipgm, .ppm or .bmp extension (case is not important here).

Note: More detail about the use of a corrections mask can be found in the pardcap user manual.

def_actuator

`def_actuator <act_type> <act_index> <name_string>`

This defines a new actuator on the pardserver. The actuator structure will be formed in the nascent state with most of its members set to 0 or 'unset' or NULL but it will be given the name you supply here.

`<act_type>` - an integer defining the type of actuator you are attempting to create. It must have a value from 1 to 4 inclusive where:

1 = Stepper

2 = Binary

3 = Mlevel

4 = Servo

`<act_index>` - an integer representing the actuator index of the actuator you are attempting to create. This which must have a value from 0 to the maximum number of allowed actuators of the type specified in `<act_ype>`, minus 1. That maximum number is specified in the source file parddefs.h and is 10 for all actuator types at the time of writing this manual so the permitted range is 0 to 9 inclusive.

`<name_string>` - a string that will uniquely identify the newly created actuator. This must not have spaces and must be less than or equal to 31 printable bytes. if you supply a name with more characters then only the first 31 printable bytes will be used. Remember that these are bytes which are not always the same as 'characters' - for example if you use UTF-8 Unicode then each character is 3 bytes long so counts as 3 bytes to your 31 byte limit.

Note: Upon successful completion the system variable `CurrActIdx` will be set equal to the index number of the newly created actuator and `CurrActType` will be set equal to the value you supplied as `<act_type>`.

Note: You will not be allowed to create an actuator with a name that is identical to the name of an existing actuator (only the first 31 chars will be checked for identity) or to the name of a user-defined or system variable.

Note: You will not be allowed to define an actuator with an index number that is identical to an existing actuator of the same type as the type you are attempting to create.

Note: The local list of all defined actuators is automatically undefined at the start of each script check and script run session. This does not undefine any actuators that may have been set up on the pardserver and Arduino by previous script commands during any previous login as well as the current login (only when the Arduino is reset / reboots are those cleared). Thus it is good practice to call the command `get_actuators` at the start of any script to populate the locally held lists of actuators with whatever happens to be defined on the Arduino or, if you want to start from scratch, you can call the `undef_actuator` command in wildcard mode for each actuator type to ensure you have a clean sheet to work from.

Note: `def_actuator` and `act_getname` are the only two actuator commands that require you to input two separate numbers to identify an actuator instead of the single `<act_id>` argument (the latter is described under `undef_actuator`).

Note: A nascent actuator, such as is created by this command, will not be usable until it is 'matured' by the application of a series of commands to set a range of mandatory properties (using `act_set`). The rules as to what constitutes the 'range of mandatory properties' will differ for each type of actuator. Any attempt to use activation commands on an immature actuator will result in a script terminating with an 'immature activator' error message.

Note: For actuators of type 'Stepper', the range of mandatory motor struct members that must be set before it will be marked as 'mature' (and so be responsive to any kind of motion commands) are:

- In all cases:
  - `pin_pul`
  - `pin_dir`
  - `pin_ena`
  - `dirn1`
  - `dirn2`
- and at least 1 homing maturity condition:
  - If you want to do homing to a limit switch:
    - `pin_lim1` and/or `pin_lim2`
  - If you want to do null homing you must set these members to a non-zero value:
    - `nhs1`
    - `nhs0` and/or `nhs2`

In the pardserver, each one of these essential settings is contributes a maturity point of 1 to the motor struct's `maturity` member. Since there are 6 essential settings, an actuator will only be considered mature by the pardserver if the total maturity score is >= 6. This maturity score only acts an an initial guard - you can still 'break' a motor's ability to work at a later stage by, for example, undefining one of its essential pins without re-defining it to an appropriate pin before next motion command attempt. If you do this and then try to move the motor the script will terminate with an error from the pardserver telling you why.

Note: For a newly created stepper actuator, once mature, the only action that it can be commanded to perform is some form of homing - either homing to a physical limit switch (in which case you must have defined pin_lim1 and/or pin_lim2 depending of which limit switch you want to home it to) or null homing (in which case you must have set at least 2 of the nhs members to be >0 as described above). Only after it is successfully homed can other motion commands be enacted.

Example: Here is an example PCS script that defines a stepper motor actuator called 'X_Axis' and matures it up ready for homing:

```
Pardus pcs 1
verbosity 3
#include ppinclude.pch   # This contains the integer values mapped to the
                         # capitalised tokens used in commands below. Without
                         # such a header you would not be able to use (e.g.)
                         # act_set CurrAct INFO_PIN_PUL  PUL_X
                         # but would have to write something like this instead
                         # act_set CurrAct 19 22
def_func main
 get_actuators  # Update the local list of extant actuators from the server
 def_actuator 1 0 X_Axis  # If this succeeds it will automatically set
                          # CurrAct to the ID of the newly formed actuator
                          # but if it fails the script will terminate with an
                          # error automatically.
 # All these are mandatory for maturity
 act_set CurrAct INFO_PIN_PUL  PUL_X
 act_set CurrAct INFO_PIN_DIR  DIR_X
 act_set CurrAct INFO_PIN_ENA  ENA_X
 act_set CurrAct INFO_DIRN1 DIRNLFU
 act_set CurrAct INFO_DIRN2 DIRNRBD

 # Only one of these is essential for maturity (assuming we will be homing
 # to a physical limit switch) but I chose to define them both:
 act_set CurrAct INFO_PIN_LIM1 OPTO_XL
 act_set CurrAct INFO_PIN_LIM2 OPTO_XR

  # The motor is now ready to be homed but I will define the diag pin so we
  # can make use of the TMC2209 StallGuard4 feature - this is optional
  act_set CurrAct INFO_PIN_DIAG DIAG_X

return
exit
```

By the end of this script, the X_Axis motor may legally be homed to one of its two physical limit switches. Only after successful homing can other motion commands be accepted. Note that you do not need to do the homing itself here - the changes you make with this script will be enacted by the pardserver and parduino to populate the motor struct. These changes will persist even after the script exits and even after you log off the server. This is one of the reasons why you should always perform a `get_actuators` command at the start of each script - so you know what actuators have already been defined. You can also use the `act_get` command with the `INFO_MATURITY` integer value argument to read how mature the motor is (the value must be 6 to be mature enough to home).

def_array

`def_array <vartype> <varname> <size> <initial_varval>`

This creates and initialises a user-defined 1-dimensional array of variables.

`<data_type>` - the data type of the elements in the array. It must be one of these: int, float, string.

`<varname>` - a string constant defining the name of the array to be created. This is case sensitive and must not be already taken by an extant variable or array, whether a system variable or user-defined variable or array (no two arrays can have the same name as each other or as any single variable), it must not contain spaces or square brackets and must not be a protected name (see p.30).

`<size>` - a varval that evaluates to the number of elements in the array. It must evaluate to an integer >= 1.

`<initial_varval>` - the value to initialise every element in the new array to. This must be a value that is compatible with the data type provided as the `<data_type>` argument.

Note: All arrays are global but can only be used from the point in the script at which they are defined. They can no longer be used ***as variables*** after they are destroyed by a call to `undef_var` (which see). Thus arrays can have manually managed temporal scope. See p.24 for more details on variable scope.

Note: The qualifier '***as variables***' in the note above is important because you may still use the name of an undefined (non-existent) variable in a script but if you do this name (and its dereferencing bracket expression) will simply be treated as a string constant argument rather than a variable that can be evaluated. This might be perfectly legal depending on the context and so might not raise any syntax error during the checking phase or runtime error. However, you may get undesired behaviour during run-time if you do this inadvertently. For example the following code will just print 'myarr[3]' as a string literal and will not print the value '20' - there will be no errors or warning because this is perfectly legal PCS code:

```
def_array int myarr 5 20 # myarr is brought into existence (all values = 20)
undef_var myarr          # myarr ceases to exist as an array variable.
```

```
print_value myarr[3] 2   # 'myarr[3]' is treated as a string literal.
```

If you comment out the `undef_var` line, then the output would be '20'. This works without error because `print_value` does not require its argument to be a variable (although it may be). Other commands have more stringent criteria for their arguments such as `set_var` which expects its first argument to be a variable that can be written to. So this script would not pass even the syntax checker:

```
def_array int myarr 5 20 # myarr is brought into existence (all values = 20)

undef_var myarr          # myarr ceases to exist as an array variable.

set_var myarr[3] 2       # ERROR: myarr[3] is not a writeable variable.
```

Note: PCS supports runtime dynamic allocation for arrays. The `<size>` argument in `def_array` may be a variable that is evaluated at the moment of execution of the `def_array` command. This allows you to create arrays whose size is determined by previous calculations or data inputs. The maximum size of an array is limited only by the available system RAM. However, for high-reliability robotics, it is recommended to keep array sizes within predictable bounds to avoid RAM allocation failure errors at runtime.

Note: The size of an array may also be changed at run time by undefining the array (with the command `undef_var`) and then re-defining the array with the new size. Undefining the array variable will delete all its current contents so if you want to re-size an array without loosing its contents you must first make a backup copy (e.g. in another array variable) before calling `undef_var` and then restore the contents to the newly (re)created array from your backup.

Note: All variables are treated as arrays in PCS so variables created using `def_var` are just arrays of size 1.

Note: Array members are accessed using a square bracket syntax like this:

`my_array[<index_varval>]`

These are the rules for indexing (accessing) members of an array with this system:

- The value of `<index_varval>` must be reducible to a non-negative integer that is within the bounds of the array.
- Array indices always start from 0 (as in the C programming language)
- As per the above rules the legal, 'in-bounds' range for an index value is from 0 to `<size>-1` inclusive. Anything more than this will results in an error that will terminate the script.
- The `<index_varval>` may be a constant number or a variable, including another array variable with an index of its own.

- The value of `<index_varval>`, if not natively of type int, will be automatically attempted to be cast to an integer using the PCS rules of (casting described on p.14). This means that even string variables may be used to index an array provided that the string value can be cast to an integer with a value in the bounds of the array.
- It is forbidden to use expressions as an index value. For example, the following are illegal and will result in an error that will terminate the script or fail the initial script-checking phase:

```
my_array[3 + 1]
my_array[ind1[idx] != idx]
```

- It is forbidden to use any indexing brackets at all in an array that is of `<size>` 1. This includes all variables created with the `def_var` command. In all such cases the following will be illegal:

```
def_array int my_array 1 0
set_var my_array[0] 3    # This is illegal
# The correct way is simply to use it like a
# a scalar variable, thus:
set_var my_array 3       # This is correct.
```

- It is forbidden to use the name of an array of <size> greater than 1 on its own, without square brackets. So this would be illegal:

```
def_array int my_array 5 0
set_var my_array 3    # This is illegal
```

Note: While only 1-dimensional arrays are permitted (i.e. you cannot do things like `my_array[idx][idy]`) you can always use your own raster indexing system to use a 1-dimensional array as if it were a multi-dimensional array. Just bear in mind that any calculations of the index value must be done separately and stored in a variable before using that index value in the array brackets. For example, to implement an equivalent of a 2D array of size2d=height*width, you can create a 1-D array of `<size>` = size2d. Then, where idy is the y coordinate and idx is the x coordinate, you may access the value at position (x,y) in the array by doing something like this:

```
set_var posxy idy
mul_var posxy width
add_var posxy idx
set_var my_array[posxy]  # Here, posxy = y*width + x
```

Note: In general, problems may occur if you use `def_var`, `def_array` or `undef_var` inside loops or blocks of code that are conditionally accessed (by `do_if`, `else` or any block that is the target of a conditional jump etc.). Such usage is not forbidden and may well pass the syntax checking phase (where conditions are not evaluated and jumps or loops are not enacted) but may results in errors or unforeseen behaviour during run time. Sometimes such placement may be desirable but it is left to the programmer to determine when this is appropriate according to their intended logic. For

example, it is almost never a good idea to use these commands inside a `loop_while` loop but it might be acceptable if you need to read large amounts of data into an array from a file and the size of that array might need to change on each iteration of the loop. In such cases using `def_array` for dynamic allocation of the array to the appropriate size followed, after reading and using the data, by `undef_var` before the loop repeats might be a resource efficient way to code this and would be perfectly safe provided the loop can never be exited by a non-local jump that comes between `def_array` and `undef_var`.

Note: Because all variables, even single variables, can be created with `def_array`, this makes `def_var` superfluous. While `def_var` is technically superfluous, it is kept as an option because it makes the PCS script coder's intention clear and makes it clear that any variable defined with it cannot be indexed with square brackets.

def_func

`def_func <function_label>`

This defines the entry point line to a function.

`<function_label>` - a string constant (not a variable), which must be unique to any function in the script and must not contain white space.

Note: There is no equivalent of 'declaring' a function separately to defining it. The `def_func` line

must be the first line of the actual function itself.

Note: Every `def_func` command starts a code block that must be terminated with one - and only one - `return` command. In between the `def_func` line and its `return` line you may have any legitimate script text except that you cannot have another `def_func` command (this implies that functions cannot be nested inside other functions).

Note: If a script contains any functions at all it must define a function called `main` before any other function is defined. When `main` returns, the script ends.

def_var

`def_var <data_type> <varname> <varval>`

This creates and initialises a user-defined variable.

`<data_type>` - the data type of the variable and must be one of these: int, float, string.

`<varname>` - a string constant defining the name of the variable to be created. This is case sensitive and must not be already taken by an extant variable or array, whether a system variable or user-defined variable (no two variables can have the same name), it must not contain spaces or square brackets and must not be a protected name (see p.30).

<varval> - the value to initialise the new variable to. This must be a value that is compatible with the data type of the new variable.

Note: All variables are global but can only be used from the point in the script at which they are defined. They can no longer be used ***as variables*** after they are destroyed by a call to undef_var (which see). Thus arrays can have manually managed temporal scope. See p.24 for more details on variable scope.

Note: The qualifier '***as variables***' in the note above is important because you may still use the name of an undefined (non-existent) variable in a script but if you do this name will simply be treated as a string constant argument rather than a variable that can be evaluated. This might be perfectly legal depending on the context and so might not raise any syntax error during the checking phase or runtime error. However, you may get undesired behaviour during run-time if you do this inadvertently. For example the following code will just print 'myvar' as a string literal and will not print the value '20' - there will be no errors or warning because this is perfectly legal PCS code:

```
def_var int myvar 20   # myvar is brought into existence with a value 20
undef_var myvar        # myvar ceases to exist as a variable.
print_value myvar 2    # myvar is treated as a string literal.
```

If you comment out the undef_var line, then the output would be '20'. This works without error because print_value does not require its argument to be a variable (although it may be). Other commands have more stringent criteria for their arguments such as set_var which expects its first argument to be a variable that can be written to. So this script would not pass even the syntax checker:

```
def_var int myvar 20   # myvar is brought into existence with a value 20
undef_var myvar        # myvar ceases to exist as a variable.
set_var myvar 2        # ERROR: myvar is not a writeable variable.
```

Note: All variables are treated as arrays in PCS so variables created using def_var are just arrays of size 1. Arrays of size 1 (whether created by def_var or def_array) cannot use square brackets to 'dereference' or 'index' them - i.e. you cannot do this: my_var[0]. See the section on def_array for more about indexing rules.

Note: In general, problems may occur if you use def_var, def_array or undef_var inside loops or blocks of code that are conditionally accessed (by do_if, else or any block that is the target of a conditional jump etc.). Such usage is not forbidden and may well pass the syntax checking phase (where conditions are not evaluated and jumps or loops are not enacted) but may results in errors or unforeseen behaviour during run time. Sometimes such placement may be desirable but it is left to the programmer to determine when this is appropriate according to their intended logic. For example, it is almost never a good idea to use these commands inside a loop_while loop but it might be acceptable if you need to read large amounts of data into an array from a file and the size

of that array might need to change on each iteration of the loop. In such cases using `def_array` for dynamic allocation of the array to the appropriate size followed, after reading and using the data, by `undef_var` before the loop repeats might be a resource efficient way to code this and would be perfectly safe provided the loop can never be exited by a non-local jump that comes between `def_array` and `undef_var`.

Note: While `def_var` is technically superfluous, it is kept as an option because it makes the PCS script coder's intention clear and makes it clear that any variable defined with it cannot be indexed with square brackets.

df_correction

```
df_correction <OFF|ON|image_file>
```

This determines whether to load and use a dark field correction image for full frame image capture (separate commands exist for preview images).

`<OFF|ON|image_file>` - a string varval. This must evaluate to either the (case sensitive) strings `ON` or `OFF` or the file name (optionally to include the full path) of an image to be used as the dark field correction image. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: If you select to supply an image file this command only loads the file and passes it to the server. It does not set the 'Use df correction' flag to ON. Whether to use that file or not depends on you issuing this command again with a specific `OFF` or `ON` argument (it defaults to `OFF` before you first make a definitive choice).

Note: A dark field correction image can only be of raw doubles format but may be mono (specify the a file name ending in _y.dou or _i.dou) or colour (specify one file name of a triplet that end in _r.dou, _g.dou, _b.dou. For example if you have these files on disc: mydfimg_r.dou, mydfimg_g.dou, mydfimg_b.dou then you could use mydfimg_g.dou as your file name argument here.). Ensure each file has its associated .qih external header file also on disc.

Note: More detail about the use of a dark field image can be found in the pardcap user manual. More information about raw doubles format images (the .dou and .qih conventions) can be found in the Biaram user manual.

div_var

```
div_var <varname> <varval>
```

This divides the value of a user-defined variable by a value.

`<varname>` - The name of the variable whose value you want to change

`<varval>` - The value (or variable containing the value) you want to divide <varname> by.

Note: This is symbolically equivalent to `<varname> /= <varval>` in C.

Note: Divide by zero will result in script termination with an error.

do_if

`do_if <varval>`

When the script encounters `do_if` command it evaluates the `<varval>` argument. If that has a value of 0 the program jumps to the associated `else` line if one exists otherwise it jumps directly to the associated `enddo` line.

`<varval>` - a binary choice varval. If it evaluates to 0 (or OFF or FALSE) then the `do_if` block is skipped. If it evaluates to non-zero (or OFF or FALSE) then the `do_if` block is entered.

Note: It is often useful (but not mandatory) to use the `do_if` functionality in combination with conditional expressions. See `set_condexp` for more information on how to use conditional expressions.

Note:

- Every `do_if` must have one, and only one, associated `enddo` command.
- Every `do_if` may (optionally) have one, and only one, associated `else` command.
- An `else` must always come before the line containing the associated `enddo`
- You can have nested `do_if` block up to a maximum of INT_MAX deep if your available computer resources allow it.

Note: The main difference between a `do_if` and a `loop_while` is that `do_if` specifies a one-off execution of its block of code but a `loop_while` must always re-assess its condition before moving on (unless the user breaks out of the loop with a `goto` which is one way how a `loop_while` can be used as a kind of `do_if` - but without the added convenience of an optional associated `else`). So, while all the logical branching functionality of the if/else system can be achieved with `loop_while` plus other lines of code, the if/else system does the same thing but with fewer lines of code and makes the control of flow easier to follow by a human reader.

else

`else`

This is an optional component of the PCS `do_if` ... `enddo` code block. See `do_if` for details.

enddo

`enddo`

This is a mandatory component of the PCS `do_if` ... `enddo` code block. See `do_if` for details.

endloop

`endloop`

This is a mandatory component of the PCS `loop_while` ... `endloop` code block. See `loop_while` for details.

execute_daemon

`execute_daemon <progname> [<arg1> ...]`

This allows the user to execute a custom command or external program on the client computer asynchronously. This method allows the command to execute without the script stalling or waiting for the command to complete execution, facilitating rapid parallel processing (the client computer processes the background task while the script carries on running).

It is managed natively by an internal, asynchronous process queue within the PARDUS interpreter engine. It limits the number of simultaneous parallel background tasks to the current value set by the `max_daemons` command (which see) to safeguard the CPU and memory from being overloaded. If the execution pool is full, extra commands are placed securely in a lightweight memory queue and execute automatically the microsecond an active background task finishes.

`<programe>` - a string which is the name of the executable file of the command to pass to the operating system's process manager.

`[<arg1> ...]` - an optional list of space-separated arguments for the program to be executed. If any of the arguments to be passed is the name of the most recently saved image, this must be specified within the argument list as `[img]` (case sensitive and including the square brackets). The PARDUS script interpreter will automatically replace `[img]` with the absolute, clean path name of the last saved image. If tissue content thresholding is active and no image was captured due to lack of tissue content, the `[img]` symbol will be replaced with the literal text `NoTissue`. Your external program should check for this specific string to handle tissue-less iterations gracefully.

Note: For example if the name of the last saved image was myframe_0342_rgb.bmp in folder C:\msys64\home\parduser and you specify the execute_daemon command as this:

```
execute_daemon myprog.exe  -i [img] -o outfile.txt
```

then the Pardus script processor will convert that into:

```
myprog.exe -i C:\msys64\home\parduser\myframe_0342_rgb.bmp -o outfile.txt
```

and pass that to the Pardus daemon. If tissue thresholding prevents an image being captured then it will be:

```
myprog.exe -i NoTissue -o outfile.txt
```

Note: On MS Windows (even when running from an MSYS2 MINGW shell) do not use "./" i.e. NOT "./prog.exe", or any other *nix-style paths but use Windows style paths or just "prog.exe" if the program to execute is in the same directory as the PARDUS client program. Full paths to the executable must be specified in the MS Windows style i.e. C:\msys64\home\parduser\prog.exe

execute_system

`execute_system <progname> [<arg1> ...]`

The `execute_system` command attempts to execute a custom command directly via the host operating system. Unlike `execute_daemon`, this command runs synchronously: script execution will pause while the target program executes, continuing only after the program terminates. No checks are made as to whether the custom command executes correctly or exits with an error. However, upon completion, the PCS system variable `Sysval` will be assigned the integer exit code returned by the executed program. Script authors can read Sysval to handle program success, failure, or custom termination states. If the program fails to start `or` crashes abruptly, `Sysval` will be assigned a value of -1.

`<programe>` - a string which is the name of the executable file of the command to pass to the command processor

`[<arg1> ...]` - an optional list of arguments for the program to be executed. The details are the same as for the command `execute_daemon` (which see).

Note: On MS Windows (even when running from MSYS2 MINGW shell) do not use "./" i.e. NOT "./prog.exe", or any other *nix-style paths but use Windows style paths or just "prog.exe" if the program to execute is in the same directory as the PARDUS client program. Full paths to the executable must be specified in MS Windows style i.e. C:\msys64\home\parduser\prog.exe

Note: Unlike older legacy engines, this command executes binaries directly without spawning an intermediary shell processor (like /bin/sh or cmd.exe). This significantly increases execution speed and completely eliminates shell-injection vulnerabilities.

exit

`exit`

This terminates the script. All scripts and header files must have an `exit` command to mark the end of coding lines. The script checker will reject the script or header file if it does not find one. An

exit command must be present even in scripts that use formal functions where the return of the function main will effectively act as an exit point at run time.

fbyte_mode

fbyte_mode <varval>

This is a latching command. If this is set to ON/TRUE/non-zero then, when integer variables are being dealt with via fset_var (or fput_var), the file will be read (or written) 1 byte at a time and into (or from) the integer variable - each integer variable value will be treated as an unsigned char. In the case of fput_var the lowest 8 bits of the integer variable will be written to the file as an unsigned char. The default value for fbyte_mode at the start of any script run is 0 (OFF).

<varval> - a binary choice varval.

ff_correction

ff_correction <OFF|ON|image_file>

This determines whether to load and use a flat field correction image in full frame image capture (separate commands exist for preview images).

<OFF|ON|image_file> - a string varval. This must evaluate to either the (case sensitive) strings ON or OFF or the file name (optionally to include the full path) of an image to be used as the flat field correction image. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: If you select to supply an image file this command only loads the file and passes it to the server. It does not set the 'Use ff correction' flag to ON. Whether to use that file or not depends on you issuing this command again with a specific OFF or ON argument (it defaults to OFF before you first make a definitive choice).

Note: A flat field correction image can only be pgm (P5), ppm (P6), bmp, or raw doubles (mono or colour). For a file name to be acceptable it must have an extension or name ending appropriate to its format (case is not important here) and must be one of: .pgm, .ppm, .bmp, _y.dou, _i.dou, _r.dou, _g.dou, _b.dou . For more information about the use of raw doubles images see the section on df_correction.

Note: More detail about the use of a flat field image can be found in the pardcap user manual. More information about raw doubles format images (the .dou and .qih conventions) can be found in the Biaram user manual.

fput_var

`fput_var <varname> <append?> <file>`

This reads data from the variable `<varname>` and writes it to a file. The PCS system variable `NWrites` is set to the number of data items written into the file from the variable or to a negative number if there was an error. Note: The `NBytes` system variable reports the total number of raw bytes written to the file during the execution of `fput_var`. It is calculated as the difference between the file position before and after the writing phase.

`<varname>` - the name of the variable to be read.

`<append?>` - an binary choice varval that specifies whether the file to be written to should be created (and that will overwrite and pre-existing file with that name) or appended to. If the append option is selected and the file does not exist then the file will be created.

`<file>` - a string varval that evaluates to the name (optionally to include the full path) of the file to be written to. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the `<file>` argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to access them.

Note: If `<varname>` is an array of size > 1 then ensure you specify the start point with the usual PCS square brackets dereferencing syntax. Data will be read from the array from that point onwards till then end of the array. If `<varname>` is a single variable then only one data item will be read from it into the file (you should not use dereferencing brackets for single variables and that include variables that were defined as an array but with a size of 1).

Note: The file writing process will stop when one of the following conditions is encountered:

- The end of the variable or array is reached
- There is an error accessing or writing to the file

In the case of an file write error, the script will not terminate but the system variable `NWrites` will be set to a negative value. So the programmer should check the value of `NWrites` after each call to `fput_var` for good defensive programming. Here are the meanings of the value of `NWrites` after a call to the `fput_var` command:

-1 = access denied (fopen failed).

-2 = This should never occur.

-3 = file IO error (ferror was set).

-4 = the default value at the start of any script. Thus this will be the value before any call to `fput_var` is made or, after any call to `fput_var`, it means 'Error, not otherwise specified'.

>= 0 = The number of data items written into <file>.

Note: The data type and content of the file will be assumed to be the same as the data type of <varname> according to the following rules:

<varname> is of type string: A 'data item' will be defined as a character string. Thus the file will be written as a text file of strings. Only the data in each string variable is output. If you wish your output to be separated by white space or have a new line character at the end then it is your responsibility to add that separator to the end of your string variable prior to calling the fput_var command or consider using the print_file and print_value commands for more managed (formatted) output of text to a file.

<varname> is of type float: A 'data item' will be defined as double (usually 64 bits on most PCs). The file will then be written as a raw binary file.

<varname> is of type int: A 'data item' will be defined as an int32_t and the file will then be written as a raw binary file. The foregoing is the default behaviour that occurs at the start of any script run. However, the size of a 'data item' may be changed to a single byte by means of the fbyte_mode command. If byte mode is enabled, only the lowest 8 bits (the least significant byte) of each int are written to the file as an unsigned char. This allows for the creation of binary files with byte-level precision.

Note: See also fset_var and fbyte_mode.

frame_number

frame_number <number>

This sets the value of the frame counter number for the file name of the next captured image. Each image captured by the frame grabber will have a number as part of its file name - this is the frame number.

<number> - an integer which sets the frame number. The frame number in the name of a captured image file will start from this value and increment by one with each subsequent image captured (until and unless this value is changed by the using this command again). See the grab_image and xyzs_scan commands for more information on the names of captured image files.

frames_to_average

frames_to_average <number>

This sets the number of frames to average when a grab_image command is executed.

<number> - an integer, >= 1, that specifies the number of frames to average.

frames_to_skip

`frames_to_skip <skip>`

This sets the number of frames for the frame grabber to grab and discard prior to saving a frame when the `grab_image` command is issued. This allows any frames stored in the grabber's frame buffer cache to be expunged to ensure that only the most recently 'seen' frames are grabbed.

`<number>` - an integer, >= 0, that specifies the number of frames to skip.

fset_var

`fset_var <varname> <skip> <file>`

This reads data from a file and puts the values it reads into the variable `<varname>`. The PCS system variable `NReads` is set to the number of data items read into the variable from the file or to a negative number if there was an error. The `NBytes` system variable is set to the number of bytes consumed from the file after the `<skip>` process. The `NewLines` system variable reports the number of logical line endings encountered on the input stream immediately following a string-based `fset_var` operation. See notes below for more details.

`<varname>` - the name of the variable to be set.

`<skip>` - an integer, >= 0, that specifies the number of ***data items*** to skip.

`<file>` - a string varval that evaluates to the name (optionally to include the full path) of the file to be read. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the `<file>` argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: If `<varname>` is an array of size > 1 then ensure you specify the entry point with the usual PCS square brackets dereferencing syntax. Data will be read into the array from that point onwards. If `<varname>` is a single variable then only one data item will be read into it (you should not use dereferencing brackets for single variables and that include variables that were defined as an array but with a size of 1).

Note: The file reading process will stop when one of the following conditions is encountered:

- The file runs out of data (EOF)
- There is an error accessing or reading from the file
- The variable runs out of space to accept new data

In the case of a file read error, the script will not terminate but the system variable `NReads` will be set to a negative. So the programmer should check the value of `NReads` after each call to `fset_var`

for good defensive programming. Here are the meanings of the value of NReads after a call to the fset_var command:

-1 = access denied (fopen failed).

-2 = file is shorter than the <skip> number of data items.

-3 = file read error (ferror was set).

-4 = the default value at the start of any script. Thus this will be the value before any call to fset_var is made or, after any call to fset_var, it means 'Error, not otherwise specified'.

>= 0 = The number of data items read into <varname>.

Note: The data type and content of the file will be assumed to be the same as the data type of <varname> according to the following rules:

<varname> is of type string: A 'data item' will be defined as a character string separated from any other data items by white space. Thus the file will be scanned looking for strings separated by white space (not by any other delimiters) and each such string will count as 1 data item for the purposes of reading (and also for the purposes of skipping <skip> data items before assignments begin). During the <skip> phase, there is no limit on the size of strings skipped. During the reading phase there is a limit put on the size of each string read into each element of <varname> of 1023 non-white-space characters. If a string is encountered that is longer than this, then the first 1023 characters will be read into the current location in <varname> (i.e. it will read a truncated version of that string) and an error warning will be printed to the script console but the program will not terminate and NReads will not go negative. The rest of the truncated string will be ignored and the next data item in the file will be read to the next position in <varname> (if it is an array of of size > 1). For <skip> purposes (but not reading purposes) the definition of a 'data item' can be modified by the fskip_size command. See below for details.

<varname> is of type float: A 'data item' will be defined as double (usually 64 bits on most PCs). The file will then be treated as a raw binary file and read as such - each 64 bits of the file being read as a double. For <skip> purposes (but not reading purposes) the definition of a 'data item' can be modified by the fskip_size command. See below for details.

<varname> is of type int: A 'data item' will be defined an int32_t and the file will then be treated as a raw binary file and read as such - each 32 bits of the file being read as an int32_t. For <skip> purposes (but not reading purposes) the definition of a 'data item' can be modified by the fskip_size command. See below for details. For reading purposes the size of a 'data item' may be changed to a single byte by means of the fbyte_mode command. If byte mode is enabled, the file is read one byte at a time. Each byte is stored in an individual int variable slot as an unsigned char; the lower 8 bits represent the byte value, while the upper 24 bits are set to 0. This effectively treats each int as a single unsigned 8-bit value. This allows for the precise reading of binary files where the file data is not an exact multiple of 4 bytes.

Note: The value of a 'data item' as described above will only pertain when the `fskip_size` is 0. If the value set by the command `fskip_size` is > 0 then the 'data item' will be measured in units of `fskip_size` bytes, regardless of the data type of the variables to be written to.

Note: The `NewLines` system variable reports the number of logical line endings encountered on the input stream immediately following a string-based `fset_var` operation. It has the following behaviour:

- *Trigger*: `NewLines` is reset to 0 at the start of any `fset_var` call targeting a string variable or array. It is updated only after the reading phase is complete and at least one item was successfully read (`NReads` > 0).
- *Normalization*: The interpreter recognizes Unix (\n), Windows (\r\n), and legacy Mac (\r) line endings. Each represents a single count.
- *Scope*: The count includes all line endings between the last string read and the very next non-whitespace character (the start of the next potential data item).
- *Horizontal Whitespace*: Spaces and tabs encountered during this look-ahead are consumed but do not affect the `NewLines` count.
- *Binary Safety*: The counting process stops immediately if a null character (\0) or any non-whitespace data is encountered, ensuring that subsequent binary data is not accidentally consumed.

Note: Use `NewLines` to detect structural breaks in files where whitespace is otherwise ignored. For example, in PNM (P2/P3) files, a `NewLines` value >= 1 after reading the header can indicate the end of a comment line or the transition to the pixel data block.

Caution (Binary Data): If a binary data block immediately follows a string block, ensure there are no trailing spaces or tabs before the binary data. The `NewLines` look-ahead logic consumes horizontal whitespace (Space/Tab) while searching for line endings. If the first byte of your binary data is value 32 or 9, it will be consumed and included in the `NBytes` count.

Note: The `NBytes` system variable reports the total number of raw bytes consumed from the file during the "reading phase" of the last `fset_var` command. The behaviour is as follows:

What it counts:

- For Numeric Types (int/float): The total bytes read into the variable(s). For example, if `fbyte_mode` is ON, `NBytes` will equal `NReads`. If OFF, `NBytes` will be `NReads` * 4 (for ints).
- For String Types: The total number of bytes consumed starting from the first character of the first string read, through to the last byte consumed by the `NewLines` look-ahead logic. The count includes all data characters, internal whitespace between tokens, and all trailing whitespace (spaces, tabs, \r, \n) consumed while the interpreter was updating the `NewLines` sytem variable.

What it ignores:

- NBytes does not include any bytes skipped during the <skip> phase of the fset_var command.

Note: NBytes is the bridge between ASCII and Binary data. By adding NBytes to a tracking variable after reading a text header, you can determine the exact byte-offset where the binary data begins. This offset can then be used with fskip_size 1 to perform a byte-precise binary <skip> for the next fset_var call.

Note: See also fput_var, fskip_size and fbyte_mode.

fskip_size

fskip_size <nbytes>

This is a latching command. If <nbytes> is > 0 then, whenever a fset_var command is issued, the <skip> value is measured in that number of bytes regardless of the data type of the variable to be set (in other words, the total bytes that will be skipped in <skip>*<nbytes>). Otherwise (if <nbytes> is 0) the <skip> value is measured in 'data items' as described in the description for fset_var. The default fskip_size value at the start of any script will be 0 meaning that 'data items' with be the unit of <skip>.

<nbytes> - an integer varval that must be >= 0.

get_actuators

get_actuators

This returns information about what actuators currently exist (are defined) on the pardserver. This information comprises:

4 integers which will update the value of the 4 system variables: NSteppers, NBinaries, NMlevels, NServos. This allows the script programmer to know how many of each type of server exist.

A name string and an index number for each type of actuator that exists. These identify what specific actuators exist in each type category. With this information you can query the server with an act_get command to retrieve any other information you may need to know for each extend actuator.

This information also lets you know what names and index numbers to avoid when calling def_actuator so as to avoid duplication errors.

Example: The following script calls get_actuators then proceeds to check if specific actuators exist with the names X_Axis, Y_Axis and Z_Axis. It only goes on to issue the act_halt command to those actuators that actually exist - thereby avoiding a fatal error by attempting to address an non-existent actuator:

```
Pardus pcs 1
#include ppinclude.pch
verbosity 1
def_func main

get_actuators    # Get info on all extant actuators on the server

act_getid X_Axis  # act_getid will set  CurrActIdx to -1 if the specified
                  # actuator does not exist - a 'safe' way to check.
set_condexp CurrActIdx >= 0
do_if Condexp
  act_halt CurrAct
  act_set CurrAct INFO_ENA_STATE 0
enddo

act_getid Y_Axis
set_condexp CurrActIdx >= 0
do_if Condexp
  act_halt CurrAct
  act_set CurrAct INFO_ENA_STATE 0
enddo

act_getid Z_Axis
set_condexp CurrActIdx >= 0
do_if Condexp
  act_halt CurrAct
  act_set CurrAct INFO_ENA_STATE 0
enddo
return
exit
```

get_camsettings

`get_camsettings`

This is a pardclient GUI update command. The very first time this is called will be automatically by the pardclient program upon any new login and, at that time, this will gather information from the server on each camera in turn in order to populate the client's internal 'CPlist' of all cameras and their parameters. The script writer will not be able to use this command in that way. Instead, when this command is issued in a script, because all the settings and resolutions data have already been gathered from the server, all it will do is ensure the GUI pages and lists of the client program have their displays and combo lists updated with the information relating to the current camera.

Thus the user should call this command after any call to `select_camera` if they want to ensure the GUI is updated to reflect the values of the camera they select.

Alternatively call this command right at the end of your script to ensure that the GUI is appropriately set if that is important to you. You don't need to call this command at all if all you want to do is run scripts from the script tab and have no intention of using the other GUI tabs for any camera functions.

Note: This command works in conjunction with the `get_resolutions` command. Not having these GUI update commands run automatically with a script-originating call to `select_camera` was a deliberate design decision because users may want to do rapid switching / multiplexing between cameras in a script and wasting time automatically setting GUI controls if the user doesn't need that will just reduce the maximum frequency of multiplexing. Hence these GUI update commands are separate commands to be run if and when the user needs them (when doing a `select_camera` from the Manual GUI drop-down combo list in pardclient, these GUI update commands *are* called automatically each time).

get_preview

`get_preview <ON|OFF> <centre_x> <centre_y>`

This causes the frame grabber to capture a preview image and send it to the client.

`<ON|OFF>` - a binary choice varval. It determines whether the preview is zoomed (`ON`) or not (`OFF`). A zoomed preview is a crop of the full-size image, cropped to the dimensions of the preview image, and so will appear zoomed if the dimensions of the full-size image are larger than the dimensions of the preview image. An un-zoomed preview showing a scaled version of the full-size image, scaled to fit in the dimensions of the preview image whilst maintaining its original aspect ratio.

`<centre_x>` - a varval of type int. This is the centre X coordinate of the crop in the full resolution image for a zoomed preview.

`<centre_y>` - a varval of type int. This is the centre Y coordinate of the crop in the full resolution image for a zoomed preview.

get_preview_focus

`get_preview_focus`

This captures a new preview image and retrieves the RGB focus stats (only) using whatever focus measure was previously set using `prv_toggle`.

The preview image itself is not sent over to the client. This is a more streamlined and quicker function than a call to `get_preview_stats` (which see) because the latter will update all the preview stats values, not just focus. On successful completion of this command, the user-accessible system variables `Ps_Focus_R`, `Ps_Focus_G`, and `Ps_Focus_B` will be updated but the other stats variables will not be updated.

This command is designed for fast access to the latest focus stats for those wishing to implement their own custom autofocus algorithm at script level (instead of using the standard PCS `autofocus` command which operates on the pardserver). See the `autofocus` command for more detail about that.

get_preview_stats

`get_preview_stats <ON|OFF>`

This retrieves stats for a preview image.

`<ON|OFF>` - a binary choice varval. If the argument evaluates to `ON` then it takes a new preview image (using the settings last used to take a preview e.g. zoomed or not and, if zoomed, the last XY coordinates used will be used again this time) and sends over the stats calculated on that new image. If it evaluates to `OFF` then it sends over the stats calculated for the last preview image that was taken (so it assumes that at least one prior call to get_preview was done).

In both cases (`ON` or `OFF`) the preview image itself is not sent over to the client. Thus a call to `get_preview_stats OFF` after a call to `get_preview` should get the stats of the preview image received from that `get_preview` command.

Note: If all you want are the current preview focus stats then use the command `get_preview_focus`. That is a leaner version of this command that will only receive and parse the focus values. See that command for more details.

get_preview_tc

`get_preview_tc <ON|OFF>`

This does one of two things in regards to tissue content (tc) measurements depending on the argument. Regardless of the argument value it takes a new preview image without applying your chosen LUT, Laplacian or median filtering options. It does, however, use other settings that were last used to take a preview e.g. zoomed or not and, if zoomed, the last XY coordinates used will be used again this time and it does carry out your choice of whether to invert intensities. However,

regardless of your colour channel choices for the previous it uses the Y channel data. Using this Y preview image data it then performs one of the following two operations (only using pixels under any preview mask you supplied and only using pixels within the upper and lower histogram limits you previously set if you selected to only calculate variance within limits).

<ON|OFF> - a binary choice varval. If it evaluates to ON it calculates the tissue content value using the NDL (Normalised Differential Laplacian) on the 3x3 median filtered Y pixel data as:

```
tissue_content_ndl = (MALcurr / MALref) * log10(VARcurr / VARref)
```

where MAL = mean absolute Laplacian and VAR is the variance.

It then updates the local system variable, CurrTC, with the result and, if the tc_threshold <lower> value is >= 0.0 (see the tc_threshold command for details), also sets the local system variables NoTissue and HasTissue according to whether CurrTC falls within or on the boundaries set by tc_threshold. The <upper> value set by tc_threshold will only be applied to the thresholding if it is non-negative.

If the argument evaluates to OFF then it performs the 3x3 median filter on the Y pixel data and calculates the MAL and VAR but, instead of using these to calculate the NDL, it simply stores them in the preview stats struct members as the NDL reference values for future tissue content calculations - the values MALref and VARref in the above formula. The local system variables mentioned above will not be altered after this call.

Note: You should call get_preview_tc OFF on a reference field (usually a blank field of illuminated background) before you call get_preview_tc ON in any given login session to a pardcap server after the pardcap server first starts up. Unless the background illumination changes or you move to a new objective or camera there should be no need to call get_preview_tc OFF again.

Note: Unless you call get_preview_tc OFF on, at least once in any given login session to a pardcap server after the pardcap server first starts up, the reference values MALref and VARref will not be set. Their default values when a pardcap server first starts are -1.0 (for each of them). If VARref is negative it means the tc reference values have not yet been set and so tc measurements (let alone tc thresholding) will not be possible.

Note: Any currently visible preview image is not altered, no preview image is sent back to the client and the general preview stats are not updated.

get_resolutions

get_resolutions

This is a pardclient GUI update command. The very first time this is called will be automatically by the pardclient program upon any new login and, at that time, this will gather information from the server on each camera in turn in order to populate the client's internal 'CPlist' of all cameras and their parameters. The script writer will not be able to use this command in that way. Instead, when

this command is issued in a script, because all the settings and resolutions data have already been gathered from the server, all it will do is ensure the GUI pages and lists of the client program have their displays and combo lists updated with the information relating to the current camera.

Thus the user should call this command after any call to `select_camera` if they want to ensure the GUI is updated to reflect the values of the camera they select.

Alternatively call this command right at the end of your script to ensure that the GUI is appropriately set if that is important to you. You don't need to call this command at all if all you want to do is run scripts from the script tab and have no intention of using the other GUI tabs for any camera functions.

Note: This command works in conjunction with the `get_camsettings` command. Not having these GUI update commands run automatically with a script-originating call to `select_camera` was a deliberate design decision because users may want to do rapid switching / multiplexing between cameras in a script and wasting time automatically setting GUI controls if the user doesn't need that will just reduce the maximum frequency of multiplexing. Hence these GUI update commands are separate commands to be run if and when the user needs them (when doing a `select_camera` from the Manual GUI drop-down combo list in pardclient, these GUI update commands *are* called automatically each time).

goto

`goto <label> [if <varval>]`

This causes the script to jump to the specific line in the script that is marked by a goto label. This command can be simple (i.e. just go to the labelled line) or extended as a conditional goto (i.e. only go to the labelled line if some condition is satisfied).

`<label>` - a text string (or a variable containing a text string) where the string is without white space and must be identical to a goto label that is defined somewhere in the script on its own separate line (excluding the terminal colon character - the latter is part of the syntax that identifies the label as a goto label but is not part of the label's name).

`if` - If an extended (conditional) goto is required then the second argument must be the word `if` and must be followed by a varval of a type described below.

`<varval>` - a binary choice varval. If the value of `<varval>` evaluates to a non-zero numerical value or to `TRUE` or `ON` then the script interpreter will immediately go to the line in the script marked by `<label>`. Otherwise the script will continue execution from the next coding line immediately following this `goto` line.

Note: A goto label itself (i.e. the token that marks the destination position in the script of a goto jump) is a string without white space followed immediately by a colon with no space between the label string and the colon - similar to goto labels in the C programming language - and that goto label must be on its own separate line. This goto label must be a string constant and not a string variable and must be unique amongst all other goto labels in the entire script. There is nothing in the

PCS language interpreter rules that prevents you from using a protected name (see p.30) or the name of an extant user variable or actuator for a label but it can make a program potentially confusing to read if you do that.

Note: Using the `Condexp` system variable as the `<varval>` argument in conjunction with the `set_condexp` command allows conditional expressions to dictate the control of flow (see `set_condexp` for detail).

Note: This `goto` command is context-aware. You may jump to any label in the script, even those outside the current function or inside/outside nested blocks. The interpreter automatically manages the following:

Function Stack Management:

If you jump to a label in a calling function (a parent), the interpreter will "unwind" the call stack, automatically closing all recursive or nested function calls between your current position and the target.

If you jump to a "foreign" function (one not currently in your active call path), the interpreter will reset the stack to the main function level. Upon reaching a return in that new function, the script will return to main (or exit if main is complete).

Loop and Block Synchronization:

If you jump out of nested `loop_while` loops or `do_if` blocks, the interpreter automatically resets the nesting counters. You do not need to manually "close" loops you have jumped out of.

If you jump into a loop or block, the interpreter will correctly identify the new nesting level. When an endloop or enddo is reached, it will behave as if you had entered that block normally.

Intra-Function Jumps:

Jumps within the same function block are "stack-neutral" and do not affect your function return path, though they will still synchronize your `loop_while`/`do_if` nesting depth.

Note on Jumping into do_if-else Blocks:

Jumping into the 'If' portion: If a `goto` lands between a `do_if` and an `else` (or `enddo`), the block is marked as entered. When the `else` or `enddo` is reached, the interpreter will treat the block as completed and skip to the end of the structure.

Jumping into the 'Else' portion: If a `goto` lands between an `else` and an `enddo`, the block is marked as not entered. The code will execute until the `enddo` is reached. This is functionally equivalent to the original `do_if` condition having been FALSE.

Safety Warning: Avoid jumping directly into the middle of a conditional block if that block relies on variables that were supposed to be set at the start of the `do_if`. The interpreter will manage the control flow, but it is the programmer's responsibility to ensure the global variable state is valid for the landing point.

Note: If a script contains no formal functions (that is, functions defined by `def_func`) or only the main function then goto labels may be be placed anywhere. If you have more than just the main function defined in a script (even if that / those other functions are never called) then all goto labels must be placed either within a function block (`def_func` ... `return`) or between the very last `return` command and the `exit` command of the script. That is to say, labels cannot exist in "No-Man's Land" (the space between function definitions). The script checker will flag these as syntax errors before the script runs.

Note: The fact that the `goto` command can accept a `<label>` argument that is a variable of the string data type and not just a string constant allows for the implementation of informal 'functions' (subroutines) that can be 'called' from anywhere in the code and that can be made to return either to whence they were called or to anywhere else. This is achieved by altering the string value of the 'return' `goto`'s `<label>` argument just prior to calling the subroutine.

For example, let's say I want a subroutine to do 100 frames of live preview. I can set it up and call it like this:

```
# pre-branching code ....
# Create a string variable to determine where the subroutine returns to
# when done:
def_var string  return_point _
# Set the value of that return point to some valid script goto label
set_var return_point return_point_1
# 'Call' the subroutine by simply going to a label just before it:
goto live_preview
# Set the return point label
return_point_1:
# post-return code ....
# [rest of main program here]  - and we can also call live_preview
# after we set_var the return_point variable to a label (like
# return_point_2, return_point_3, return_point_4, etc.) where each
# return point label is placed just after the call, as we did above]
goto end_of_script  # This is effectively the end of the main program

# All goto-based subroutines start after here and must end with a goto
# that acts as a 'return' command to return control of flow to some
# return point
```

```
# The live preview subroutine entry point goto label
live_preview:
# ... [live preview code]
# Now we return to wherever the string in the variable return_point points
# to (assuming that string is a valid goto label somewhere in this script).
# This subroutine itself might alter the string value contained in the string
# variable 'return_point' so this subroutine does not necessarily have to
# return to the place that called it.
goto return_point

# Any other subroutines go here, then finally we have:

# The exit goto label (note that it would be illegal to place a goto label
# here if the script had more than 1 formal function defined because this
# would be an instance of a goto label being placed outside a
# def_func ... return block).
end_of_script:
exit
```

AI Tip: For high-performance state machines, store your destination labels in a string array. This allows you to dispatch control flow using an index (e.g., goto state_labels[idx]). This 'Jump Table' approach is more efficient for LLMs to manage than complex branching logic.

grabber_predelay

`grabber_predelay <seconds>`

This sets the number of seconds to wait before activating the frame grabber after the command to `grab_image` is given

`<seconds>` - an integer between 0 to 172800 inclusive

Note: A frame capture operation may be a single image capture or a multi-frame averaging series. In the latter case, this delay is applied only before the very first image in the average sequence.

grabber_retries

`grabber_retries <number>`

When commanded to capture an image the frame grabber may fail to deliver an image in the allotted time before declaring a timeout failure. This command sets the number of such failed attempts to ignore (and so just trying again) before finally giving up and reporting an error.

`<number>` - an integer between 0 to 4096 inclusive

Note: The grabber will wait a certain amount of time for each frame in this set before giving up and going onto the next try (see `grabber_timeout`).

grabber_timeout

`grabber_timeout <seconds>`

This sets the number of seconds for the frame grabber to wait for a frame to be delivered from the camera driver after requesting one before it gives up and either retries (see `grabber_retries`) or reports an error.

`<number>` - an integer between 0 to 4096 inclusive

grab_image

`grab_image`

This activates the camera frame grabber with the intent of capturing a full-size image.

This command takes no arguments but the user must have previously prepared the parameters of image capture using these commands: `save_format`, `save_resolution`, `save_root`, `frames_to_skip`, `frame_number` and `frames_to_average`. While the former settings are mandatory before calling grab_image there are many other optional settings as well as described elsewhere in this manual. For example, the use of correction images (dark field, flat field, corrections mask) and your choice for tissue content thresholding (`tc_threshold`) and whether or not to also save a coordinates text file with the image (`save_coords`).

jpeg_quality

`jpeg_quality <number>`

This sets the compression level of files saved in the jpeg format on the client. The lower the number the more the compression (and so the lower the image quality).

`<quality_integer>` - an integer from 1 to 100 (100 = best image quality).

Note: This is a local command (it does not get communicated to the pardserver) and only affects files saved by the client to their chosen location. It does not affect the quality of any JPEG stream coming from the camera at the server end.

loop_while

`loop_while <varval>`

This allows for while-loop functionality. This marks the beginning of a block of code which must be terminated by the command `endloop` (which see). The contents of this block will be executed only if the `<varval>` evaluates to a non-zero value and, if the `endloop` command is reached, the control of flow will return back to its associated `loop_while` test line.

`<varval>` - a binary choice varval. This will be evaluated to test whether the block of code is entered for execution or skipped. If it evaluates to 0 (or OFF or FALSE) then the block is skipped, otherwise it is entered.

Note: While loops may be nested up to a maximum of INT_MAX deep.

Note: The maximum number of while loops allowed in a script is UINT_MAX.

Note: You may use the `goto` command to break out of a while loop block at any stage.

math_var

`math_var <var_receptor> <math_func> [varval]`

This performs a mathematical function, `<math_func>`, on the argument `<var_receptor>`. If the mathematical function requires a second argument it uses `[varval]`. It puts the result into `<var_receptor>` replacing its original value.

`<var_receptor>` - a variable capable of receiving the numerical result and initially containing a numerical value used as an argument for the mathematical function to be performed.

`[varval]` - a numerical constant or a variable containing a numerical value.

`<math_func>` a varval of type string that evaluates to one of the following string values:

```
sin
cos
tan
asin
acos
atan
atan2
sinh
```

cosh

tanh

exp

log

log10

pow

sqrt

ceil

floor

abs

imod

fmod

Note: The option of using a string variable for <math_func> gives the PCS programmer great flexibility because the same math_var line in a script can be made to perform different mathematical operations by varying the contents of the variable used as <math_func> according to some logic.

Note: This command is symbolically the equivalent of doing this: <var_receptor> = math_func(<var_receptor>,[varval]) in C.

Note: The meaning of the functions are as described in the C programming language with the exceptions / modifications that abs will return the absolute value of either an float or an int value input and imod is the integer modulus of the input values equivalent to the '%' operation of the C programming language. In cases where the mathematical function requires two arguments names 'x' and 'y' in the book 'The C Programming Language' by Kernighan and Ritchie, the 'y' argument shall be the [varval] argument specified here. Otherwise the sole argument of the function shall be the <var_receptor> argument supplied here.

Note: The value of <var_receptor> and [varval] shall be cast to a float (double) in the script interpreter if they are not already of float type for all but the imod function (when a cast to an int will be performed if the input arguments are not already int). If <var_receptor> is an int for any of the functions that return floating point values then the result will be attempted to be cast into an int with a 0.5 added for rounding i.e.: int_result = (int)(0.5 + float_result).

Note: An error (such as a range error or failure to cast float to int) will result in script termination with an error message. A syntax error will be called if you specify a <math_func> that requires two arguments but fail to supply [varval]. However, no error will result if you supply the optional [varval] for functions that only require 1 argument - in that case the 'extra' [varval] argument will just be ignored and not used. This allows the use of a string variable for <math_func> where the value of the string in that variable may take on the value specifying any of the allowed mathematical functions that require only one argument on some occasions and two arguments on others.

Note: You can make extensive multi-step mathematical calculations by successive application of these maths primitives, possibly contained in a function (see `def_func`) but if you need significantly more complex or extensive mathematical operations then it might be more convenient to do these in an external program and call it using the PCS commands `execute_system` or `execute_daemon`, which see.

Note: For simple arithmetic, see the commands `add_var`, `sub_var`, `mul_var`, `div_var` and `set_var`.

Note: If using string variables an automatic cast to a numerical value will be done for you provided the string variable contains a string that is consistent with a numerical value. String variables used here must initially contain a numerical string at the time of definition otherwise the script checker will reject it because the script checker will not perform any `set_str` commands in the script.

For example, if you define a string like this: '`def_var string str1 _`' and then go on later to do this '`set_str str1 -1.3`' before using it in a `math_var`, the script checker will only have the original defined string value to work with (that is '_') and so this is not a valid number and so it will raise an error. Even though you did a `set_str` to make it a number, that `set_str` will only be executed during run-time, not check-time, so the checker will only see '_'. To avoid this, define any string variable that may, at any time, be used as a numerical argument as a number at definition - for example like this: '`def_var string str1 0.0`' - then the script checker will not complain.

max_daemons

`max_daemons <number>`

This sets the maximum number of asynchronous background commands (invoked via `execute_daemon` or `xyzs_process` - which see) that are allowed to execute simultaneously on the client host computer. This command serves as a dynamic throttle to protect system memory and CPU cores from thrashing during high-density operations like automated XY scans or tight loops in scripts.

`<number>` - a varval of type int that evaluates to between 1 and 100 (inclusive) specifying the parallel process ceiling.

Note: If a script does not explicitly invoke this command, the interpreter defaults to a value of 1. This forces background commands to run in an orderly, sequential, one-by-one queue.

Note: This parameter can be modified dynamically at any point within a script. If `max_daemons` is increased, the internal queue manager will instantly pump and fire extra pending background tasks to fill the newly opened processing slots. If `max_daemons` is decreased below the number of currently executing processes, running tasks are permitted to finish safely without corruption, but the queue will halt spawning new tasks until the active count drops below the newly defined threshold.

Note: It is advisable to keep the `max_daemons` number to just below your total number of CPU cores so you don't over-work the CPU and end up slowing down the whole system.

milliduration

`milliduration <start_millis> <receptor>`

This calculates the monotonic time in milliseconds from the value provided in `<start_millis>` to the current time (the time the command is issued).

`<start_millis>` - a varval of type int >= 0 representing a monotonic millisecond count

`<receptor>` - a variable capable of receiving an int that will receive the duration time (in milliseconds) between `<start_millis>` and the time the command is executed.

Example:

```
Pardus pcs 1
verbosity 1

def_var string msg _
def_var int idx 0   # Loop index
def_var int millidur 0 # Testing the milliduration command
def_var int startmillis 0

def_func main
  print_clear
  set_str msg --- Starting milliduration test ---
  print_value msg 2

  set_var startmillis Millis
  set_str msg Starting the stop watch at:
  add_var msg Space
  add_var msg startmillis
  print_value msg 2

  set_condexp idx == 0
  loop_while Condexp
      sleep 0.25
```

```
      millidur startmillis millidur
      set_str msg milliseconds elapsed:
      add_var msg Space
      add_var msg millidur
      print_value msg 2

      add_var idx 1
      set_condexp idx < 5
  endloop
return
exit
```

The result of running this will look something like this:

--- Starting millidur test ---

Starting the stop watch at: 763205

milliseconds elapsed: 272

milliseconds elapsed: 527

milliseconds elapsed: 782

milliseconds elapsed: 1036

milliseconds elapsed: 1290

mul_var

`mul_var <varname> <varval>`

This multiplies the value of a user-defined variable by some value.

`<varname>` - The name of the variable whose value you want to change

`<varval>` - The value (or variable containing the value) you want to multiply `<varname>` by.

Note: This is symbolically equivalent to `<varname> *= <varval>` in C.

poll_user

`poll_user <wait> <return> <varval>`

This gets a value from the user input edit box in pardclient and puts it into <varval>.

<wait> - a binary choice varval. If it evaluates to ON then the interpreter will wait for at least one character to be entered into the ext box before allowing the PCS script execution to continue. If it evaluates to OFF then the edit box will be checked immediately and any character or characters in it will be read and processed and the PCS script execution will continue regardless of whether there was anything in the edit box or not.

<return> - a binary choice varval. If it evaluates to OFF then the polling function will not wait for the user to hit the return key on the keyboard before reading the contents of the edit box. If it evaluates to ON then it will only read the contents of the box once return has been hit. However, even if <return> is ON, the script will not wait for the user to enter anything, let alone hit return unless <wait> is also ON. If <wait> is OFF when <return> is ON then the interpreter will simply register that it must not read the contents of the entry box until the user hits return and it will carry on with the script. If, during script execution, the user enters something and hits return then it will set a flag to say return has been hit and if (and only if) the script polls the user again at some point after this will the interpreter read the contents of the entry box. If <wait> is ON and <return> is OFF then the interpreter will wait for the user to enter something in the entry box and, as soon as something has been entered, the interpreter will read it and carry on executing the script - without waiting for the user to hit return. This means that if the user is typing with the keyboard they will only have time to enter one character before the interpreter reads it and carries on. If the user wants to enter multiple characters in this 'automatic' mode then they need to copy the characters into the general clipboard and paste them all in one go to the entry box. This might be inconvenient, in which case the following configuration will help. If both <wait> and <return> are ON then the interpreter will wait for a user to enter something and, furthermore, it won't carry on till the user hits return. This configuration allows a user to manually type however many characters they like (up to a maximum of FILENAMEMAX) at their leisure.

Note: The behaviour described above can be briefly summarised in this table:

<wait>	<return>	Behaviour
OFF	**OFF**	Immediate check; reads input box contents instantly regardless of content.
OFF	**ON**	Continues execution immediately; reads data only if a future poll_user call occurs after the user hits Return.
ON	**OFF**	Pauses execution until at least one character is entered, reading instantly without waiting for Return.
ON	**ON**	Pauses execution until user enters characters and hits Return.

Note: The user can use the GUI direction and focus buttons to provide the input to the entry box if they select the 'Divert to script' check box option on the 'Manual' tab. In this mode the buttons will emit a single character to the entry box when they are clicked, that character being their main label excluding any '[' or ']'. So if you want your script to react to these GUI button click you must set (in

your PCS script) the inputs to respond to the single characters '<', '>', '^', 'v', '+' and/or '-'. This provides a convenient ready-made way to allow the user to use a GUI to interact with a script instead of having to type in or copy-paste in text manually. One advantage of using this method for 'manual' control of the microscope stage motors (for example) is that with an interactive script you can enact a real-time preview while responding to motion button clicks but you can't do that using the direction buttons natively in the 'Manual' tab to move the stage. While the 'Divert to script' option is ticked the outline around the button groups will turn red and the buttons will have no other effect so can't be used to effect motor motion from the Manual tab in the usual way.

Note: After the script interpreter reads the entry box it will always clear its contents.

preview_clear

`preview_clear`

Clears the full VGA preview display area.

preview_file

`preview_file <file_path>`

Sets the file path and file name which will be used to save preview images with the command `preview_save` (which see).

`<file_path>` - a string varval. This must evaluate the name of the file (optionally with full path) to be saved. No white space is allowed. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: You must call this command at least once for any camera from which you wish to save a preview image or the command `preview_save` will have no effect.

preview_save

`preview_save`

Saves the last retrieved preview image to disc as a p6 ppm image. This uses the file name and path set using the command `preview_file` (which see). This saves either the full 640x480 VGA PreviewImg buffer or it saves the PreviewQ buffer that stores the QGVA and QQVGA previews. Which one is saves depends on the preview resolution setting of the camera that is currently selected at the time of issuing this command.

Note: This command will have no effect if the preview save path has not been set for the current camera using the `preview_file` command.

preview_tile

`preview_tile <number>`

This determines (more precisely, 'suggests') whereabouts in the 640x480 main preview area to begin drawing a preview image whenever the `get_preview` command succeeds. The `<number>` you supply here will be valid for previews obtained from the currently selected camera. Thus each camera can have its own `preview_tile` number.

`<number>` - An integer from 0 to 15 inclusive which determines where the top left pixel of the preview image is placed within the standard VGA preview area of the client program. The positions 0 - 15 map to jumps of QQVGA dimensions in X and Y with 0 being the top left corner of the main VGA display area, 1 being (row,col) = (0,160), 2 being (0,320), 3 being (0,480), 5 being (120,0), 6 being (120,160) and so on, in raster fashion, till we get to 15 being position (360,480).

Note: This is called a 'suggestion' because the interpreter will not allow preview images to spill-over the boundaries of the VGA main preview area or to overlap each other. So if the preview image retrieved from the server is of size VGA then the `preview_tile` value will be ignored and the preview image will be displayed starting at (0,0) and filling the whole main preview area. If the preview image from the server is of size QVGA then only `preview_tile` values of 0, 2, 8 and 10 will be honoured - anything else will be overridden with 0 and 0 will be used. If the preview image coming from the server is of size QQVGA then all possible <number> values will be honoured.

print_clear

`print_clear`

This clears the text buffer in the script console, including anything out of sight (i.e. ant text that would have needed scrolling to be seen).

print_colour

`print_colour <R/P> [<G> <B>]`

This sets the foreground colour of any subsequent `print_value` prints to the script console. This is a latching command - the colour you set here remains active until either you change it with another `print_colour` command or the script terminates. At the start of each script run, the default foreground colour for `print_value` output will always be white.

`<R/P>` - an integer varval >= 0 and <= 255 and is the red component of a 24 bit colour or a 256 colour palette entry.

`<G>` - an integer varval >= 0 and <= 255 and is the green component of a 24 bit colour

<B> - an integer varval >= 0 and <= 255 and is the blue component of a 24 bit colour

Note: 8-bit Palette Mode (1 argument): This selects a colour from a fixed RGB332 mapping:

Red: Bits 7-5 (8 levels)

Green: Bits 4-2 (8 levels)

Blue: Bits 1-0 (4 levels)

Note: In 8-bit palette mode a <R/P> value of 0 is black and 255 is white.

Note: 24-bit True Colour Mode (3 arguments): The three input values are the red, green and blue components of a 24 bit colour. This allows for precise control to select over 16.7 million colours.

print_size

```
print_size <point_size>
```

This sets the font size of any subsequent `print_value` prints to the script console. This is a latching command - the colour you set here remains active until either you change it with another `print_size` command or the script terminates. At the start of each script run, the default font size for `print_value` output will always be 12.

<point_size> - an integer varval >= 1 and <= 72.

print_cursor

```
print_cursor <X> <Y>
```

This sets the position of the cursor in the script console such that the next print_value command will commence an overwrite print from that position.

<X> - an integer varval >= 0 which marks the number of character positions from the left

<Y> - an integer varval >= 0 which marks the number of lines up from he bottom

Note: Take care to avoid new lines in your `print_value` command formatting when using this command (unless the effect of a newline is a deliberate design choice on your part).

Note: The print_cursor positioning is non-latching - it will default back to 0,0 (bottom left) after any text is printed at the set coordinates.

print_file

```
print_file <exclusive?> <file_path>
```

This allows the `print_value` output to be diverted to a file. This acts as a behaviour modifier for the `print_value` command (which see) - it does not actually print anything itself.

<exclusive> - a binary choice varval. If this evaluates to non-zero (ON) it will cause any print to file output to be printed to the chosen file in <file_path> only and will block any printing to the console or log file. If this evaluates to zero (OFF) then the print_value output will be printed to both the log file +/ script console as well as to the chosen file.

<file_path> - a string varval that evaluates to the file name (or full path including file name) of the file to be written to (without white space), or the word NULL (all in capitals). If this is not NULL then the interpreter will assume it is a file name and will attempt to print the print_value output to that file. If this is NULL then no attempt will be made to print the print_value output to a file.

Note: Any printing to a file will be done with the 'a' (append) option. This means that if the file does not already exist a new file will be created otherwise any existing file will have the new output appended to it.

Note: The printed string will not automatically have a newline character added to the end of it. It will be the user's responsibility to add any required formatting as per the print_value formatting options.

Note: This command acts as a latching behaviour switch for the print_value command. All print_value commands that follow a print_file command will show the behaviour dictated by that print_file command until another print_file command is issued, at which point the behaviour of all subsequent print_value commands will follow the new behaviour. Thus you can switch on and off printing to a file by issuing a print_file command with a valid file name at the start (to turn on file printing) and, when you have finished printing to file, issue another print_file command with NULL as your file path to switch off printing to file.

Note: Each time you call print_file, any currently open file for printing (from a previous call to print_file) will be closed first before the new file is opened (even if the 'new' file is the same as the previous one). Only one file can be open for printing to at a time.

Note: If the <file_path> is not NULL but is also not openable for writing in append mode, then an information notice will be printed to the script console but the program will continue and no file printing will be done until you call print_file again with a valid, usable <file_path>.

print_value

```
print_value <varval> [format_number]
```

This causes the current value of <varval> to be printed to the script console and/or to a file. Where it prints to is dictated by the arguments to the command print_file, which see. The default behaviour is to print to the script console only.

<varval> - a varval of any type.

[format_number] - a varval which must evaluate to an integer from 0 to 7 (inclusive) and determines the print format. For details see the notes below.

Note: The `[format_number]` conveys the on/off status of following three binary flags: debug, newline and overwrite. These flags have the following meaning:

- *debug*: When on this causes not only the value but also other information about the varval (such as it's name and data type) to be printed.
- *newline*: When on this ensures that the printed string ends in a newline character.
- *overwrite*: This first deletes the last line on the script console and then prints the current message where that line used to be. For this to function as an overwrite printer you must also have the newline flag set to on otherwise the print start position for the current text to be printed will jump up one line each time till it eventually erases all the text that came before it.

These are private flags in the interpreter, they are not variables that the script writer can access by name but they control their on/off status by the value of the single integer they supply for the `[format_number]` argument. This value, when converted to a 3-bit binary number, sets the on/off (i.e. 1/0) states of the three flags in the order [debug][newline][overwrite] like this:

`[format_number]` = 0 i.e. binary 000 = all 3 flags are off

`[format_number]` = 1 i.e. binary 001 = overwrite on and the other two off

`[format_number]` = 2 i.e. binary 010 = newline on and the other two off

`[format_number]` = 3 i.e. binary 011 = newline and overwrite on (only)

`[format_number]` = 4 i.e. binary 100 = debug on and the other two are off

`[format_number]` = 5 i.e. binary 101 = debug and overwrite are on (only)

`[format_number]` = 6 i.e. binary 110 = debug and newline on (only)

`[format_number]` = 7 i.e. binary 111 = all three flags on

Note: If a `print_value` command is issued without the optional `[format_number]` then the behaviour defaults to the last `[format_number]` issued in a previous `print_value` command. If there was no previous use of the `[format_number]` option then the default behaviour is equivalent to `[format_number]` = 6.

Note: The `[format_number]` value will reset itself to 6 before and after every script run.

Note: As noted above, for a single line to overprint in situ you need to have both the overwrite and newline flags on because the overwrite flag tells the console printer to delete all text from the end of the current text buffer up to the first newline character it encounters before printing the current text. If the current text has no newline in it then the delete operation will always go up a line till it finds the newline character of the line above. This 'delete the line above' feature may be useful in cases where you print several lines - e.g. three lines, one for a red value, one for a green value and one for a blue value (like variance for example) and then you want to delete all those three lines before printing the next set of three lines. In that case just print an empty string with no newline and with the overwrite flag enabled 3 times before starting to print your new set of three lines.

Note: The `[format_number]` value will remain at whatever you last set it to, even when using the `print_value` command after that without using the optional `[format_number]` argument.

prv_cdf_use

```
prv_cdf_use <ON|OFF|image_file>
```

Whether or not to apply subtraction of a custom preview colour dark field image. This is not the same as the dark field image used for the full resolution image dark field subtraction.

`<ON|OFF|image_file>` - a string varval. Supply either the (case sensitive) strings `ON` or `OFF` or the file name (optionally to include the full path) of an image to be used as the colour dark field image. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: Acceptable formats for the image file are: PPM (P6) and BMP (24 bpp). No other options are allowed. For a file name to be acceptable it must have a .ppm or .bmp extension (case is not important here).

Note: The `ON/OFF` actions will only work if you have loaded a custom colour dark field image before setting this to `ON` or `OFF`. Submitting a file name will cause pardcap to assess the file and set 'pending' flag to 1 if it is successful. The `ON/OFF` status will be whatever it happens to be at the pardcap end and will not change just by virtue of submitting an image file. So you need to call this function twice the very first time you use it in a script. The first call is to submit your file. Then, if the file was accepted OK, call it again with either `ON` or `OFF`.

prv_fc_posn

```
prv_fc_posn <y>
```

This sets the row (`<y>`) pixel coordinate of the top left corner of the focus assist bar(s) as displayed in the preview image.

`<y>` - an integer in the range 20 to 428 inclusive.

Note: The range above will be the same regardless of what you have set the preview size to be with the command `prv_size`. This is because the position coordinate you supply here will automatically be scaled down at the pardcap end if the preview image is set to be smaller than full VGA.

prv_fc_show

```
prv_fc_show <show_mode>
```

Whether to overlay the focus assist bar(s) on the preview and, if so, which bars to show.

`<show_mode_int>` - an integer determining what to show. The options are:

- 0 = invisible
- 1 = all three RGB channels (only applies if previewing in colour)
- 2 = red channel or mono Y
- 3 = green channel (only applies if previewing in colour)
- 4 = blue channel (only applies if previewing in colour)

prv_hgm_posn

`prv_hgm_posn <x> <y>`

The column (`<x>`) and row (`<y>`) pixel coordinates of the top left corner of the histogram displayed in the preview.

`<x>` - an integer in the range 2 to 382 inclusive.

`<y>` - an integer in the range 2 to 222 inclusive.

Note: The ranges above will be the same regardless of what you have set the preview size to be with the command `prv_size`. This is because the position coordinates you supply here will automatically be scaled down at the pardcap end if the preview image is set to be smaller than full VGA.

prv_hgm_satlim_l

`prv_hgm_satlim_l <R/Y> <G> <B>`

Sets the preview image lower histogram saturation limits for the red (or mono), green and blue channels.

`<R/Y>` - an integer from 0 to 254 inclusive

`<G>` - an integer from 0 to 254 inclusive

`<B>` - is an integer from 0 to 254 inclusive

Note: Any pixels with a value less than the lower saturation limit contributes 1 to the total count of lower saturated pixels in each colour channel.

Note: If, when setting these values, the corresponding upper saturation limit is found to be equal to or lower than the value been set, then that upper saturation limit will be set to the lower limit + 1. No error message will be issued in such cases so it is up to the user to ensure the upper saturation limits are set according to the rule that the upper saturation limit values must be greater than the

lower saturation limit values if they want their chosen upper saturation limit values to be set and not overruled.

prv_hgm_satlim_u

`prv_hgm_satlim_u <R/Y> <G> <B>`

Set the preview image upper histogram saturation limits for the red (or mono), green and

blue channels.

`<R/Y>` - an integer from 1 to 255 inclusive

`<G>` - an integer from 1 to 255 inclusive

`<B>` - is an integer from 1 to 255 inclusive

Note: Any pixels with a value greater than the upper saturation limit contributes 1 to the total count of upper saturated pixels in each colour channel.

Note: If, when setting these values, the corresponding lower saturation limit is found to be equal to or higher than the value been set, then that upper saturation limit requested here will be overridden and set to the lower saturation limit + 1. If this override occurs in any 1 or more of the colour channels then an error message will be issued back to the client of the type 'Failed to apply a setting due to out of range'. To avoid this error the user should set saturation limits for both upper and lower in adjacent calls to `prv_hgm_satlim_l` (first - because this will never generate an error as long as the values supplied are in the specified range) followed by a call to `prv_hgm_satlim_u` ensuring that the values you supply are indeed greater than the values you supplied to the prior call to `prv_hgm_satlim_l`.

prv_hgm_scales

`prv_hgm_scales <R/Y> <G> <B>`

Sets the preview image histogram manual scaling factors to be used (only) when `hgm_nofit` is set to `ON` using the `prv_toggle` command (which see).

`<R/Y>` - a float >= 0.0

`<G>` - is a float >= 0.0

`<B>` - is a float >= 0.0

prv_lut

`prv_lut <LUT_number>`

Sets the preview image look-up-table (LUT) for display.

<LUT_number> is an integer between 0 to 5 inclusive and defines the LUT according to:

- 0 = Linear
- 1 = Inverted
- 2 = Logarithmic (base 10)
- 3 = Exponential (base 10)
- 4 = Square root
- 5 = Square

prv_mask_use

```
prv_mask_use <ON|OFF|image_file>
```

Whether or not to apply a custom preview mask to the calculation of preview stats. This is not the same as whether or not to display that mask over the preview image - for that see the command `prv_toggle`.

<ON|OFF|image_file> - a string varval. Supply either the (case sensitive) strings `ON` or `OFF` or the file name (optionally to include the full path) of an image to be used as the mask image. File names and paths must not contain spaces. This argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: Acceptable formats for the image file are: PGM (p5), PPM (P6) and BMP (8 bpp or 24 bpp). No other options are allowed. If you supply a colour format image then the 'intensity' component of an HSI transform is calculated for each pixel as (R+G+B)/3 and this is thresholded to be an 'on' pixel if it is >127 and and 'off' pixel otherwise. The same threshold is used to binarise grey scale images. For a file name to be acceptable it must have a .pgm, .ppm or .bmp extension (case is not important here).

Note: The `ON/OFF` actions will only work if you have loaded a mask image before setting this to `ON` or `OFF`. Submitting a file name will cause pardcap to assess the file and set 'pending' flag to 1 if it is successful. The `ON/OFF` status will be whatever it happens to be at the pardcap end and will not change just by virtue of submitting a file. So you need to call this function twice the very first time you use it in a script. The first call is to submit your file. Then, if the file was accepted OK, call it again with either `ON` or `OFF`.

prv_m_bias

```
prv_m_bias <number_of_grey_levels>
```

This sets the number of grey levels to add (or subtract) from the preview image after all frames have been integrated (see `prv_m_integral`). This only applies when previewing in monochrome mode.

`<number_of_grey_levels>` - an integer between -(PREVINTMAX-1)*255 and 512.

Note: At the time of writing, PREVINTMAX = 20 which makes the lower limit for `<number_of_grey_levels>` = -4845.

prv_mcor_eject

```
prv_mcor_eject
```

This ejects any loaded custom preview mono dark and flat field correction images.

prv_mdf_set

```
prv_mdf_set <image_file>
```

This loads the supplied dark field corrections image and sets it as active for use with monochrome previewing.

`<image_file>` - a string varval which evaluates to the name (optionally to include the full path) of the image to use. This must not contain white space. This string argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: The image must be of raw doubles mono format with its name ending in _y.dou (case is not important here) and be of the same dimensions as the preview image.

Note: The only way to deactivate its use is to perform a `prv_mcor_eject` command, which see.

prv_mff_set

```
prv_mff_set <image_file>
```

This loads the supplied flat field corrections file and sets it as active for use with monochrome previewing.

`<image_file>` - a string varval which evaluates to the name (optionally to include the full path) of the image to use. This must not contain white space. This string argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: The image must be of raw doubles mono format with its name ending in _y.dou (case is not important here) and be of the same dimensions as the preview image.

Note: The only way to deactivate its use is to perform a `prv_mcor_eject` command, which see.

prv_m_integral

`prv_m_integral <number_of_frames>`

This sets the number of frames to integrate when previewing in monochrome mode.

`<number_of_frames>` - an integer between 1 and PREVINTMAX.

Note: At the time of writing, PREVINTMAX = 20

prv_size

`prv_size <size_number>`

This sets the size of the preview image as displayed in the VGA-sized preview area of pardclient.

`<size_number>` - an integer between 0 and 2 and determines the size of the preview image as follows:

- 0 = 640x480 (VGA - the whole size of the preview area)
- 1 = 320x240 (QVGA)
- 2 = 160x120 (QQVGA)

Note: For preview sizes that are smaller than full VGA, the position of where that preview image is displayed within the full VGA-sized display area of pardclient can be set using the command `preview_tile`.

Note: The size of the preview set here does not affect the coordinate system used with the preview overlay positioning commands `prv_fc_posn` and `prv_hgm_posn`. Those commands will always use coordinates based on the full VGA size. The coordinates will automatically be scaled down at the pardcap end if the preview image is set to be smaller than full VGA.

prv_toggle

`prv_toggle <option> <ON|OFF>`

This sets one of the preview-related options that only accept a binary choice varval argument (and nothing else such as a file name). These options correspond to individual check boxes in the pardcap GUI.

`<option>` - a string which must be one of the following:

`mono`

`invert`

`flip_h`

`flip_v`

`laplacian`

`premedian`

`postmedian`

`dctpostlut`

`fc_dct`

`fc_mal`

`fc_vmul`

`mask_show`

`hgm_show`

`hgm_cum_r`

`hgm_cum_g`

`hgm_cum_b`

`hgm_nofit`

`hgm_restrictvar`

Their meanings are as follows:

`mono` Previewing in monochrome mode

`invert` Invert the preview colours

`flip_h` Flip the preview horizontally

`flip_v` Flip the preview vertically

`laplacian` Display an absolute Laplacian preview.

`premedian` Do a 3x3 median filter before Laplacian calc.s

`postmedian` Do a 3x3 median filter after Laplacian calc.s

`dctpostlut` If using DCT, do the calculation after applying the LUT

`fc_dct` Use medium frequency DCT as a focus assist measure

`fc_mal` Use mean absolute Laplacian as a focus assist measure

`fc_vmul` Multiply the focus measure by the variance.

`mask_show` Show the preview mask

`hgm_show` Overlay the histogram on the preview image

`hgm_cum_r` Use a cumulative histogram for the red (or mono) channel

`hgm_cum_g` Use a cumulative histogram for the green channel

`hgm_cum_b` Use a cumulative histogram for the blue channel

`hgm_nofit` Prevents automatic scaling of each colour channel data in the histogram so you can get a sense of the absolute relationships between the colour channel data.

`hgm_restrictvar` If this is `ON` the mean and variance calculations will only use pixels whose values are greater than the lower saturation limit and less than the upper saturation limit.

`<ON|OFF>` - a binary choice varval

Notes:

1. The options `laplacian`, `premedian` and `postmedian` only have any effect when previewing in monochrome mode.

2. The '`fc_...`' (focusser statistic) options are multiplicative cumulative in effect. If none of these are `ON` then the variance alone will be used as the default focus statistic. If `fc_dct` alone is `ON` then the mean mid-frequency discrete cosine transform power (DCT statistic) will replace variance as the focus statistic. If `fc_mal` alone is `ON` then the mean absolute Laplacian (MAL statistic) will replace variance as the focus statistic. If *both* `fc_dct` *and* `fc_mal` are `ON` then the focus statistic will be the product DCTxMAL. If `fc_vmul` is `ON` then the focus statistic will be whatever it would otherwise have been multiplied by the variance (but `fc_vmul` status has no effect if both `fc_dct` and `fc_mal` are `OFF` - so it won't result in variance `x` variance being used). The 'focus statistic' is the value stored in the system variable Ps_Focus_R, Ps_Focus_G and Ps_Focus_B whenever these are retrieved from the server with the command `get_preview_stats`.

3. The `fc_dct`, when applied, will calculate the DCT statistic on the preview image (optionally under any preview mask if this is used). This calculation may be done before or after the display LUT transforms are applied (according to your current setting for `dctpostlut`). When in monochrome mode if the preview has been transformed into a Laplacian image then the DCT will be calculated on that Laplacian image ONLY if dctpostlut is `ON` and, in that case the DCT will be calculated also after any pre- and/or post-Laplacian median filter is applied. If `dctpostlut` is `OFF` then the DCT will be calculated before the Laplacian image is formed but after any pre-Laplacian median filtering is performed. For colour previews the median filter options are never implemented at any stage and the Laplacian image is never formed (only a Laplacian statistic is calculated when requested - by toggling `fc_mal` to `ON` - but the preview image itself is never transformed into a Laplacian image) so the only options for DCT in colour preview mode is whether to calculate it pre or post application of the LUT transforms. In all cases, monochrome or colour, the other stats for the preview image (including histograms and variance) are calculated before the application of the LUTs. In monochrome mode (only) these stats are calculated after any median filters are applied

and after the image is transformed into a Laplacian image (if that is selected by the user via the Laplacian option).

Scientific Warning: The pixel values in a the mean absolute Laplacian image may have a significantly different distribution to the original pixel values of the image from which they were calculated. Thus, the llim and ulim set for the original image will likely be "wrong" for a Laplacian image. For this reason be sure you set appropriate limits when using that option with a Laplacian-transformed preview.

4. `mask_show` will have no effect unless you previously loaded a preview mask image with the `prv_mask_use` command, which see.

5. For best visual effect, the `hgm_nofit` option should be used with appropriate manual scale factors. See the full pardcap manual for details. See also `prv_hgm_scales`.

6. There are many other preview options not covered by this `prv_toggle` command because those options require something more or other than a binary `ON/OFF` argument. For example the `prv_fc_show` command requires an integer argument from 0 to 4 and not just `ON/OFF`. That is why `prv_fc_show` is a separate command and we don't have an '`fc_show`' option for `prv_toggle`.

return

```
return
```

This closes a function definition code block - that is, a block which is opened with the command `def_func` (which see). You must have strictly one - and only one - `return` paired to each `def_func`.

save_coords

```
save_coords <ON|OFF>
```

This allows you to determine whether each captured image is accompanied by a text file containing the coordinates of all 3 stage axes the time that image was taken. This is a latch setting.

`<ON|OFF>` - a binary choice varval.

Note: The coordinate information will be saved in a plain text file with the same name as the image but with the image file name extension replaced with '_coords.txt'.

Note: The coordinates will always be in atomic step units. It is up to you to make separate arrangements to record any scale factors if you need to convert these to calibrated units at any time.

save_doubles

```
save_doubles <OFF|ON|FITS>
```

This sets the option of whether to save a main image in a 64 bpp doubles format.

`<OFF|ON|FITS>` - a case-sensitive string that means the following:

> `OFF` = don't use the 'save as doubles' pardcap option (this is the default at first startup of a pardcap server)
>
> `ON` = save as raw doubles (.dou and .qih file pair). One such pair for a mono image or 3 such pairs for RGB images, one or each of the RGB channels).
>
> `FITS` = As for `ON` but instead of the raw format (.dou and .qih pairs) it saves the doubles pixel values in one (mono) or 3 (RGB) FITS files.

Note: This command sets the save format modifier options 'save as raw doubles' or 'save as FITS' and so helps determine the save format of the image data coming back from a pardcap server according to the rules described in the section on `save_format` command (which, see).

Note: The .qih file is a plain text external header file. The .dou file contains the actual pixel data.

save_format

```
save_format <format>
```

This allows you to set the file format of an image saved to disk by the `grab_image` command but the format you select here is not the only determining factor of the type of image will ultimately get saved. The actual format and extension of the saved image will also depend on the option you selected with the `save_doubles` command and also whether you have opted to do multiframe averaging with the `frames_to_average` command. The rules of how these options combine to determine the actual file format of the saved image will be detailed below. The actual file format dictated by those rules will determine the file name ending that will be appended to the root file name you supply via the command `save_root`) and the frame number you supply with the command `frame_number` to form the full file name for the saved image.

`<format>` - must be a string constant (not a variable) with one of the following (case sensitive): `yuv`, `pgm`, `bm8`, `int`, `ppm`, `bmp`, `png`, `jpg`

Note: The meanings of the format options are as follows:

> `yuv` - raw binary array of unsigned shorts (16bpp) in YUYV format.
>
> `pgm` - 8bpp p5 greyscale image in `pgm` format
>
> `bm8` - 8bpp greyscale `bmp` image
>
> `int` - the intensity component of an HSI transform of a 24 bpp colour image (saved as raw doubles)
>
> `ppm` - 24bpp colour p6 ppm image
>
> `bmp` - 24bpp colour bmp image

`png` - 24bpp colour png image

`jpg` - 24bpp colour jpeg image

Note: See also `save_doubles` and `jpeg_quality`

Note: The pardcap server will take your above choice and, depending on the following factors, the server will send back image data in one of several formats to be written to disc. The additional factors are: Whether pardcap is using the MJPEG stream of the camera or the raw YUYV stream (at the time of writing, only raw YUYV image streams are used when pardcap is in server mode), your choice of whether to do multi-frame averaging or not via the `frames_to_average` command and your choice of whether to 'save as doubles' or 'save as FITS' as set by the `save_doubles` command. The format of the image data sent back to the pardclient in the blob from pardcap is the 'write as' format and that may be the same as the format option you supplied here (the 'save as' format) or might differ. It is the 'write as' format that actually determines what type of image is saved to disc after a successful `grab_image` command. The rules on how the 'write as' format is determined as described below.

If the 'save as' format going out to the server was `int` then the 'write as' format coming back will be `dou` (raw doubles for single channel images) and the image will be saved with the _i.dou file name ending. All other cases of 'write as' format `dou` will be saved with the _y.dou file name ending. Note that `dou` is used for monochrome images as opposed to `do3` which is the format specifier for 3-channel (colour) images saved as raw doubles or FITS. Note also that the `dou` and `do3` formats are not available as `<format>` options to send to the pardcap server via this `save_format` command. Any attempt to use them in this way will result in an error.

If the 'save as' format going out to the server was `yuv` then the 'write as' format coming back from the server will be `yuv` because multiframe averaging is not permitted for YUYV format captures and the 'save as raw doubles' and 'save as FITS' options are ignored for raw YUYV format image captures and no corrections are applied to the image (masking, flat field / dark field). If the camera was streaming in V4L2_PIX_FMT_MJPEG format we would not have got this far because a request to do image capture in raw YUYV format would have been be rejected at the pardcap end as a forbidden task and an error would have resulted. Files saved in YUYV format will have the name ending '_yuyv.raw' appended to the root.

If the 'save as' format going out to the server was `jpg` then the 'write as' format coming back could be `do3` or `rgb` (used by the server to denote it is sending over a 3-channel colour image) or `jpg` depending on certain options as follows:

- `do3` with 3 raw doubles images in the blob sent back by pardcap: This occurs if the 'save as raw doubles' or 'save as FITS' flags were set (using the PCS command `save_doubles` (which, see)) AND (if averaging was done or the camera was ***not*** streaming in V4L2_PIX_FMT_MJPEG format).
- `rgb` with an RGB image in the blob sent back by pardcap: This occurs if averaging was done or the camera was not streaming in V4L2_PIX_FMT_MJPEG AND neither the 'save as raw doubles' nor 'save as FITS' options were selected. In these cases the RGB image in the blob

will be saved as a jpeg using the interpreter's function raw_to_jpeg with a name having a .jpg extension.

- `jpg` with a jpeg image in the blob: This occurs regardless of the value of the 'save as raw doubles' and 'save as FITS' options. It occurs if averaging was ***not*** done AND the camera ***was*** streaming in V4L2_PIX_FMT_MJPEG format. In these cases the jpeg data in the blob can be directly written to file with a .jpg extension.

Apart from the rules for the case where 'save as' format was `int` (see above), all cases where 'write as' format is `dou` (regardless of what the outgoing 'save as' format was) will be saved with a _y.dou or _y.fit as a FITS file if the 'save as FITS' option is selected (the 'save as FITS' value trumps the 'save as raw doubles' value if both are selected)

All cases where the 'write as' format is `do3`, the file will be saved in 3 files with their names ending in _r.dou, _g.dou and _b.dou or as 3 FITS files with similar endings but having the .fit extension if the 'save as FITS' option is selected (the 'save as FITS' value trumps the 'save as raw doubles' value if both are selected).

The above rules in prose may seem a bit hard to decipher so here is a distilled table summary:

Requested Format (<format>)	**Image Type**	**Actual File Extension(s)**	**Notes / Dependencies**
yuv	Raw 16bpp Binary	_yuyv.raw	Direct capture; ignores averaging/save_doubles/image corrections.
int	HSI Intensity	_i.dou or _i.fit	Depending on save_doubles
pgm / bm8	8bpp Greyscale	_y.pgm / _y.bmp (or _y.dou or _y.fit)	Returns dou or .fit if save_doubles options are set.
ppm / bmp / png	24bpp Colour	_rgb.ppm / _rgb.bmp / _rgb.png (or _r/g/b.dou or _r/g/b.fit)	Returns do3 or .fit if save_doubles options are set.
jpg	24bpp JPEG	.jpg (or _r/g/b.dou or _r/g/b.fit)	Direct save if MJPEG; re-encoded if RGB/Averaged and then save type depends on save_doubles.

save_path

```
save_path <image_full_save_path>
```

This sets the absolute path name (the directory) for saving images and any files associated with them.

`<image_full_save_path>` - a string varval specifying a directory path. It must not contain white space and must not include the file name - only the directory path. This string argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: Any argument longer than SCNFNAME_MAX bytes will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: The `<image_full_save_path>` must contain at least one directory separator (/ or \ depending on your OS) to be valid. The interpreter will automatically use the last separator found in your path to join the directory to the file name. You do not need to end the path with a slash; if you do, the resulting file path will contain a double slash (e.g., path//root_0001.bmp).". While this double slash may be handled correctly by your OS, it is cleaner and better practice to avoid the use of a terminal slash in case some OS or applications have difficulties with double slashes in path names.

save_resolution

`save_resolution <width> <height>`

This allows you to specify the dimensions of any images saved to disk using the `grab_image` or `xyzs_scan` commands (which see).

`<width>` - the image width in pixels supplied as a varval of type int >= 1.

`<height>` - the image height in pixels supplied as a varval of type int >= 1.

Note: The `<width> <height>` combination must be supported by the currently attached camera in order for this command to be valid.

Note: Changing the resolution will invalidate any correction images that may have previously been loaded (such as a corrections mask, a flat field image or a dark field image).

save_root

`save_root <image_name_root>`

This allows you to input the string used to construct the common part of file names generated when you grab an image with `grab_image`.

`<image_name_root>` is a string varval specifying a string without white space which will be used to construct the file name of the saved image and any associated file (like an external header file for saving in raw doubles format). This string argument may be up to SCNFNAME_MAX bytes long (usually 4095 for POSIX or 259 bytes for older MS Windows systems).

Note: If the argument is longer than SCNFNAME_MAX bytes it will be truncated to that limit when read by the interpreter so longer file or path names will likely result in an error when the interpreter attempts to read them.

Note: This may include a path component if you want to store the image in a particular location but the path must not contain any white space. If you do this, avoid the use of a slash at the start of the `<image_name_root>` (that will be handled for you automatically by the interpreter).

Note: The full file name of the saved image will be a combination of the `<image_name_root>` string, an integer number that increments with each image captured (and can be set or reset using the `frame_number` command, which see) and an extension chosen automatically depending on the format of the image to be saved (see the `save_format` command for details). Associated files (like external header files) will have the same basic name but with a modified extension appropriate to their type.

select_camera

`select_camera <number>`

Selects the currently active pardcap server for all camera commands including preview functions.

`<number>` - an integer from 1 (the default at startup) upwards to the maximum number of cameras (pardcap servers) connected to the pardserver.

Note: Any camera or preview command issued in the script only applies to the currently selected camera and will not take effect in any other camera unless you select that camera and then issue the command.

Note: The number of cameras you can have is limited only by your computing resources and from a pure code limit point of view it is capped at just over 2 billion (actually 2147483647, the number of positive values a 32 bit int can store). Because cameras are connected via TCP/IP, the cameras don't all need to be running on the same PC.

Note: When the pardclient first starts a login session the camera list is freed and a call to get_cameras is made which re-populates it. The current camera is then set to 1.

Note: You can see what is the current camera number as well as how many cameras you have available by looking at the 'Camera Settings' tab on the pardclient GUI. This will display a list of the settings for the current camera with a note that this is 'Camera # of #' where the first number is the current camera number and the second is the total number of cameras available.

Note: Calling select_camera from a script does not automatically update the GUI list of available camera settings displayed in the Camera Settings tab but calling select_camera from the GUI 'Manual' tab does. If you want the GUI updated from within a script then make a call to `get_camsettings` and `get_resolutions` after you call `select_camera`.

Note: If you have been doing manual operations on cameras from the GUI Manual tab and then load or run a script in the Script tab, several settings may change for your current selection of

parameters for the camera because when a script is checked (this happens automatically upon loading a script file and also just prior to running it) then the interpreter runs its internal function nullify_correction_images_and_masks() and also does a resize_blob(). Thus, if you return to manual GUI ops after any script work, be prepared to set your camera settings all over again.

Note: When it comes to correction images (flat field, dark field and corrections mask), these are all individual to each camera. You will need to set and load each such image for each camera. This is also true for preview correction images.

set_condexp

`set_condexp <varval1> <condition> <varval2>`

This causes the script to set the value of the system variable `Condexp` to 1 or 0 depending on the result of a conditional expression (i.e. a comparison between two values specified by the arguments). `Condexp` may then be used as a value argument in other commands (including those commands that act on conditional expressions such as `do_if`, `goto`, `call_func` and `loop_while`).

`<varval1>` - a varval of any type.

`<condition>` - a varval of type string which evaluates to a symbol defining the comparison to be made between `<varval1>` and `<varval2>`. The symbol must be one of these: `==`, `>=`, `>`, `<=`, `<`, `!=` and these have the same meanings as the same logical comparator symbols in the C programming language.

`<varval2>` - a varval of any type.

Note: The comparison aspect of a `set_condexp` command is symbolically equivalent to conditional expressions in the C programming language e.g. (varval1 > varval2) or (varval1 <= varval2) or (varval1 == varval2), etc.

Note: If the right hand value `<varval2>` is of a different data type to the left hand value `<varval1>` then the interpreter will attempt to cast the right hand value into the same data type as the left hand value prior to the comparison (this will not affect the intrinsic value of `<varval2>`, it is just a temporary cast for the purpose of making the comparison).

Note: Only `==` and `!=` are allowed between arguments of type string.

Note: If you use the `Condexp` itself as your argument for `<varval1>` or `<varval2>` then the current value of `Condexp` will be used (either 0 or 1 as was set by the last call to `set_condexp` or 0 if this is the first time `set_condexp` is being called in a script). Accordingly the command:

```
set_condexp Condexp == Condexp
```

will always be true and will result in the value of `Condexp` being set to value 1. Similarly the command

```
set_condexp Condexp != Condexp
```

will always be false and will result in the value of `Condexp` being set to value 0. In this way the script writer can manipulate the value of `Condexp` at will.

Note: if you wish to combine multiple comparison expressions with either a logical OR or a logical AND between expressions, this can be effected by the use of multiple variable assignments as shown in the following examples:

```
def_var int exp1 0
def_var int exp2 0
def_var int exp3 0
def_var int i1 5
def_var int i2 -2
def_var int i3 8
set_condexp i1 < i2
set_var exp1 Condexp  # This stores the result of the first conditional
                      # expression into exp1
set_condexp i3 > i2
set_var exp2 Condexp  # This stores the result of the second conditional
                      # expression into exp2
add_var exp3 exp1
add_var exp3 exp2
# With the above setup here are some possibilities for compound
# logical expressions:
set_condexp exp1 > exp2     # If (  exp1  AND !exp2 )
                            # Condexp will equal 1 otherwise 0
                            # (note that '!' denotes NOT in these comments
                            # but you cannot use it like that in PCS)
set_condexp exp1 < exp2     # If ( !exp1  AND  exp2 )
                            # Condexp will equal 1 otherwise 0
set_condexp exp1 == exp2    # If (( exp1  AND  exp2 ) OR ( !exp1 AND !exp2 ))
                            # Condexp will equal 1 otherwise 0
                            # i.e. (exp1  XNOR  exp2)
set_condexp exp1 != exp2    # If (  exp1  XOR  exp2 )
```

```
                            # Condexp will equal 1 otherwise 0
set_condexp exp3 == 1       # If (  exp1  XOR  exp2 )
                            # Condexp will equal 1 otherwise 0
set_condexp exp3 > 0        # If (  exp1   OR  exp2 )
                            # Condexp will equal 1 otherwise 0
set_condexp exp3 == 2       # If (  exp1  AND  exp2 )
                            # Condexp will equal 1 otherwise 0
set_condexp exp3 == 0       # If ( !exp1  AND !exp2 )
                            # Condexp will equal 1 otherwise 0
                            #i.e. (exp1 NOR exp2)
set_condexp exp3 < 2        # If !(exp1 AND exp2)
                            # Condexp will equal 1 otherwise 0
                            # i.e. (exp1 NAND exp2)
```

More complex logical expressions can be made using more variables to combine more pairs of `set_condexp` command results.

set_str

`set_str <STR_VAR> [[str1] [[str2] ...]]`

This allows you to set a user-defined variable of string type to a string of up to 24 words. If no [str] arguments are provided the string variable is set to an empty string. No space will be printed to the end of the list of argument strings.

`<STR_VAR>` - the string variable to be modified

`[[str1] [[str2] ...]]` - a list of between 0 and 24 string constants separated by white space. These will replace the existing contents of `<STR_VAR>`

Note: Up to 24 string arguments may be used in one command. The limitations on what arguments are permitted are as follows:

1. Each string argument in the list of words cannot contain white space and each argument must have no more than MAX_ARGLEN bytes (see Note and Caution below).
2. Each string argument cannot contain the hash character (#) anywhere in it - as soon as a hash character is encountered the word will be truncated to that point and any characters or words after the hash will be ignored. You can add hash characters to a string if you need to by using the add_var command with the Hash system variable or the cat_char command with an argument of value 35.

3. Only one plain space character (ASCII value 32) will be printed between each string argument - multiple spaces are ignored. You can add more spaces to a string if you need to using the separate add_var command with the Space system variable or the add_var command with an argument of value 32.
4. If any of the string arguments are the names of user-defined or system variables they will be added verbatim without conversion to their current value. Use the set_var or add_var commands if you need that functionality.
5. Standard formatting / control / escape characters like '\n' will be printed verbatim and will not have their intended special effect.
6. Recall that PCS does not use delimiters of any kind to define strings or characters to be inserted into strings. If you use single or double quotes as either the whole or part of an argument then those quotes will become part of the string.

Note: Each `[str]` argument will be appended to the `<STR_VAR>` separated by a single space (ASCII character 32) regardless of whatever type or number of white space character(s) was used to separate arguments in the command. No space will be added between the initial value of `<STR_VAR>` and the first `[str]` argument. No space will be printed to the end of the list of argument strings.

Note: Each `[str]` argument will be treated as a string constant and not a variable (of any data type). If a variable name is included in these arguments it will be treated just as a string and will not be evaluated (so the variable's name will be concatenated 'as is'). If you want to add the value contained in some variable to the end of a string then use the `add_var` command instead (which see).

Note: The interpreter has a limit on the maximum size of any argument. This is set by the MAX_ARGLEN value in the interpreter source code which, at the time of writing is 64 bytes. It is important to note that bytes do not necessarily equate to characters. While standard ASCII characters are 1 byte each, UTF-8 Unicode characters (such as box-drawing symbols or emojis) typically consume 3 or 4 bytes each. For example, the ═ symbol uses 3 bytes; therefore, a single argument can contain a maximum of 10 of these characters before exceeding the 64-byte limit.

Caution: Do not exceed MAX_ARGLEN bytes per argument. If a string exceeds this limit, the interpreter will truncate it at the byte level. This may result in **malformed UTF-8 sequences** for Unicode characters, likely causing the string to fail to render. For ASCII text, the string will simply be truncated.

set_var

`set_var <varname> <varval> [charpos]`

This assigns a value to a user-defined variable. This is symbolically equivalent to 'varname = varval'. The `[charpos]` option allows additional operations on string variables.

`<varname>` - a character string constant representing the name of the variable whose value is to be set.

<varval> - a varval and is the value that is to be assigned to the variable specified in <varname>.

[charpos] - a integer varval specifying the position in a string to operate on (starting from 0) or a negative number if you want the string length to be evaluated. See notes for details.

Note: The [charpos] optional argument is ignored and has no effect unless the data type of both <varname> and <varval> are also string.

Note: When <varname> and <varval> are string data then, if [charpos] is supplied and:

- [charpos] is >= 0 : The the set operation will only pertain to the single character in the the <varval> string that is the [charpos]th character in that string (the first character being at position 0) unless the value of [charpos] is >= the number of characters in the string of <varval> in which case no operation is performed (<varname> is left unaltered) but this will not result in an error - the script will silently continue.
- [charpos] is < 0 : The string value of the variable <varname> will be assigned a numerical integer value (in ASCII form, as a string) which is the length of the string of <varval> as returned by the C function strlen.

Note: Thus the optional [charpos] argument allows you to extract a specific single character from the string of <varval> or return the length of that string. It serves no function for for other data types.

skip_frames

`skip_frames <skip>`

This causes the camera frame grabber to activate and discard a certain number of frames. This allows any frames stored in the grabber buffers to be expunged. This clearing of the buffer queue might be useful to ensure a subsequent grab command begins at the most recent thing stationed in front of the camera or to ensure that any change of camera setting (like brightness, gain, colour balance, etc.) is fully enacted in the next image you capture (otherwise if you simply take the next image in the buffer it will be the one that was put there before your setting change was made).

<skip> - a varval of type int >= 0. It tells the frame grabber how many frames to grab and discard.

Note: This is a 'one-off' operation that will happen immediately when you issue the command. This will not change the pardcap internal global variable 'skiplim' which is set using the command `frames_to_skip` (which see).

Note: At the time of writing pardcap uses a buffer queue of 2, so if you set <skip> to 3 this should ensure the buffers are purged and the next image in the buffer queue is up-to-date with any settings or image currently on the camera sensor.

sleep

`sleep <value>`

This pauses execution of the script for the specified amount of time in seconds.

<value> - a varval of type float and >= 0.0

Note: Fractions of a second are respected - there is no need to use whole numbers.

Note: This may be useful to allow any temporary fluctuation in illumination or wobble in the mechanical axes (e.g. due to sudden deceleration), or other process to stabilise before going on to do other things (such as capture an image sequence). It may also be used if you want to pause while PARDUS communicates with another computer (such as an external image processing computer).

sub_var

`sub_var <varname> <varval>`

This subtracts a value from a user-defined variable.

`<varname>` - The name of the variable whose value you want to change

`<varval>` - The value (or variable containing the value) you want to subtract from <varname>.

Note: This is symbolically equivalent to `<varname> -= <varval>` in C.

tc_threshold

`tc_threshold <lower> <upper>`

This sets lower and upper tissue content thresholds. See notes below for more information.

`<lower>` - a varval of type float. It must not be greater than `<upper>`

`<upper>` - a varval of type float. It must not be less than `<lower>` (unless `<upper>` is negative - which is permitted but which also disables the use of this upper limit value as described below).

Note: Tissue content thresholding is an optional test to determine whether a captured image is discarded or saved. Even if an image is discarded due to not passing the tissue content threshold test you may still request a coordinates file by setting the `save_coords` value to `ON` (see `save_coords`).

Note: If the `<lower>` value supplied here as negative then tissue content thresholding will not be used at all and any image capture request (if the capture succeeds without error) will result in an image to be saved. If only the `<upper>` value supplied here is negative (when the `<lower>` is non-negative) then tissue content thresholding will be done but without an upper limit imposed in the thresholding process.

Note: Tissue content threshold tests are applied to the number returned from the tissue content analysis (TCA) algorithm applied to the Y-component of a preview image taken just prior to a full-size frame image capture event (see the command `get_preview_tc` for details of that TCA algorithm). If the threshold test is passed then the full image capture goes ahead. If the test fails then the full image capture does not go ahead.

Note: Unless you call `get_preview_tc OFF` on, at least once in any given login session to a pardcap server after the pardcap server first starts up, the reference values MALref and VARref will not be set. Their default values when a pardcap server first starts are -1.0 (for each of them). If VARref is negative it means the tc reference values have not yet been set and so tc measurements (let alone tc thresholding) will not be possible and so will not be applied regardless of what values you set with this `tc_threshold` command.

Note: The thresholding tests are implemented in the pardcap C source code according to the pseudocode logic shown below (tcval is the result of the TCA algorithm applied to the preview image):

```
if (<lower> < 0.0) go_ahead_with_full_image_capture(); // No thresholding
else {
  int score = 0;
  int passmark = 1;
  grab_a_preview_image_and_calculate_tcval_from_it();

  if (tcval >= <lower>) score++;

  if (<upper> >= 0.0) {
    passmark++;
    if (tcval <= <upper>) score++;
  }

  if (score == passmark) go_ahead_with_full_image_capture();
}
```

tmc_chconf_parse

`tmc_chconf_parse <chconf_info> <receptor>`

Extracts a piece of config information from the current value of the locally stored TMC stepper motor driver chop conf register (CHOPCONF).

`<chconf_info>` - an integer determining what piece of config information to get.

`<receptor>` - a variable which will receive the information.

Note: The received information will be an integer.

Note: This is a purely local command with no traffic to the server or Arduino.

Note: The value of the local copy of CHOPCONF can be updated with the separate `act_get` command using the INFO_TMCCHCONF option.

Note: This parsing command uses a custom PARDUS C function and does not use the TMCStepper library code. It thereby bypasses the commonly known bug in that library for parsing the microstep denominator (TMC_CI_MRESMSDEN) value.

Note: The currently acceptable `<chconf_info>` options are defined in parddefs.h as:

```
// TMC2209 chopconf info item
#define TMC_CI_TOFFLEVEL 1
#define TMC_CI_HYSTSTART 2
#define TMC_CI_HYST__END 3
#define TMC_CI_CPBLANKTM 4
#define TMC_CI_VSENSITIV 5
#define TMC_CI_MRESMSDEN 6
#define TMC_CI_INTERPOLN 7
#define TMC_CI_DOUBLEDGE 8
#define TMC_CI_DISSHRTTG 9
#define TMC_CI_DISSHRTTL 10
```

Note: Information returned will conform to the following ranges:

For TMC_CI_TOFFLEVEL the result will be a number between 0 and 15.

For TMC_CI_HYSTSTART the result will be a number between 1 and 8.

For TMC_CI_HYST__END the result will be a number between -3 and 12.

For TMC_CI_CPBLANKTM the result will be a number between 0 and 3 with the following meaning:

0=16 clocks

1=24 clocks

2=36 clocks

3=54 clocks

For TMC_CI_MRESMSDEN the result will be a number between 1 and 256.

For all other options the result will be 1 (for true) or 0 (for false).

For more details on these data items see the TMC2209 data sheet.

tmc_status_parse

`tmc_status_parse <status_info> <receptor>`

Extracts a piece of status information from the current value of the locally stored TMC stepper motor driver status register (DRV_STATUS).

`<status_info>` - an integer determining what piece of status information to get.

`<receptor>` - a variable which will receive the information.

Note: The received information will be an integer.

Note: This is a purely local command with no traffic to the server or Arduino.

Note: The value of the local copy of DRV_STATUS can be updated with the separate `act_get` command using the INFO_TMCSTATUS option

Note: The currently acceptable `<status_info>` options are defined in parddefs.h as:

```
// TMC2209 status info item
#define TMC_SI_OVER_TEMP 1
#define TMC_SI_PREO_TEMP 2
#define TMC_SI_SHORTGNDA 3
#define TMC_SI_SHORTGNDB 4
#define TMC_SI_SHORTSUPA 5
#define TMC_SI_SHORTSUPB 6
#define TMC_SI_OPENLOADA 7
#define TMC_SI_OPENLOADB 8
#define TMC_SI_CS_ACTUAL 9
#define TMC_SI_TEMP_THRS 10
#define TMC_SI_STNDSTILL 11
#define TMC_SI_SPRDCYCLE 12
```

Note: Information returned will conform to the following ranges:

For TMC_SI_CS_ACTUAL the result will be a number between 0 and 31.

For TMC_SI_TEMP_THRS the result will be a number between 0 and 15.

For all other options the result will be 1 (for true) or 0 (for false).

For more details on these data items see the TMC2209 data sheet.

undef_actuator

`undef_actuator <act_id>`

This undefines (frees) an actuator definition. This clears all data associated with an actuator such that it cannot be used in any way after it is undefined. You may re-define it in order to use it (using `def_actuator`) but it will then be created in the completely nascent state (unhomed, step count invalid, requiring all pins to be re-defined, etc.).

`<act_id>` - This is a string that uniquely identifies the actuator. It may be one of 2 things:

1. The name of an extant actuator
2. The word `CurrAct`

In both cases these are case sensitive and may be supplied either as a string constant or in a string variable.

Internally, the command uses the values of the system variables `CurrActIdx` (actuator index) and `CurrActType` (actuator type value) to identify the actuator to undefine. If you supply `CurrAct` as `<act_id>` then it simply uses whatever values those two system variables happen to have at the time. If you supply the name of an actuator then it attempts to find the index and type of that actuator and will set `CurrActIdx` and `CurrActType` to those values before using them.

The actuator of the specified type and index must exist on the server for this command to succeed, otherwise an error will occur at run time. The script check procedure will not fail if the specific actuator does not exist because it can't predict what actuators will come into existence upon a run-time call to `get_actuators` so it is up to the script writer to ensure these will exist at run time to avoid error. To find out if an actuator exists, do a `get_actuators` command to refresh the local list of extant actuators from the server's data, then do either an `act_getname` or `act_getid` command (which see).

undef_var

`undef_var <varname>`

This destroys a user-defined variable (including an array) and deletes any information it contained.

`<varname>` - a string constant defining the name of the variable or array to be destroyed. This is case sensitive and must pertain to a extant user-defined variable or array (not a system variable).

Note: This command works for arrays of size > 1 as well as single variables. When operating on an array only supply the base name and do not use dereferencing brackets. This is the only occasion, other than with `def_var`, where an array of size > 1 can (and must) be used without dereferencing brackets.

Note: This allows you to use system resources (in particular, RAM) more efficiently by giving variables temporally limited scope. Bear in mind, however, that for the duration in which a variable exists it will always have global scope. For mote details about variable scope rules see p.24 . For example you can create a set of temporary variables to act as 'arguments' to pass values to a function that expects those variables and then undefine those variables at the end of the function just before the function returns. Alternatively you can define a set of variables at the beginning of a function to act as temporary working variables and then undefine them just before you return from the function. However, unlike in other languages, these temporary variables are not 'automatic' or limited in global scope. You can also define a large array for reading in data and then undefine that array when it is no longer needed. It will be entirely up to the PCS programmer to establish and stick to a valid protocol for defining, using and destroying those variables.

Note: The script checker part of the interpreter imposes certain restrictions on the use of variables and their existence. For details see the dedicated section on that topic on page 24 .

Note: In general, problems may occur if you use `def_var`, `def_array` or `undef_var` inside loops or blocks of code that are conditionally accessed (by `do_if`, `else` or any block that is the target of a conditional jump etc.). Such usage is not forbidden and may well pass the syntax checking phase (where conditions are not evaluated and jumps or loops are not enacted) but may results in errors or unforeseen behaviour during run time. Sometimes such placement may be desirable but it is left to the programmer to determine when this is appropriate according to their intended logic. For example, it is almost never a good idea to use these commands inside a `loop_while` loop but it might be acceptable if you need to read large amounts of data into an array from a file and the size of that array might need to change on each iteration of the loop. In such cases using `def_array` for dynamic allocation of the array to the appropriate size followed, after reading and using the data, by `undef_var` before the loop repeats might be a resource efficient way to code this and would be perfectly safe provided the loop can never be exited by a non-local jump that comes between `def_array` and `undef_var`.

Note: By using the `def_array -> undef_var -> def_array` sequence you can effect dynamic re-sizing of an array at run time. See the notes in the description of `def_array` for more detail.

update_gui_coords

`update_gui_coords <server_sync?>`

This updates the client side values and display on the GUI controls of the current XYZ stage coordinates and calibration information.

`<server_sync?>` - is a binary choice varval. If it evaluates to non-zero then the command queries the server to get the latest values of the stage XYZ coordinates, calibration factors, calibration units name and currently used unit type (as set by `act_unit`) and updates the local client side copies of these variables. If this evaluates to zero then the GUI coordinate values are updated with whatever coordinates are currently held on the client.

Note: The reason that making a syncing call to the server is optional is that most 'normal' motion commands that wait for a PARDUS movement report (PMR) will already have the very latest coordinates held locally in the client once the motion has ended and so there is no need to query the server again, only to update the GUI widgets with those values using the pardclient function `update_axis_positions_gui()`; (which this command will ultimately call regardless of the value of `<server_sync?>`). However, there are occasions when these coordinate values might change on the server and Arduino without the latest values coming over to the client (such as after an `act_set` command using the `INFO_STEPCOUNT` info item) in which case using a `<server_sync?>` value of `TRUE` (or similar) will allow full syncing to what is current on the server.

Note: Using `act_get` with an info item of `INFO_STEPCOUNT` or `INFO_XYZCOORDS` will only update script variables with the latest coordinate values but will not change the client local PMRep globals. Only using this `update_gui_coords` command will do that.

Note: This command will not work unless all 3 stage axis stepper motors are defined (X, Y and Z).

verbosity

`verbosity <level>`

This sets how much output to show in the server console, script console and log file.

`<level>` is a varval of type int and can be:

0 = only show and log error messages

1 = as for 0 plus show (but don't log) the results of print_value commands

2 = as for 1 plus log the results of print_value commands

3 = show and log everything.

Note: Having a verbosity less than 3 can be useful, esp. for live preview and other rapid loops.

Note: The default value is 3 and this is reset to 3 before and after every script run.

xyzs_af_period

`xyzs_af_period <period>`

This command sets the period of performing an autofocus attempt during an XY-scan performed by the command `xyzs_scan` (which see).

`<period>` - a varval of type int that must evaluate to >= 0. It is the period (in terms of number of scan moves) at which autofocus is triggered during an XY scan.

Note: If `<period>` is 0 then no autofocus will be performed throughout the whole scan. Otherwise an autofocus procedure will be attempted every `<period>` moves the scan takes. Use a value of 1 to perform autofocus at every step. If tissue content thresholding is on, scan positions will only be

counted towards this period if there is tissue detected in the field, so the actual period will only relate to the number of tissue-containing positions in the scan.

Note: If <period> is >= 1 then the pre-requisite commands for the PCS command autofocus to function must have been called at some point before calling xyzs_scan. See autofocus for details.

xyzs_finish

```
xyzs_finish <coord_x> <coord_y>
```

Set the end-point coordinates of an XY scan performed by the command xyzs_scan (which see).

<coord_x> - a varval of type float and must be non-negative (>= 0.0). It is the X coordinate for the end of the scan.

<coord_y> - a varval of type float and must be non-negative (>= 0.0). It is the Y coordinate for the end of the scan.

Note: All coordinates are expected to be supplied in either calibrated units (if you set act_unit to > 0 for steppers) or in atomic steps (1/maxmsdenom microsteps). Positions are measured from 0 (= the homed position) and no negative values are allowed. Because a stepper motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply here for distances and positions will be rounded to the nearest *onestep* value (see p.54). For example, when using atomic steps, if you set <coord_x> to 15 when maxmsdenom in 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) then this will result in an effective <coord_x> of 0 because (uint32_t)(0.5 + (15.0 / 32.0)) = 0. However if you set <coord_x> = 16 then the effective value will be 1 because (uint32_t)(0.5 + (16.0 / 32.0)) = 1.

xyzs_process

```
xyzs_process <progname> [<arg1> ...]
```

Spawn an external non-blocking command during an XY scan. This is similar to execute_daemon but is run at every step of an automated XY scan run by the command xyzs_scan. You must issue this command at least once before issuing xyzs_scan but it you do not want any processes to run on the scanned images you may use NULL (case sensitive) as your <progrname> argument.

<programe> - a string which is the name of the executable file of the command to pass to the command processor. Alternatively supply NULL if you do not want any process to run on your scanned images.

[<arg1> ...] - an optional list of arguments for the program to be executed. If any of the arguments to be passed is the name of the most recently saved image then this must be specified within the argument list as '[img]' (without the quote marks) and the PARDUS script interpreter will replace that with the name of the last scanned image (this will be the full path name). If tissue content thresholding is active and no image was captured due to lack of tissue content then the

`[img]` symbol will be replaced with `NoTissue` so your external program should check for this specific string (if it is expecting an image file name) to decide what to do.

Note: For more details on the rules see `execute_daemon`.

xyzs_scan

`xyzs_scan`

Begin an XY scan of moves in a snake pattern from a start position to an end position with various options pre-set using other commands. This has no arguments but for this command to work without an error certain pre-requisite commands must have been successfully issued before calling it. These commands are listed below but see their individual entries in the manual for detail:

- All the commands required for `grab_image` to work (see `grab_image` for the list).
- `xyzs_start` - sets the (x,y,z) start position of the scan.
- `xyzs_finish` - sets the (x,y) end position of the scan.
- `xyzs_process` - sets what external process to run on each image of the scan.
- `xyzs_stepsize` - sets the width and height and depth of each move in the scan pattern.
- `xyzs_af_period` - sets how often to perform autofocus during the scan. If this is set to anything more than 0 then you must also have called the pre-conditional commands required for `autofocus` to work (see the section on `autofocus` for the list).

Note: For a scan to work a pardcap server must be connected with a valid camera available and all three stage axes (X, Y and Z) must be extant, mature and homed.

Note: A text file will be opened at the start of the scan to record scan parameters and also to record the positions of each image (stage coordinates) and whether they had tissue or not (if tissue thresholding is used). The name of this file will be constructed from your image capture path and root file name settings and have the name ending in "_xys.xys". This text file output must succeed for the scan to complete successfully - any file IO errors will terminate the scan. This file is also used to load a scan into the GUI for the pardclient scan view feature.

Note: At the end of a scan, the preview image will be attempted to be saved to disc with a file name beginning with your image capture path and root file name settings and have the name ending in "_preview.ppm" (it is saved in p6 ppm format).

xyzs_semino_z

`xyzs_semino_z <slices>`

Set the number of focus plane slices in a scan performed by the command `xyzs_scan` (which see).

<slices> - a varval of type int and must be non-negative (>= 0). It is the number of slices to be captured at each XY position of the scan both below and above the current Z position (hence 'semi').

Note: For a scan involving Z slices (i.e. if <slices> is > 0) then you must also have previously called the commands af_setup successfully even if you have no intention of using autofocus in your scan. The reason for this is that a call to af_setup ensures that the Z motor motion is well characterised and, in particular, the client ensures it knows which direction is 'up' and which is 'down' because these are needed to perform Z stacks.

Note: The way this works is that, at each XY scan position, if xyzs_semino_z is >0, the Z motor will move to the current 'best focus' Z position (if it is not there already) and then move a total distance of (<slices> x <size_z>) distance units in Z in the 'down' direction (where <size_z> is the value supplied in the command xyzs_stepsize). It then takes a picture and moves <size_z> up. If <slices> is > 1 this 'picture-then-move-up' process is repeated for a total of <slices> times. At the end of this first phase the Z stage will be at the position it started from. The process is continued for another <slices> times at which point the Z stage returns to its original starting point and the XY scan process continues. Thus at each XY position there will be acquired a Z-stack of a total of <slices>+1 images.

xyzs_start

`xyzs_start <coord_x> <coord_y> <coord_z>`

Set the start point coordinates for an XY scan performed by the command xyzs_scan (which see).

<coord_x> - a varval of type float and must be non-negative (>= 0.0). It is the X coordinate for the end of the scan.

<coord_y> - a varval of type float and must be non-negative (>= 0.0). It is the Y coordinate for the end of the scan.

<coord_z> - a varval of type float and must be non-negative (>= 0.0). It is the Z coordinate for the end of the scan.

Note: All coordinates are expected to be supplied in either calibrated units (if you set act_unit to > 0 for steppers) or in atomic steps (1/maxmsdenom microsteps). Positions are measured from 0 (= the homed position) and no negative values are allowed. Because a stepper motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply here for distances and positions will be rounded to the nearest *onestep* value (see p.54). For example, when using atomic steps, if you set <coord_x> to 15 when maxmsdenom in 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) then this will result in an effective <coord_x> of 0 because (uint32_t)(0.5 + (15.0 / 32.0)) = 0. However if you set <coord_x> = 16 then the effective value will be 1 because (uint32_t)(0.5 + (16.0 / 32.0)) = 1.

xyzs_stepsize

`xyzs_stepsize <size_x> <size_y> <size_z>`

Set the width and height for each move in an XY scan performed by the command `xyzs_scan` (which see).

`<size_x>` - a varval of type float and must be non-negative (>= 1.0e-3). It is the search step distance for each X motion in the scan.

`<size_y>` - a varval of type float and must be non-negative (>= 1.0e-3). It is the search step distance for each Y motion in the scan.

`<size_z>` - a varval of type float and must be non-negative (>= 1.0e-3). It is the search step distance for each Z motion in the scan.

Note: The term 'step' in `xyzs_stepsize` refers to each step in the scan pattern, not the microstep size of the motors. If you set the `<size_x>` and `<size_y>` values so that the stage moves exactly one camera frame width in X and one camera frame height in Y for each scan move then your XY scan images will not overlap. If you set them to smaller values then your scan images will have overlap at the boundaries of each image (which may be useful for 'stitching' the images together into a larger image after the scan is complete.

Note: The value you set for all of these arguments must be >= 1.0e-3 to satisfy the syntax checker even if you do not intend to scan along a particular axis. For example, the value you supply for `<size_z>` will never be used if you are not doing any Z slices above and below the current Z position (see the command `xyzs_semino_z`).

Note: All step size distances are expected to be supplied in either calibrated units (if you set `act_unit` to >0 for steppers) or in atomic steps (1/maxmsdenom microsteps). Positions are measured from 0 (= the homed position) and no negative values are allowed. Because a stepper motor can only travel in steps of a minimum of 1/msdenom microsteps, this means that whatever values you supply here for distances and positions will be rounded to the nearest *onestep* value (see p.54). For example, when using atomic steps, if you set `<size_x>` to 15 when maxmsdenom in 256 and msdenom is 8 (giving a onestep value of 256/8 = 32) then this will result in an effective `<size_x>` of 0 because (uint32_t)(0.5 + (15.0 / 32.0)) = 0. However if you set `<size_x>` = 16 then the effective value will be 1 because (uint32_t)(0.5 + (16.0 / 32.0)) = 1.

yuyv_bias

`yuyv_bias <value>`

This sets the value of the bias constant used in the YUYV-to-RGB conversion that occurs for colour images captured with a camera streaming raw YUYV data.

`<value>` is a varval of type float.

Note: There is no restriction on the value. By default the value is 0.0.

yuyv_gain

`yuyv_gain <value>`

This sets the value of the gain coefficient used in the YUYV-to-RGB conversion that occurs for colour images captured with a camera streaming raw YUYV data.

`<value>` is a varval of type float.

Note: There is no restriction on the value. By default the value is 1.0.